DOCUMENTAÇÃO TÉCNICA OFICIAL

UpperZetta

Linguagem · Compilador · Runtime · IDE

Versão **1.0** · Runtime **UVLM** · Extensão **`.uz`**

Bytecode GLP · AES-256 · ZettaSource IDE

GUIA COMPLETO PARA DESENVOLVEDORES

Sintaxe · Semântica · Bytecode · Ferramentas · Boas Práticas

25 de abril de 2026

Índice

PARTE I · FUNDAMENTOS

01 Visão Geral
02 Instalação e Configuração
03 Arquivos e Extensões
04 Estrutura de um Programa
05 Comentários e Estilo de Código
06 Tipos de Dados
07 Variáveis e Constantes
08 Operadores

PARTE II · CONTROLE E ABSTRAÇÃO

09 Controle de Fluxo
10 Funções
11 Classes
12 Componentes
13 Arrays
14 Saída e Depuração
15 Sistema de Pacotes

PARTE III · COMPILADOR E RUNTIME

16 Modelo de Compilação
17 Bytecode UVLM
18 Formato GLP
19 Selar e Desselar Arquivos (.uzs)
20 DEVICE KEY e Migração entre Máquinas

PARTE IV · FERRAMENTAS

21 CLI Reference
22 ZettaSource IDE

PARTE V · PRÁTICA E REFERÊNCIA

23 Convenções e Style Guide
24 Padrões e Anti-padrões
25 Performance e Otimização

26 Limitações Conhecidas
27 Erros Comuns e Troubleshooting
28 Códigos de Erro
29 Migrando de Outras Linguagens
30 Perguntas Frequentes (FAQ)
31 Glossário
32 Referência Rápida
33 Exemplos Completos

PARTE VI · APÊNDICES

A Tutorial Completo: Sistema de Biblioteca
B Cookbook: Receitas Comuns
C Padrões de Design Adaptados
D Estruturas de Dados Implementadas
E Algoritmos Clássicos
F Receitas Matemáticas
G Estratégias de Teste
H Deploy e Distribuição
I Comparações com Outras Linguagens
J Roadmap e Histórico de Versões
K Tipos em Profundidade
L Organização de Projetos

PARTE I

Fundamentos

Visão geral, instalação, sintaxe básica e tipos

1. Visão Geral

UpperZetta (frequentemente abreviada como **UZ** ou **Uzet**) é uma linguagem de programação compilada e tipada estaticamente, executada sobre a **UpperZetta Virtual Language Machine (UVLM)** — uma máquina virtual de pilha que recebe bytecode previamente gerado a partir do código-fonte `.uz`.

A linguagem foi projetada com três objetivos principais: **legibilidade explícita** (anotações de tipo obrigatórias e visíveis), **segurança via tipagem estática** (erros são capturados em tempo de compilação) e **proteção de propriedade intelectual** (o formato `.uzs` permite distribuir aplicações com código-fonte criptografado por senha e atrelado ao dispositivo).

1.1 Características Principais

- **Tipagem estática:** toda variável, parâmetro e retorno declara seu tipo de forma explícita usando a notação `nome.tipo`. Não há inferência automática.
- **Orientação a objetos:** suporte a classes (`class.public`) e a um conceito próprio de componente (`componente`), que serve simultaneamente como ponto de entrada e contêiner de estado.
- **Compilação para bytecode:** o código-fonte `.uz` é convertido para o formato binário `.uzb`, executado pela UVLM.
- **Bytecode GLP:** o formato binário usa o esquema *Generative Logic Palindrome*, que armazena código e seu espelho na mesma estrutura, permitindo verificação de integridade sem checksum externo.
- **Criptografia integrada:** o utilitário `seal` produz arquivos `.uzs` criptografados com AES-256, opcionalmente atrelados a uma chave única do dispositivo (DEVICE KEY).
- **IDE oficial:** a ZettaSource oferece editor com syntax highlighting, compilação one-click, painel de saída UVLM e suporte nativo aos formatos selados.

1.2 Filosofia de Design

UpperZetta evita "açúcar sintático" implícito: tipos não são inferidos, conversões automáticas são limitadas a contextos de concatenação de strings, e conceitos têm uma forma canônica única (não há quatro maneiras de declarar uma variável). O objetivo é que código UZ seja imediatamente legível por outro desenvolvedor, sem necessidade de consultar regras de inferência ou contexto externo.

A separação entre class.public e componente é deliberada: classes representam dados/comportamento reutilizáveis, enquanto componentes representam unidades de execução com estado. Isso espelha o modelo conceitual de aplicações modernas, onde a UI/lógica principal vive em componentes e os modelos de domínio em classes.

1.3 Quando Usar UpperZetta

UpperZetta é especialmente adequada para:

- Aplicações desktop ou utilitários onde proteção do código distribuído é importante (via .uzs).
- Projetos didáticos: a sintaxe explícita e a ausência de "magia" facilitam o ensino de tipagem e estruturas de dados.
- Ferramentas internas que precisam ser distribuídas como bytecode autocontido (.uzb), sem dependências externas além da UVLM.
- Projetos onde a separação clara entre modelo de dados (classes) e ponto de entrada (componente) reflete bem a arquitetura desejada.

1.4 Comparação Rápida com Outras Linguagens

Aspecto	UpperZetta	Java	Python
Tipagem	Estática, explícita	Estática, explícita	Dinâmica
Anotação de tipo	nome.tipo	Tipo nome	nome: tipo (opcional)
Compilação	.uz → .uzb	.java → .class	Interpretada / .pyc
Runtime	UVLM (stack VM)	JVM (stack VM)	CPython
Entry point	componente Home.render()	main(String[] args)	if __name__ == "__main__"
Orientação a objetos	class.public + componente	class	class
Proteção de fonte	.uzs (AES-256 + DEVICE KEY)	Obfuscação manual	Não nativa

1.5 Modelo de Execução em Alto Nível

Um programa UpperZetta passa por três fases distintas:

1. **Compilação:** o compilador lê o arquivo `.uz`, faz análise lexical, sintática e gera bytecode UVLM no formato GLP, salvando em `.uzb`.
2. **Carregamento:** a UVLM lê o arquivo `.uzb`, valida a estrutura palindrômica do GLP e prepara as estruturas internas (constant pool, classes, funções).
3. **Execução:** a UVLM localiza o componente `Home`, invoca o método `render()` como ponto de entrada e executa as instruções na pilha, retornando ao sistema operacional ao final.

> **NOTA**
> Esta documentação cobre UpperZetta v1.0 sobre UVLM Runtime. Versões futuras podem introduzir novas construções, mas a sintaxe descrita aqui é estável e mantém compatibilidade retroativa garantida dentro da linha 1.x.

2. Instalação e Configuração

UpperZetta é distribuída como um único arquivo Java (Main.jar) que contém compilador, máquina virtual, utilitários CLI e bibliotecas internas. Não há dependências externas além de uma JVM compatível.

2.1 Requisitos do Sistema

Requisito	Versão Mínima	Recomendado
Java Runtime (JRE)	Java 11	Java 17 ou superior
Memória RAM	512 MB	2 GB
Espaço em disco	50 MB	200 MB (com IDE)
Sistema operacional	Windows / Linux / macOS	Qualquer com JVM

2.2 Verificando o Java

Antes de instalar UpperZetta, confirme que o Java está disponível no PATH:

```shell
java -version
# saída esperada (exemplo):
# openjdk version "17.0.8" 2023-07-18
# OpenJDK Runtime Environment ...
```

> **ATENÇÃO**
> Se "java" não for reconhecido como comando, instale o JRE/JDK pelo site oficial da Oracle, OpenJDK, Adoptium ou via gerenciador de pacotes do seu sistema (apt, brew, choco) antes de prosseguir.

2.3 Estrutura de Diretórios Recomendada

Uma instalação padrão em ambientes Unix/macOS organiza os arquivos da seguinte forma:

```
~/upperzetta/
├── Main.jar              # compilador + UVLM + CLI
├── docs/                 # documentação local
│   └── reference.md
├── examples/             # exemplos prontos
│   ├── hello.uz
│   └── calc.uz
└── ~/.uvlm/              # criado na primeira execução
    └── .uvlm_dk          # DEVICE KEY desta máquina
```

2.4 Configurando um Alias

Para evitar digitar o caminho completo a cada compilação, configure um alias de shell. No `~/.bashrc` ou `~/.zshrc`:

~/.zshrc

```
alias uz="java -cp ~/upperzetta/Main.jar Main"
```

Após recarregar o shell, todos os comandos da CLI ficam disponíveis com o prefixo curto:

shell

```
uz programa.uz           # compilar e executar
uz programa.uzb          # executar bytecode
uz seal programa.uz      # selar
```

> **DICA**
> No Windows, crie um arquivo uz.bat em uma pasta listada no PATH com o conteúdo "@java -cp C:\\upperzetta\\Main.jar Main %*" para obter o mesmo efeito.

2.5 Primeira Execução

A primeira invocação da UVLM cria automaticamente o diretório ~/.uvlm/ e gera uma DEVICE KEY única para o computador. Esta chave é usada no esquema UZS1 de criptografia (ver Seção 20).

shell

```
echo 'componente Home { fun render() { System.print("ok"); } }' > teste.uz
```

```
uz teste.uz
# Saída: ok
```

2.6 Verificando a Versão Instalada

```shell
uz --version
# UpperZetta 1.0 · UVLM Runtime
# Build GLP-encoder v1
```

3. Arquivos e Extensões

O ecossistema UpperZetta opera com três extensões de arquivo bem definidas, cada uma representando um estado específico do código no ciclo de desenvolvimento e distribuição.

3.1 Tabela de Extensões

Extensão	Propósito	Legível?
.uz	Código-fonte UpperZetta (texto puro UTF-8)	Sim
.uzb	Bytecode compilado pela UVLM (formato GLP binário)	Não (binário)
.uzs	Arquivo selado: .uz ou .uzb criptografado com senha	Não (cifrado)

3.2 Fluxo de Transformação

Cada extensão pode ser convertida nas demais por meio dos utilitários da CLI:

```
                compilar              executar
   main.uz    ──────────►  main.uzb  ──────────►  saída no terminal
      │                       │
      │ seal                  │ seal
      ▼                       ▼
   main.uzs                main.uzs
      │                       │
      │ unseal                │ unseal
      ▼                       ▼
   main.uz                 main.uzb
```

3.3 Quando Usar Cada Formato

3.3.1 Use .uz quando...

- Estiver desenvolvendo ativamente o código (ele será editado).
- Compartilhando código aberto, snippets ou exemplos didáticos.
- Submetendo trabalhos acadêmicos onde o avaliador precisa ler o fonte.

3.3.2 Use .uzb quando...

- Distribuindo a aplicação para usuários finais que apenas precisam executá-la.
- Testando se o programa compila com sucesso, separadamente da execução.
- Empacotando a aplicação para entrega rápida (.uzb evita recompilação a cada execução).

3.3.3 Use .uzs quando...

- Distribuindo software comercial onde o código-fonte é propriedade intelectual sensível.
- Enviando código a um cliente sob acordo de confidencialidade que será aberto com senha compartilhada.
- Arquivando versões legadas do código com proteção contra leitura por terceiros.

3.4 Organização Recomendada de Projeto

Para projetos com múltiplos arquivos, uma organização clara facilita manutenção e build:

```
meu-projeto/
├── src/
│   ├── main.uz              # ponto de entrada (componente Home)
│   ├── modelos/
│   │   ├── usuario.uz       # class.public Usuario
│   │   └── produto.uz       # class.public Produto
│   └── utilitarios/
│       └── strings.uz       # funções globais auxiliares
├── build/
│   └── main.uzb             # bytecode gerado (não versionar)
├── dist/
│   └── main.uzs             # versão selada para distribuição
├── exemplos/
│   └── exemplo.uz
└── README.md
```

> **NOTA**
> O compilador atual processa um arquivo .uz por vez. Projetos multi-arquivo requerem que todas as definições estejam no mesmo arquivo .uz no momento da compilação.

4. Estrutura de um Programa

Todo programa UpperZetta segue uma estrutura previsível: pacote no topo, seguido por declarações globais, classes e finalmente o componente que serve como ponto de entrada. Esta ordem não é apenas convenção — o compilador a espera para resolver corretamente referências entre construções.

4.1 Programa Mínimo

O programa mais curto possível em UpperZetta tem três elementos: declaração de pacote (opcional, mas recomendada), o componente Home e o método render():

main.uz

```
package in meu.pacote;

componente Home {
    fun render() {
        System.print("Olá, UpperZetta!");
    }
}
```

4.2 Ordem das Declarações

Em arquivos com múltiplas construções, a ordem canônica é a seguinte. Cada elemento é opcional, exceto o componente Home (sem ele, o programa não tem ponto de entrada):

4. **Declaração de pacote** — `package in nome.do.pacote;` (uma única vez, primeira instrução do arquivo).
5. **Constantes globais** — declaradas com `e.const`, visíveis em todo o arquivo.
6. **Funções globais** — declaradas com `fun`, podem ser chamadas por qualquer componente ou método.
7. **Classes** — declaradas com `class.public`, usadas como modelos de dados.
8. **Componentes** — declarados com `componente`, um deles deve ser `Home`.

4.3 Programa Completo Anotado

aplicacao.uz

```
// 1. Declaração de pacote
package in app.exemplo;

// 2. Constantes globais
e.const VERSAO.str = "1.0.0";
```

```
e.const MAX_ITENS.int = 100;

// 3. Funções globais
fun saudar(nome.str) >> str {
    return "Olá, " + nome + "!";
}

// 4. Classes (modelos de dados)
class.public Item {
    let nome.str;
    let preco.float;

    fun rotulo() >> str {
        return nome + " (R$ " + preco + ")";
    }
}

// 5. Componente — ponto de entrada
componente Home {
    let titulo.str = "Aplicação Exemplo";

    fun render() {
        System.print(titulo + " v" + VERSAO);
        System.print(saudar("Mundo"));

        let i.Item = new Item();
        i.nome = "Café";
        i.preco = 12.50;
        System.print(i.rotulo());
    }
}
```

4.4 Por que componente Home?

A UVLM, ao carregar o bytecode, procura especificamente por um componente chamado `Home` e invoca seu método `render()` como ponto de entrada. Esta é uma decisão de design fixa: não é configurável.

A escolha do nome "Home" e do método "render" reflete a inspiração da linguagem em arquiteturas baseadas em componentes (como vistas em frameworks de UI modernos), mesmo que UpperZetta não seja primariamente uma linguagem para interfaces gráficas.

> **ATENÇÃO**
> Se o arquivo não contiver um componente chamado Home, ou se Home não tiver um método render(), a UVLM aborta a execução com um erro UZ-1001.

4.5 Múltiplos Componentes no Mesmo Arquivo

É permitido declarar vários componentes no mesmo arquivo, mas apenas Home é executado. Os demais ficam disponíveis como referências, embora a versão atual do compilador não permita instanciá-los como objetos. Esta é uma limitação a se ter em mente quando se planeja arquiteturas mais complexas:

```
componente Sidebar {
    fun render() { System.print("sidebar"); }
}

componente Home {
    fun render() {
        // não há como invocar Sidebar.render() a partir daqui
        System.print("apenas Home executa");
    }
}
```

> **NOTA**
> Se você precisa de múltiplas unidades de comportamento, organize-as como classes (class.public) ou funções globais. Componentes adicionais são, na prática atual, código morto.

5. Comentários e Estilo de Código

5.1 Comentários de Linha

Comentários de linha começam com duas barras (//) e continuam até o final da linha. São ignorados pelo compilador e servem exclusivamente para documentação humana:

```
// Calcula o valor com desconto
let preco.float = 100.0;
preco = preco * 0.9;   // aplica 10% de desconto
```

5.2 Comentários de Bloco

Comentários de bloco delimitam-se por /* e */ e podem ocupar várias linhas. Úteis para documentação extensa, comentários de cabeçalho e desativação temporária de blocos de código:

```
/*
 * Função: calcularJuros
```

```
 * Parâmetros: principal (float), taxa (float), meses (int)
 * Retorno: valor final com juros compostos aplicados
 */
fun calcularJuros(principal.float, taxa.float, meses.int) >> float {
    let resultado.float = principal;
    for (let i.int = 0; i < meses; i = i + 1) {
        resultado = resultado * (1.0 + taxa);
    }
    return resultado;
}
```

ATENÇÃO
Comentários de bloco não aninhados: /* /* */ */ é interpretado como um comentário fechando no primeiro */, deixando o segundo */ como erro de sintaxe.

5.3 Convenções Recomendadas

5.3.1 Indentação

- Use 4 espaços para cada nível de indentação. Tabs são aceitas, mas a comunidade UpperZetta padronizou espaços para garantir consistência visual em diferentes editores.
- Indente o conteúdo de blocos {} um nível à direita do delimitador.
- Continue indentação consistente em estruturas aninhadas.

5.3.2 Nomenclatura

Construção	Convenção	Exemplo
Variáveis e parâmetros	camelCase	idadeUsuario, precoTotal
Constantes	UPPER_SNAKE	MAX_TENTATIVAS, PI
Funções	camelCase	calcularMedia, ehAdulto
Classes	PascalCase	Usuario, ContaCorrente
Componentes	PascalCase	Home, Dashboard
Pacotes	lowercase.dot	app.util, meu.projeto

5.3.3 Espaçamento

- Espaço único após vírgulas em listas de argumentos.
- Espaço em torno de operadores binários (a + b, não a+b).
- Sem espaço após (ou antes de).
- Linha em branco entre funções e métodos para legibilidade.

5.3.4 Comentários Úteis vs. Ruído

Bons comentários explicam o "porquê", não o "o quê":

```
// ❌ Ruim: descreve o óbvio

let i.int = 0;
i = i + 1;   // incrementa i em 1
```

```
// ☑ Bom: explica a intenção

// Reseta o contador entre usuários para evitar
// que o cache de sessão anterior afete o atual
let i.int = 0;
```

5.4 Caracteres Permitidos em Identificadores

Identificadores (nomes de variáveis, funções, classes) seguem regras tradicionais de linguagens de programação:

- Devem começar com letra (a-z, A-Z) ou underscore (_).
- Podem conter letras, dígitos (0-9) e underscores nas posições subsequentes.
- São case-sensitive: usuario e Usuario são identificadores distintos.
- Não podem coincidir com palavras-chave reservadas (ver Seção 32.1).

> **ATENÇÃO**
> Caracteres acentuados (á, é, ç, ã) NÃO são permitidos em identificadores. Use apenas ASCII básico — ainda que strings literais aceitem qualquer caractere UTF-8, identificadores não.

6. Tipos de Dados

UpperZetta possui um conjunto pequeno e bem-definido de tipos primitivos, complementado por arrays e referências a objetos definidos pelo usuário. Toda variável tem tipo declarado e o compilador valida compatibilidade em todas as operações.

6.1 Tabela de Tipos Primitivos

Tipo	Descrição	Faixa / Exemplo
int	Número inteiro com sinal de 32 bits	-2.147.483.648 a 2.147.483.647

Tipo	Descrição	Faixa / Exemplo
float	Número de ponto flutuante (IEEE 754, 64 bits)	$\pm 5.0 \times 10^{-324}$ a $\pm 1.7 \times 10^{308}$
str	Cadeia de caracteres UTF-8	"Olá, mundo!"
boolean	Valor lógico binário	true ou false
array	Vetor heterogêneo de valores	[1, "dois", true]
void	Ausência de retorno (apenas em funções)	— (não há literal)

6.2 int — Inteiros

O tipo int representa números inteiros com sinal armazenados em 32 bits, seguindo o padrão de complemento de dois. Operações que excedam a faixa causam overflow silencioso (não há exceção).

exemplos de int

```
let idade.int = 21;
let temperatura.int = -5;
let zero.int = 0;
let max.int = 2147483647;       // limite máximo
let semInicializacao.int;       // valor padrão é 0
```

ATENÇÃO
Atenção ao overflow: max + 1 não causa erro, mas resulta em -2147483648. Para valores muito grandes, considere se float é mais adequado (apesar da perda de precisão exata).

6.3 float — Ponto Flutuante

O tipo float usa precisão dupla IEEE 754. É adequado para cálculos científicos, financeiros (com cuidado), físicos e qualquer operação que envolva frações:

exemplos de float

```
let pi.float = 3.14159;
let preco.float = 9.99;
let negativo.float = -0.5;
let cientifico.float = 8.5;       // sempre use ponto, nunca vírgula
let inteiroComoFloat.float = 5.0; // explícito é mais claro
```

ATENÇÃO
Comparar floats com == é perigoso devido a erros de arredondamento. Em vez de "if (x == 0.1)", prefira verificar se a diferença está dentro de uma tolerância pequena: "if (x > 0.099 && x < 0.101)".

6.4 str — Strings

Strings são sequências imutáveis de caracteres delimitadas por aspas duplas. UpperZetta não aceita aspas simples para strings:

```
exemplos de str

let nome.str = "Maria";
let mensagem.str = "Olá, mundo!";
let vazia.str = "";
let comAcento.str = "São Paulo";
let comNumeros.str = "Rua 25 de Março";
```

PERIGO
Use SEMPRE aspas duplas. 'Maria' (aspas simples) causa erro de sintaxe.

6.5 boolean — Valores Lógicos

Apenas dois valores válidos: true e false. Diferente de outras linguagens, UpperZetta não tem "valores verdadeiros" implícitos (não-zero não é true, string vazia não é false):

```
exemplos de boolean

let ativo.boolean = true;
let admin.boolean = false;
let resultado.boolean = (10 > 5);   // true

// ❌ ERRADO: int não converte automaticamente para boolean
// if (1) { ... }   // erro de tipo

// ☑ CORRETO: comparação explícita
let n.int = 1;
if (n != 0) { System.print("não-zero"); }
```

6.6 array — Vetores

Arrays são heterogêneos: podem misturar tipos no mesmo vetor. Acessam-se elementos com colchetes e índice baseado em zero:

```
exemplos de array
```

```
let numeros.array = [1, 2, 3, 4, 5];
let nomes.array = ["Ana", "Bruno", "Carlos"];
let misto.array = [1, "dois", true, 3.14];
let vazio.array = [];

let primeiro.int = numeros[0];        // 1
let ultimo.str = nomes[2];            // "Carlos"
```

> **NOTA**
> Embora arrays aceitem tipos diferentes, é boa prática manter homogeneidade dentro de um array sempre que possível. Arrays heterogêneos dificultam iteração (você não sabe que tipo arr[i] retornará).

6.7 void — Ausência de Retorno

void não é um tipo de valor — você não pode declarar "let x.void". Ele aparece exclusivamente como tipo de retorno implícito de funções que não retornam nada:

void implícito

```
fun imprimir(msg.str) {
    System.print(msg);
    // sem "return" — implicitamente retorna void
}
```

6.8 Conversões entre Tipos

UpperZetta não realiza conversões implícitas exceto em um caso: o operador + com pelo menos um operando string converte o outro lado para string automaticamente.

Operação	Resultado
"valor: " + 42	"valor: 42" (str)
"x = " + 3.14	"x = 3.14" (str)
"flag: " + true	"flag: true" (str)
10 + "px"	"10px" (str)
10 + 5	15 (int)
10 + 5.0	erro de tipo
true + 1	erro de tipo

DICA
Para "converter" um número para string sem usar +, basta concatenar com string vazia: "" + 42 produz "42". Para o caminho inverso (string para número), a versão atual da linguagem não oferece função nativa de parsing.

7. Variáveis e Constantes

7.1 Variáveis com let

Variáveis são declaradas com a palavra-chave `let`, seguida do nome, ponto, tipo e (opcionalmente) inicialização. A sintaxe canônica é:

```
let nomeDaVariavel.tipo;
let nomeDaVariavel.tipo = valorInicial;
```

Exemplos cobrindo todos os tipos primitivos:

```
let idade.int;                          // declarada, sem valor (padrão: 0)
let nome.str = "João";                  // declarada e inicializada
let ativo.boolean = true;
let preco.float = 9.99;
let numeros.array = [10, 20, 30];
let usuario.Usuario = new Usuario();    // tipo é uma classe
```

7.1.1 Por que ponto e não dois-pontos?

A escolha de `nome.tipo` (com ponto) em vez de `nome: tipo` (com dois-pontos, como em TypeScript ou Python) é deliberada. UpperZetta reserva `:` para uso futuro em rotulagem de blocos e mantém `.` como o operador unificado de "atributo de" — seja para tipos, seja para acesso a campos.

7.1.2 Valores Padrão

Quando uma variável é declarada sem inicialização, recebe o valor padrão de seu tipo:

Tipo	Valor padrão
int	0
float	0.0
str	"" (string vazia)
boolean	false

Tipo	Valor padrão
array	[] (array vazio)
Classe	null (referência nula)

7.2 Reatribuição

Variáveis podem ser reatribuídas livremente, sem repetir let, contanto que o novo valor seja compatível com o tipo declarado:

```
let contador.int = 0;
contador = contador + 1;       // OK: int = int
contador = 100;                // OK
// contador = "texto";         // ERRO: tipo incompatível
```

7.3 Constantes com e.const

Constantes são declaradas com `e.const`, exigem inicialização imediata e **só podem aparecer no escopo global** (fora de funções, classes e componentes).

```
e.const PI.float = 3.14159;
e.const APP_NAME.str = "MinhaApp";
e.const MAX_TENTATIVAS.int = 3;
e.const VERSAO.str = "1.0.0";
```

7.3.1 Por que constantes só no escopo global?

Esta restrição existe por dois motivos: (1) constantes representam valores que não mudam ao longo da execução do programa inteiro — ter "constantes locais" seria semanticamente equivalente a let; (2) o compilador armazena constantes na constant pool do bytecode, e isso é resolvido em tempo de compilação, não em tempo de execução.

> **PERIGO**
> Tentar declarar e.const dentro de uma função ou método produz erro UZ-2105: "e.const só permitido no escopo global".

7.4 Escopo de Variáveis

UpperZetta tem três escopos distintos, hierarquicamente organizados:

7.4.1 Escopo Global

- Constantes (e.const) e funções (fun) declaradas no nível mais alto do arquivo.
- Visíveis em todo o arquivo, dentro de qualquer classe, componente ou função.

7.4.2 Escopo de Classe / Componente

- Campos declarados com let dentro de class.public ou componente.
- Acessíveis pelos métodos da própria classe (sem necessidade de "this").

7.4.3 Escopo de Função / Método

- Variáveis locais declaradas com let dentro do corpo de uma função ou método.
- Existem apenas durante a execução da função.
- Sombreiam (ocultam) variáveis globais ou campos com o mesmo nome.

7.5 Exemplo Integrado de Escopos

```
package in exemplo.escopos;

// ESCOPO GLOBAL
e.const TAXA.float = 0.10;

fun calcularImposto(valor.float) >> float {
    // ESCOPO LOCAL DE FUNÇÃO
    let imposto.float = valor * TAXA;   // acessa constante global
    return imposto;
}

class.public Pedido {
    // ESCOPO DE CLASSE
    let total.float;

    fun aplicarImposto() >> float {
        // ESCOPO LOCAL DE MÉTODO
        let comImposto.float = total + calcularImposto(total);
        return comImposto;
    }
}

componente Home {
    let bemVindo.str = "Bem-vindo!";   // escopo de componente

    fun render() {
        let p.Pedido = new Pedido();   // escopo local
        p.total = 1000.0;
        System.print(bemVindo);
        System.print("Total: " + p.aplicarImposto());
    }
}
```

7.6 Boas Práticas

- **Inicialize sempre que possível:** variáveis sem inicialização explícita podem causar bugs sutis. Prefira `let x.int = 0;` a `let x.int;` quando o valor padrão for o desejado — torna a intenção visível.
- **Use constantes para "números mágicos":** qualquer valor literal que apareça mais de uma vez no código, ou que tenha significado conceitual (taxa de juros, máximo de tentativas), merece ser uma `e.const`.
- **Escopo o mais restrito possível:** declare variáveis no escopo mais interno onde forem necessárias. Isso reduz acoplamento e facilita manutenção.
- Nomes descritivos: prefira "quantidadeItens" a "qi", "precoFinal" a "pf". Espaço em disco é barato; tempo de leitura de código não é.

8. Operadores

UpperZetta oferece os operadores aritméticos, relacionais e lógicos esperados de uma linguagem de propósito geral, com poucas particularidades. Esta seção documenta cada categoria com exemplos práticos e detalhes sobre precedência.

8.1 Operadores Aritméticos

Operador	Operação	Exemplo	Resultado
+	Adição	5 + 3	8
+	Concatenação	"a" + "b"	"ab"
-	Subtração	10 - 4	6
*	Multiplicação	6 * 7	42
/	Divisão	20 / 4	5
/	Divisão (float)	7.0 / 2.0	3.5

8.1.1 Comportamento da Divisão

- **int / int** resulta em divisão inteira (parte fracionária descartada). Ex: `7 / 2` = 3, não 3.5.
- **float / float** resulta em divisão real. Ex: `7.0 / 2.0` = 3.5.
- **Divisão por zero** em int causa exceção em runtime; em float, produz Infinity ou NaN conforme o caso.

8.1.2 Concatenação com +

O operador + é polimórfico: se pelo menos um operando é string, a operação se torna concatenação e o outro operando é convertido implicitamente para sua representação textual:

```
let nome.str = "Ana";
let idade.int = 28;

System.print("Nome: " + nome);                  // Nome: Ana
System.print("Idade: " + idade);                // Idade: 28
System.print("Nome: " + nome + ", Idade: " + idade); // Nome: Ana, Idade: 28
System.print("Total: " + (10 + 5));             // Total: 15
System.print("Total: " + 10 + 5);               // Total: 105 (!)
```

ATENÇÃO
A última linha é uma armadilha clássica. Sem parênteses, "Total: " + 10 vira string "Total: 10", e + 5 concatena para "Total: 105". Use parênteses para forçar a precedência aritmética.

8.2 Operadores Relacionais

Comparam dois valores e produzem um boolean. Funcionam para tipos numéricos, strings (comparação lexicográfica) e booleans (apenas == e !=):

Operador	Significado	Exemplo	Resultado
==	Igual	5 == 5	true
!=	Diferente	5 != 3	true
<	Menor que	3 < 5	true
>	Maior que	5 > 3	true
<=	Menor ou igual	5 <= 5	true
>=	Maior ou igual	5 >= 6	false

8.3 Operadores Lógicos

Operador	Significado	Exemplo
&&	AND lógico (curto-circuito)	a > 0 && b > 0
||	OR lógico (curto-circuito)	erro || aviso

8.3.1 Avaliação de Curto-Circuito

8.4 Atribuição

Há três formas de atribuição:

```
x = 10;                  // variável simples
obj.campo = "novo";      // campo de objeto
arr[2] = 99;             // elemento de array por índice
```

> **NOTA**
> UpperZetta não tem operadores de atribuição compostos (+=, -=, *=). Use a forma expandida: x = x + 1 em vez de x += 1.

8.5 Tabela de Precedência Completa

Da maior para a menor precedência. Operadores com maior precedência são avaliados primeiro. Em caso de empate, a associatividade determina a ordem:

Nível	Operadores	Associatividade
1	() chamadas, . acesso, [] indexação	esquerda
2	* /	esquerda
3	+ -	esquerda
4	< > <= >=	esquerda
5	== !=	esquerda
6	&&	esquerda
7	||	esquerda
8	= (atribuição)	direita

8.5.1 Exemplos de Precedência

```
let a.int = 2 + 3 * 4;          // 14 (não 20): * antes de +
let b.int = (2 + 3) * 4;        // 20: parênteses forçam ordem
let c.boolean = 5 > 3 && 2 < 4; // true: relacionais antes de &&
let d.boolean = 5 > 3 || 1 == 0 && 2 == 2; // true: && antes de ||
```

DICA

Quando em dúvida, use parênteses. Mesmo que sejam tecnicamente desnecessários, parênteses tornam a intenção explícita e evitam erros de leitura por outras pessoas (e por você daqui a 6 meses).

PARTE II

Controle e Abstração

Estruturas de fluxo, funções, classes, componentes e dados

9. Controle de Fluxo

UpperZetta oferece três construções de controle de fluxo: condicional (if/else), loop com pré-condição (while) e loop indexado (for). Não há construções modernas como switch/match, foreach ou comandos de salto incondicional (break, continue, goto).

9.1 Condicionais — if / else

9.1.1 Sintaxe Básica

```
if (condicao) {
    // executa quando condicao é true
}

if (condicao) {
    // executa quando true
} else {
    // executa quando false
}

if (condicao1) {
    // ...
} else if (condicao2) {
    // ...
} else {
    // caso padrão
}
```

9.1.2 Chaves Opcionais

Para blocos com uma única instrução, as chaves podem ser omitidas. Esta forma é compacta mas pode reduzir a legibilidade — use com cautela:

```
if (x > 0)
    System.print("positivo");

// Equivalente, mais explícito:
if (x > 0) {
    System.print("positivo");
}
```

ATENÇÃO
Aninhamento sem chaves é especialmente perigoso. O classic "dangling else": "if (a) if (b) x; else y;" — o else pertence ao if interno. Sempre use chaves em código que outras pessoas lerão.

9.1.3 Padrão de Guarda (Early Return)

Em funções, é boa prática validar precondições logo no início e retornar cedo, evitando ninhos profundos:

```
fun calcularDesconto(preco.float, ehCliente.boolean) >> float {
    // Guarda 1: preço inválido
    if (preco <= 0.0) { return 0.0; }

    // Guarda 2: não é cliente
    if (ehCliente == false) { return preco; }

    // Caso principal: cliente com preço válido
    return preco * 0.9;
}
```

9.2 Loop — while

O while testa a condição antes de cada iteração. Se a condição for false na primeira verificação, o corpo nunca executa:

```
let i.int = 0;
while (i < 10) {
    System.print("iteração: " + i);
    i = i + 1;
}
```

9.2.1 Quando Usar while

- Quando o número de iterações não é conhecido antecipadamente.
- Quando a condição depende de processamento dentro do loop (ex: ler até encontrar marcador).
- Quando o critério de parada é baseado em estado (flag, contador externo).

9.2.2 Loops Infinitos

Tome cuidado para sempre modificar o estado que afeta a condição dentro do loop, sob risco de loop infinito:

```
// ❌ LOOP INFINITO
let i.int = 0;
while (i < 10) {
    System.print(i);
    // esqueceu i = i + 1
}

// ☑ CORRETO
```

```
let i.int = 0;
while (i < 10) {
    System.print(i);
    i = i + 1;
}
```

PERIGO
UpperZetta não tem mecanismo nativo para interromper um loop infinito a partir de dentro do programa. Você terá que matar o processo da UVLM externamente (Ctrl+C no terminal).

9.3 Loop — for

O `for` agrupa inicialização, condição e incremento em uma só linha. É a forma idiomática para iterar um número conhecido de vezes ou percorrer um array por índice:

```
for (init; condicao; incremento) {
    // corpo
}
```

Exemplos:

```
// Contar de 0 a 4
for (let i.int = 0; i < 5; i = i + 1) {
    System.print("i = " + i);
}

// Contar de trás para frente
for (let i.int = 10; i > 0; i = i - 1) {
    System.print("i = " + i);
}

// Pular de dois em dois
for (let i.int = 0; i < 100; i = i + 2) {
    System.print("par: " + i);
}

// Iterar array
let nomes.array = ["Ana", "Bia", "Caio"];
for (let i.int = 0; i < 3; i = i + 1) {
    System.print(nomes[i]);
}
```

9.3.1 for vs. while

A regra prática: se você sabe quantas vezes vai iterar (ou está percorrendo uma sequência indexada), use for. Se a parada depende de uma condição que não envolve um contador, use while.

9.4 A Ausência de break e continue

UpperZetta v1.0 não suporta break (interromper o loop) nem continue (pular para a próxima iteração). Esta é uma limitação do UVLM atual. Os padrões abaixo simulam o comportamento:

9.4.1 Simulando break com Flag

```
// Procurar primeiro número negativo
let arr.array = [3, 7, -2, 8, -5];
let achou.boolean = false;
let posicao.int = -1;

for (let i.int = 0; i < 5 && achou == false; i = i + 1) {
    if (arr[i] < 0) {
        achou = true;
        posicao = i;
    }
}

if (achou == true) {
    System.print("primeiro negativo no índice: " + posicao);
}
```

9.4.2 Simulando continue com if

```
// Imprimir apenas pares
for (let i.int = 0; i < 20; i = i + 1) {
    let metade.int = i / 2;
    if (metade * 2 == i) {        // i é par
        System.print(i);
    }
    // ímpares são ignorados
}
```

9.5 Aninhamento

Estruturas podem ser aninhadas livremente, mas evite mais de 2-3 níveis de profundidade — código aninhado é difícil de entender. Refatore extraindo partes para funções auxiliares:

```
// ❌ Difícil de ler — 4 níveis aninhados
for (let i.int = 0; i < 10; i = i + 1) {
```

```
    for (let j.int = 0; j < 10; j = j + 1) {
        if (i == j) {
            if (i > 5) {
                System.print(i + j);
            }
        }
    }
}

// ☑ Mais limpo
fun ehDiagonalAlta(i.int, j.int) >> boolean {
    return i == j && i > 5;
}

for (let i.int = 0; i < 10; i = i + 1) {
    for (let j.int = 0; j < 10; j = j + 1) {
        if (ehDiagonalAlta(i, j)) {
            System.print(i + j);
        }
    }
}
```

10. Funções

Funções em UpperZetta são unidades nomeadas de código com parâmetros tipados e tipo de retorno explícito. Podem ser declaradas no escopo global ou como métodos dentro de classes/componentes.

10.1 Declaração

```
fun nomeDaFuncao(param1.tipo1, param2.tipo2) >> tipoRetorno {
    // corpo
    return valor;
}
```

- A palavra-chave `fun` inicia a declaração.
- Parâmetros usam a mesma sintaxe `nome.tipo` das variáveis.
- O símbolo `>>` separa a lista de parâmetros do tipo de retorno.
- Se a função não retorna valor, omita `>> tipo` e `return` (a função tem retorno void implícito).

10.2 Exemplos por Categoria

10.2.1 Sem Parâmetros, Sem Retorno

```
fun saudar() {
    System.print("Olá, mundo!");
}

// Chamada
saudar();
```

10.2.2 Com Parâmetros e Retorno

```
fun somar(a.int, b.int) >> int {
    return a + b;
}

let resultado.int = somar(7, 5);
System.print(resultado);    // 12
```

10.2.3 Múltiplos Tipos de Retorno (via if)

```
fun classificar(nota.float) >> str {
    if (nota >= 9.0) { return "Excelente"; }
    else if (nota >= 7.0) { return "Bom"; }
    else if (nota >= 5.0) { return "Regular"; }
    else { return "Insuficiente"; }
}
```

10.2.4 Função com Float

```
fun media(a.float, b.float) >> float {
    return (a + b) / 2.0;
}
```

10.3 Funções Recursivas

Funções podem chamar a si mesmas. Útil para problemas que se decompõem naturalmente em subproblemas menores (fatorial, Fibonacci, percurso em árvores). Sempre defina um caso base claro para evitar recursão infinita:

```
fun fatorial(n.int) >> int {
    // caso base
    if (n <= 1) { return 1; }
    // caso recursivo
    return n * fatorial(n - 1);
}
```

```
System.print(fatorial(5));    // 120
System.print(fatorial(0));    // 1
```

> **ATENÇÃO**
> Recursão muito profunda pode estourar a pilha de chamadas da UVLM. Para n > ~5000 chamadas aninhadas, prefira versão iterativa com loop. A UVLM atual não otimiza tail call recursion.

10.4 Passagem de Parâmetros

10.4.1 Tipos Primitivos: por Valor

Quando você passa int, float, boolean ou str para uma função, o valor é copiado. Modificações no parâmetro dentro da função NÃO afetam a variável original do chamador:

```
fun modificar(x.int) {
    x = x * 2;
    System.print("dentro: " + x);
}

let n.int = 10;
modificar(n);
System.print("fora: " + n);
// dentro: 20
// fora: 10  (n não mudou)
```

10.4.2 Objetos: por Referência

Objetos (instâncias de class.public) são passados por referência: a função recebe um ponteiro para o mesmo objeto. Modificações nos campos do objeto SÃO visíveis no chamador:

```
class.public Caixa {
    let valor.int;
}

fun zerar(c.Caixa) {
    c.valor = 0;    // afeta o objeto original
}

componente Home {
    fun render() {
        let cx.Caixa = new Caixa();
        cx.valor = 42;
        zerar(cx);
        System.print(cx.valor);   // 0
    }
```

```
}
```

10.5 Funções Puras vs. Impuras

Funções puras dependem apenas de seus parâmetros e produzem o mesmo resultado para os mesmos argumentos, sem efeitos colaterais. Funções impuras leem ou modificam estado externo (campos, constantes globais via mutação, saída):

```
// PURA: sem efeitos colaterais, determinística
fun quadrado(x.int) >> int {
    return x * x;
}

// IMPURA: tem efeito colateral (saída)
fun saudarComLog(nome.str) {
    System.print("logado: " + nome);
}
```

> **DICA**
> Prefira funções puras sempre que possível. São mais fáceis de testar mentalmente, reutilizar e refatorar. Reserve funções impuras para a "borda" do programa: I/O, atualização de estado.

10.6 Convenções de Nomenclatura

- **camelCase:** `calcularMedia`, `ehAdulto`, `enviarEmail`.
- **Verbos para ações:** `salvar`, `remover`, `atualizar` — funções que executam algo.
- **Prefixo "eh" ou "tem" para predicados:** `ehAdulto`, `temPermissao` — funções que retornam boolean.
- **Prefixo "obter" ou "get" para acessores:** `obterUsuario`, `getNome` — funções que retornam valores.

11. Classes

Classes em UpperZetta são modelos para criar objetos. Encapsulam dados (campos) e comportamento (métodos). São declaradas com class.public e instanciadas com new.

11.1 Declaração

```
class.public NomeDaClasse {
    let campo1.tipo;
    let campo2.tipo;

    fun metodo(param.tipo) >> tipoRetorno {
        // corpo
        return campo1;
    }
}
```

11.1.1 Por que .public no nome?

O sufixo `.public` indica que a classe é exportável — visível e instanciável de qualquer parte do arquivo. A versão atual não suporta classes "internas" ou "privadas"; todas são públicas. O sufixo é mantido por consistência com convenções de outras linguagens orientadas a objeto e para reservar espaço sintático para futuras visibilidades.

11.2 Acesso a Campos: Sem this

Diferente de Java/JavaScript, métodos de uma classe acessam seus próprios campos diretamente pelo nome, sem precisar de prefixo (this.campo, self.campo). O UVLM injeta a referência ao objeto no slot local 0, e nomes não-locais sobem automaticamente para o objeto:

```
class.public Pessoa {
    let nome.str;
    let idade.int;

    fun apresentar() >> str {
        return nome + " tem " + idade + " anos.";
        // nome e idade são resolvidos no objeto
    }
}
```

> **ATENÇÃO**
> Se você declarar uma variável local com o mesmo nome de um campo, ela sombreará o campo dentro do método. Para evitar confusão, use nomes distintos.

11.3 Instanciação

Para criar uma instância (objeto) de uma classe, use new:

```
let p.Pessoa = new Pessoa();
p.nome = "Maria";
p.idade = 30;
System.print(p.apresentar());
```

- `new Pessoa()` aloca memória para o objeto e retorna sua referência.
- Os campos são inicializados com valores padrão de seus tipos (0, "", false, []).
- Use `objeto.campo = valor` para definir campos após a criação.
- Use `objeto.metodo()` para invocar métodos.

11.4 Construtores: Não Existem

UpperZetta v1.0 não suporta construtores. Para inicializar um objeto, defina os campos manualmente após o new ou crie uma função fábrica que faz isso:

```
// Padrão: função fábrica
fun criarUsuario(nome.str, idade.int) >> Usuario {
    let u.Usuario = new Usuario();
    u.nome = nome;
    u.idade = idade;
    u.ativo = true;
    return u;
}

componente Home {
    fun render() {
        let user.Usuario = criarUsuario("Carlos", 35);
        System.print(user.nome);
    }
}
```

11.5 Exemplo Completo: Classe com Lógica

```
class.public Usuario {
    let nome.str;
    let idade.int;
    let ativo.boolean;

    fun ehAdulto() >> boolean {
        return idade >= 18;
    }

    fun podeEntrar() >> boolean {
        if (ativo == true && idade >= 18) {
            return true;
```

```
        } else {
            return false;
        }
    }

    fun rotulo() >> str {
        if (ativo == false) { return "inativo"; }
        else if (idade < 18) { return "menor"; }
        else { return "adulto_ativo"; }
    }

    fun aniversario() {
        idade = idade + 1;
    }
}

componente Home {
    fun render() {
        let u.Usuario = new Usuario();
        u.nome = "Maria";
        u.idade = 25;
        u.ativo = true;

        System.print(u.nome + ": " + u.rotulo());
        u.aniversario();
        System.print("Após aniversário: " + u.idade);
    }
}
```

11.6 Composição

UpperZetta v1.0 não suporta herança, mas suporta composição: um objeto pode conter outros objetos como campos:

```
class.public Endereco {
    let rua.str;
    let cidade.str;
}

class.public Cliente {
    let nome.str;
    let endereco.Endereco;

    fun resumo() >> str {
        return nome + " - " + endereco.cidade;
    }
}

componente Home {
```

```
    fun render() {
        let c.Cliente = new Cliente();
        c.nome = "João";
        c.endereco = new Endereco();
        c.endereco.rua = "Rua A";
        c.endereco.cidade = "Franca";

        System.print(c.resumo());
    }
}
```

11.7 Classes vs. Componentes

Aspecto	class.public	componente
Inicialização de campo	Não permitida	Permitida
Instanciável com new	Sim	Não
Ponto de entrada	Não	render() é entry point
Múltiplas instâncias	Sim	Singleton implícito
Uso típico	Modelos de dados	Lógica principal / UI
Pode chamar outras	Sim	Sim

DICA
Regra prática: se representa "uma coisa" do domínio (Usuário, Pedido, Produto) e pode existir em várias instâncias, é uma class. Se é a aplicação rodando ou a tela atual, é um componente.

12. Componentes

`componente` é um tipo especial de classe com duas características que o distinguem: pode inicializar campos diretamente na declaração, e seu método `render()` (quando o componente se chama `Home`) é o ponto de entrada do programa.

12.1 Declaração

```
componente NomeDoComponente {
    let campo.tipo = valorInicial;

    fun render() {
```

```
        // código principal
    }

    fun metodoAuxiliar(param.tipo) >> tipo {
        // ...
    }
}
```

12.2 Inicialização de Campos

Diferentemente de classes, componentes podem ter valores iniciais nos campos. Estes valores são definidos quando o componente é instanciado pela UVLM no início da execução:

```
componente Home {
    let titulo.str = "Sistema de Estoque";
    let versao.str = "1.0.0";
    let contador.int = 0;
    let ativo.boolean = true;

    fun render() {
        System.print(titulo + " v" + versao);
    }
}
```

> **ATENÇÃO**
> Tentar inicializar um campo dentro de class.public causa erro UZ-2210. Apenas componentes suportam essa sintaxe.

12.3 O Método render()

render() é convencionado como ponto de entrada do componente Home. A UVLM, ao iniciar, equivale a executar:

```
// Pseudo-código do que a UVLM faz
let home.Home = new Home();    // implícito
home.render();                  // entry point
// processo termina ao retornar de render()
```

- `render()` não recebe parâmetros.
- Não retorna valor (void implícito).
- Pode invocar quaisquer outros métodos do próprio componente, funções globais ou criar instâncias de classes.

12.4 Métodos Auxiliares no Componente

Componentes podem ter múltiplos métodos. render() chama os demais conforme necessário:

```
componente Home {
    let saldo.float = 0.0;

    fun render() {
        depositar(100.0);
        depositar(50.0);
        sacar(30.0);
        exibirSaldo();
    }

    fun depositar(valor.float) {
        saldo = saldo + valor;
    }

    fun sacar(valor.float) {
        if (valor <= saldo) {
            saldo = saldo - valor;
        } else {
            System.print("saldo insuficiente");
        }
    }

    fun exibirSaldo() {
        System.print("saldo atual: R$ " + saldo);
    }
}
```

12.5 Padrão: Componente como Orquestrador

Em programas maiores, o componente Home funciona como um orquestrador: ele coordena classes e funções globais sem implementar lógica de domínio diretamente:

```
package in app.vendas;

class.public Produto {
    let nome.str;
    let preco.float;
}

class.public Carrinho {
    let total.float;

    fun adicionar(p.Produto) {
        total = total + p.preco;
    }
```

```
}

fun aplicarCupom(valor.float, percentual.float) >> float {
    return valor * (1.0 - percentual);
}

componente Home {
    fun render() {
        let p1.Produto = new Produto();
        p1.nome = "Café";
        p1.preco = 25.0;

        let c.Carrinho = new Carrinho();
        c.adicionar(p1);

        let final.float = aplicarCupom(c.total, 0.10);
        System.print("Total final: R$ " + final);
    }
}
```

DICA
Quanto mais limpo for o componente Home, mais reutilizáveis tendem a ser as classes e funções do projeto. Use Home apenas para amarrar peças, não para conter regras de negócio.

13. Arrays

Arrays em UpperZetta são vetores ordenados, indexados por inteiros começando em zero. Aceitam valores de qualquer tipo, inclusive misturados, embora a uniformidade de tipos seja recomendada.

13.1 Criação

```
let inteiros.array = [1, 2, 3, 4, 5];
let strings.array = ["Ana", "Bruno", "Carlos"];
let booleans.array = [true, false, true];
let floats.array = [1.5, 2.5, 3.5];
let vazio.array = [];
let misto.array = [1, "dois", true, 3.14];   // permitido, mas evite
```

13.2 Acesso por Índice

Use colchetes com o índice (baseado em zero) para ler ou escrever um elemento:

```
let nomes.array = ["Ana", "Bruno", "Carlos"];

let primeiro.str = nomes[0];   // "Ana"
let segundo.str = nomes[1];    // "Bruno"
let terceiro.str = nomes[2];   // "Carlos"

nomes[1] = "Beatriz";          // modifica o segundo elemento
System.print(nomes[1]);        // "Beatriz"
```

PERIGO
Acessar um índice fora dos limites (ex: nomes[10] em um array de 3 elementos) lança erro UZ-3001 em runtime e termina o programa.

13.3 Iteração com for

Como UpperZetta não tem foreach, use for indexado:

```
let valores.array = [10, 20, 30, 40, 50];

for (let i.int = 0; i < 5; i = i + 1) {
    System.print("valores[" + i + "] = " + valores[i]);
}
```

ATENÇÃO
A versão atual da linguagem não tem operador nativo para obter o tamanho de um array. Use uma constante ou variável para armazenar o tamanho conhecido. Ex: e.const TAM.int = 5; ou let n.int = 5; antes de criar o array.

13.4 Arrays como Parâmetros

Você pode passar arrays para funções. Como objetos, são passados por referência — modificações afetam o array original:

```
fun dobrarTodos(arr.array, n.int) {
    for (let i.int = 0; i < n; i = i + 1) {
        arr[i] = arr[i] * 2;
    }
}

componente Home {
    fun render() {
        let v.array = [1, 2, 3, 4, 5];
        dobrarTodos(v, 5);
```

```
        // v agora é [2, 4, 6, 8, 10]
        for (let i.int = 0; i < 5; i = i + 1) {
            System.print(v[i]);
        }
    }
}
```

13.5 Arrays como Retorno

```
fun gerarSequencia(n.int) >> array {
    let resultado.array = [0, 0, 0, 0, 0];
    for (let i.int = 0; i < n; i = i + 1) {
        resultado[i] = i * i;
    }
    return resultado;
}

componente Home {
    fun render() {
        let quadrados.array = gerarSequencia(5);
        for (let i.int = 0; i < 5; i = i + 1) {
            System.print(quadrados[i]);
        }
    }
}
```

13.6 Algoritmos Comuns

13.6.1 Encontrar o Maior Valor

```
fun maior(arr.array, n.int) >> int {
    let m.int = arr[0];
    for (let i.int = 1; i < n; i = i + 1) {
        if (arr[i] > m) {
            m = arr[i];
        }
    }
    return m;
}
```

13.6.2 Soma de Elementos

```
fun soma(arr.array, n.int) >> int {
    let total.int = 0;
    for (let i.int = 0; i < n; i = i + 1) {
        total = total + arr[i];
    }
```

```
    return total;
}
```

13.6.3 Busca Linear

```
fun encontrar(arr.array, n.int, alvo.int) >> int {
    for (let i.int = 0; i < n; i = i + 1) {
        if (arr[i] == alvo) {
            return i;       // retorna índice
        }
    }
    return -1;              // não encontrado
}
```

13.7 Arrays Multidimensionais

UpperZetta não tem sintaxe nativa para arrays 2D. Para representá-los, use array de arrays:

```
// Matriz 3x3
let matriz.array = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
];

// Acessar elemento da linha 1, coluna 2
let v.int = matriz[1][2];   // 6

// Iterar matriz inteira
for (let i.int = 0; i < 3; i = i + 1) {
    for (let j.int = 0; j < 3; j = j + 1) {
        System.print("m[" + i + "][" + j + "] = " + matriz[i][j]);
    }
}
```

14. Saída e Depuração

A versão atual de UpperZetta oferece apenas uma função de saída nativa: System.print. Não há função de leitura de entrada do teclado, escrita em arquivo ou interação com bibliotecas externas. Toda saída do programa é texto enviado ao terminal.

14.1 System.print

Aceita um único argumento de qualquer tipo e o exibe no terminal seguido de quebra de linha:

```
System.print("Texto literal");
System.print(42);
System.print(3.14);
System.print(true);
System.print(["a", "b", "c"]);   // arrays também
```

14.2 Comportamento por Tipo

Tipo do argumento	Saída no terminal
str	Texto exatamente como está, sem aspas
int	Número inteiro decimal
float	Número com ponto decimal (ex: 3.14)
boolean	"true" ou "false" (em inglês)
array	Representação textual do array, ex: [1, 2, 3]
null	A palavra "null"
Concatenação +	Converte o lado não-string para string e imprime

14.3 Concatenação para Saída Formatada

```
let nome.str = "Felipe";
let idade.int = 21;
let altura.float = 1.78;
let casado.boolean = false;

System.print("Nome:    " + nome);
System.print("Idade:   " + idade + " anos");
System.print("Altura:  " + altura + "m");
System.print("Casado:  " + casado);

// Saída:
// Nome:    Felipe
// Idade:   21 anos
// Altura:  1.78m
// Casado:  false
```

14.4 Estratégias de Debug

Sem debugger interativo, depuração em UpperZetta é feita via "printf debugging" — espalhar System.print pelo código para entender o fluxo:

14.4.1 Marcadores de Fluxo

```
fun calcular(x.int) >> int {
    System.print("[debug] entrou em calcular com x=" + x);

    let y.int = x * 2;
    System.print("[debug] após multiplicação, y=" + y);

    if (y > 100) {
        System.print("[debug] y > 100, retornando -1");
        return -1;
    }

    System.print("[debug] retornando y normal");
    return y;
}
```

14.4.2 Inspeção de Estado

```
class.public Conta {
    let saldo.float;
    let nome.str;

    fun debug() {
        System.print("Conta(nome=" + nome + ", saldo=" + saldo + ")");
    }
}

// Use em pontos críticos
let c.Conta = new Conta();
c.nome = "Felipe";
c.saldo = 100.0;
c.debug();    // Conta(nome=Felipe, saldo=100.0)
```

14.5 Logs Estruturados

Em projetos maiores, padronize formato dos logs para facilitar busca e análise:

```
fun log(nivel.str, modulo.str, mensagem.str) {
    System.print("[" + nivel + "] [" + modulo + "] " + mensagem);
}

componente Home {
    fun render() {
        log("INFO", "Auth", "usuário logado");
        log("WARN", "DB", "conexão lenta detectada");
        log("ERROR", "Net", "timeout após 30s");
    }
}
```

```
// Saída:
// [INFO] [Auth] usuário logado
// [WARN] [DB] conexão lenta detectada
// [ERROR] [Net] timeout após 30s
```

> **DICA**
> Antes de distribuir um programa, remova ou comente os System.print de debug. Saída de log poluída irrita usuários e pode vazar informações sensíveis.

15. Sistema de Pacotes

Pacotes em UpperZetta são namespaces lógicos que identificam o módulo a que um arquivo pertence. A declaração de pacote é a primeira instrução do arquivo (após comentários iniciais), e ainda que a versão atual do compilador não use pacotes para resolução cross-arquivo, eles servem como documentação e metadados.

15.1 Sintaxe

```
package in nome.do.pacote;
```

- A palavra-chave "package in" inicia a declaração.
- O nome do pacote é uma sequência de identificadores separados por pontos.
- Termina com ponto-e-vírgula.
- Apenas uma declaração por arquivo.
- Deve ser a primeira instrução não-comentário do arquivo.

15.2 Convenções de Nomenclatura

Padrão	Exemplo
Tudo minúsculo	app.util
Domínio reverso (formal)	br.com.empresa.modulo
Hierárquico (informal)	meu.projeto.modelos
Por funcionalidade	app.auth, app.banco, app.api
Por camada	app.dominio, app.servico, app.ui

15.3 Exemplos Práticos

```
// Aplicação simples
package in app.principal;

// Biblioteca de utilidades
package in util.strings;

// Projeto acadêmico
package in tcc.calculadora;

// Padrão domínio reverso
package in br.com.axyon.dashboard;

// Submódulos
package in sistema.usuarios.autenticacao;
```

15.4 Quando Omitir o Pacote

A declaração de pacote é opcional. Para scripts pequenos, exemplos didáticos ou snippets de teste, pode-se omitir sem prejuízo de funcionalidade. Para qualquer projeto que pretenda ser revisitado ou compartilhado, declare um pacote para documentar a origem e a intenção do código.

15.5 Limitações Atuais

Importante entender o que o sistema de pacotes da v1.0 não faz:

- Não há mecanismo de "import" ou "use" para referenciar código de outro arquivo.
- Pacotes não criam visibilidade ou escopo cruzado entre arquivos.
- O compilador processa um arquivo .uz por vez como unidade isolada.
- A informação de pacote é armazenada como metadado no bytecode .uzb e no envelope .uzs.

> **NOTA**
> Versões futuras devem evoluir para um sistema de módulos que use a declaração de pacote para resolução de imports e gerenciamento de dependências. Manter declarações de pacote consistentes desde já facilita migração futura.

PARTE III

Compilador e Runtime

Pipeline de compilação, bytecode UVLM, formato GLP e selagem

16. Modelo de Compilação

Esta seção documenta o pipeline completo que transforma um arquivo .uz em bytecode .uzb executável. Entender esta sequência é útil para depurar erros de compilação, otimizar código e contribuir com o ecossistema da linguagem.

16.1 Visão Geral do Pipeline

```
Código-fonte (.uz)
       │
       ▼
   ┌─────────┐
   │ LEXER   │   tokenização via expressões regulares
   └─────────┘
        │  fluxo de tokens
        ▼
   ┌─────────┐
   │ PARSER  │   construção da AST (Abstract Syntax Tree)
   └─────────┘
        │  árvore sintática validada
        ▼
   ┌─────────┐
   │ CODEGEN │   geração de bytecode UVLM
   └─────────┘
        │  instruções da máquina de pilha
        ▼
   ┌──────────────┐
   │ GLP-ENCODER  │   formato palindrômico (Block A + espelho B)
   └──────────────┘
          │
          ▼
   Bytecode (.uzb)
          │
          ▼
   ┌─────────┐
   │  UVLM   │   execução na máquina virtual
   └─────────┘
```

16.2 Fase 1 — Análise Léxica (Lexer)

O lexer (também chamado scanner) lê o código-fonte caractere por caractere e produz uma sequência de tokens. Cada token é uma unidade léxica significativa: palavra-chave, identificador, literal, operador ou pontuação.

16.2.1 Categorias de Token

Categoria	Exemplos
KEYWORD	package, in, e.const, let, fun, return, class.public, componente, if, else, while, for, new, true, false
IDENT	nome, idade, calcularJuros, Usuario
NUMBER_INT	42, -7, 100
NUMBER_FLT	3.14, -0.5, 1.0
STRING	"Olá", "texto"
OPERATOR	+, -, *, /, ==, !=, <, >, <=, >=, &&, ||, =
PUNCTUATION	(,), {, }, [,], ;, ,, .
ARROW	>>

16.2.2 Erros de Lexer

Erros nesta fase são raros e geralmente envolvem caracteres inválidos ou strings não fechadas:

```
let x.int = 5 @ 3;        // ERRO: caractere @ inválido
let nome.str = "sem fim;  // ERRO: string não fechada
```

16.3 Fase 2 — Análise Sintática (Parser)

O parser consome o fluxo de tokens e tenta encaixá-los em uma árvore sintática (AST) que reflete a estrutura gramatical da linguagem. Se a sequência de tokens não corresponde a nenhuma regra da gramática, é gerado um erro de sintaxe.

16.3.1 Exemplo de AST

Para o código let x.int = 2 + 3;, o parser produz uma árvore conceitualmente assim:

```
        VAR_DECL
       /   |    \
      /    |     \
  "x"    "int"   ASSIGN
                   |
                 PLUS
                 /  \
             INT(2) INT(3)
```

16.3.2 Erros Comuns de Parser

```
fun soma(a, b) >> int { return a + b; }
//        ^ ERRO: parâmetro 'a' sem anotação de tipo

let x.int = 10
//             ^ ERRO: ponto-e-vírgula faltando

class Pessoa { ... }
//   ^ ERRO: esperado .public após 'class'
```

16.4 Fase 3 — Geração de Código (CodeGen)

Com a AST validada, o gerador de código percorre a árvore e emite instruções de bytecode UVLM. Cada nó da árvore se traduz em uma ou mais instruções de máquina:

Construção UZ	Instruções aproximadas
let x.int = 42;	PUSH_INT 42 → STORE_LOCAL slot_x
x + y	LOAD_LOCAL x → LOAD_LOCAL y → ADD
if (cond) { ... }	avaliação → JUMP_IF_FALSE label_fim
while (cond) { ... }	label_inicio: cond → JUMP_IF_FALSE fim → corpo → JUMP label_inicio
fun f(...) { ... }	FUNCTION_BEGIN f → corpo → RETURN
System.print(x)	LOAD_LOCAL x → CALL_BUILTIN print

16.5 Fase 4 — Codificação GLP

Após gerado o bytecode bruto, ele é envolvido no formato GLP (ver Seção 18 para detalhes completos). Esta etapa adiciona o cabeçalho, computa o espelho palindrômico e produz o arquivo .uzb final.

16.6 Invocando o Compilador

```shell
# Modo desenvolvimento: compilar e executar em sequência
java -cp Main.jar Main programa.uz

# Apenas executar bytecode pré-compilado
java -cp Main.jar Main programa.uzb

# Ver bytecode em forma legível (disassembly)
java -cp Main.jar Main programa.uzb --disasm

# Com alias configurado
uz programa.uz
uz programa.uzb
uz programa.uzb --disasm
```

16.7 Mensagens de Erro do Compilador

Erros do compilador seguem o formato: UZ-CÓDIGO:linha:coluna mensagem. Exemplos:

```
UZ-1023:5:14 expected ';' after expression
UZ-1045:12:8 expected '.tipo' after parameter name
UZ-2105:8:5 'e.const' não permitido fora do escopo global
UZ-2210:14:9 inicialização de campo não permitida em class.public
UZ-3001:42:12 array index 10 out of bounds (size 5)
```

> **NOTA**
> Códigos UZ-1xxx são erros léxicos/sintáticos, UZ-2xxx são semânticos (compile-time), UZ-3xxx são de runtime. Ver Seção 28 para tabela completa.

17. Bytecode UVLM

A UpperZetta Virtual Language Machine (UVLM) é uma máquina virtual de pilha que executa o bytecode produzido pelo compilador. Esta seção descreve sua arquitetura interna e o modelo de execução.

17.1 Arquitetura da Máquina

- **Modelo:** máquina de pilha (stack machine), similar à JVM e à .NET CLR.
- **Frame de chamada:** cada chamada de função/método cria um frame com 256 slots de variáveis locais.

- **Pilha de operandos:** instruções operam sobre valores no topo da pilha (push, pop, duplicate, swap).
- **Heap:** objetos e arrays são alocados em heap; locais armazenam apenas referências.
- **Constant pool:** literais (strings, números) são deduplicados em uma tabela global por arquivo.

17.2 Tipos Internos de Valor

Cada valor na pilha ou em um slot local carrega um identificador de tipo, usado pela UVLM para verificações em tempo de execução:

ID	Tipo UZ	Representação interna
1	int	Inteiro 32 bits com sinal
2	str	Referência para objeto String UTF-8
3	boolean	0 (false) ou 1 (true)
4	objeto	Referência para instância em heap
5	null	Marcador de ausência (campos não-inicializados de classe)
6	float	IEEE 754 dupla precisão (64 bits)
7	array	Referência para vetor heterogêneo

17.3 Modelo de Execução

A execução de cada instrução segue o ciclo clássico fetch-decode-execute:

9. Fetch — UVLM lê a próxima instrução a partir do PC (program counter).
10. Decode — interpreta o opcode e os operandos imediatos.
11. Execute — manipula a pilha de operandos, frame local ou heap conforme a operação.
12. Avança — incrementa o PC (ou salta para um endereço, em caso de jump).

17.4 Frame de Chamada

Quando uma função ou método é invocado, a UVLM cria um novo frame contendo:

- 256 slots locais (numerados 0 a 255) para variáveis locais e parâmetros.
- Para métodos de classe/componente, o **slot 0 contém a referência para o objeto** (this implícito).
- Parâmetros explícitos são alocados a partir do slot 1 (em métodos) ou slot 0 (em funções globais).
- Variáveis locais declaradas com let recebem slots subsequentes.

- Pilha de operandos própria do frame.
- Endereço de retorno (PC do chamador).

> **ATENÇÃO**
> O limite de 256 slots por frame raramente é atingido em código normal, mas funções com muitas variáveis locais ou parâmetros (>200) podem estourar este limite. Se isso acontecer, refatore extraindo lógica para funções auxiliares.

17.5 Disassembly: Lendo o Bytecode

O comando --disasm produz uma representação textual do bytecode, útil para entender o que o compilador gera ou para depurar problemas de baixo nível:

```shell
uz programa.uzb --disasm
```

Para o programa abaixo:

```soma.uz
fun soma(a.int, b.int) >> int {
    return a + b;
}

componente Home {
    fun render() {
        let r.int = soma(3, 4);
        System.print(r);
    }
}
```

A saída do --disasm seria conceitualmente:

```disassembly
FUNCTION soma (locals=2)
  0000  LOAD_LOCAL    0    ; a
  0001  LOAD_LOCAL    1    ; b
  0002  ADD_INT
  0003  RETURN

COMPONENT Home
  FUNCTION render (locals=2)
    0000  PUSH_INT      3
    0001  PUSH_INT      4
    0002  CALL          soma
```

```
0003  STORE_LOCAL   1    ; r
0004  LOAD_LOCAL    1
0005  CALL_BUILTIN  print
0006  RETURN
```

17.6 Operações de Pilha Comuns

Categoria	Operações típicas
Carregamento	PUSH_INT, PUSH_FLOAT, PUSH_STR, LOAD_LOCAL, LOAD_FIELD
Armazenamento	STORE_LOCAL, STORE_FIELD, STORE_INDEX
Aritméticas	ADD_INT, SUB_INT, MUL_INT, DIV_INT, ADD_FLOAT, ...
Comparação	CMP_EQ, CMP_NEQ, CMP_LT, CMP_GT, CMP_LE, CMP_GE
Lógicas	AND, OR (com curto-circuito implementado via JUMP)
Controle	JUMP, JUMP_IF_TRUE, JUMP_IF_FALSE, CALL, RETURN
Objetos	NEW, GET_FIELD, SET_FIELD, INVOKE_METHOD
Arrays	NEW_ARRAY, ARRAY_GET, ARRAY_SET
Built-ins	CALL_BUILTIN (ex: print)

NOTA
Os opcodes específicos podem variar entre versões do UVLM. O compilador e o runtime são projetados em conjunto, e o formato GLP garante compatibilidade entre versões da mesma linha (1.x).

18. Formato GLP

GLP (Generative Logic Palindrome) é o formato binário usado para armazenar bytecode UVLM em arquivos .uzb. Sua característica distintiva é a estrutura palindrômica: o arquivo contém o código (Block A) seguido de seu espelho (Block B), permitindo verificação de integridade sem checksum externo.

18.1 Motivação do Design

Formatos binários tradicionais (como .class do Java) usam checksums separados (ex: hash MD5 ou CRC32) anexados ao arquivo. O GLP integra a verificação na própria estrutura: se algum byte do Block A for corrompido, o espelho em Block B revela a discrepância imediatamente.

- **Sem checksum externo** — a integridade está embutida.
- **Detecção rápida** — comparação byte-a-byte ao carregar.
- **Resistente a corrupção parcial** — apenas o bloco corrompido falha; o outro pode reconstruir o conteúdo.
- **Verificação determinística** — não depende de algoritmos de hash externos.

18.2 Estrutura do Arquivo .uzb

```
┌──────────────────────────────────────┐
│  HEADER (16 bytes)                   │
│   - magic ("UZB\0")                  │
│   - versão (4 bytes)                 │
│   - tamanho do Block A (8 bytes)     │
├──────────────────────────────────────┤
│  BLOCK A (bytecode)                  │
│   - constant pool                    │
│   - tabela de funções                │
│   - tabela de classes                │
│   - tabela de componentes            │
│   - código de cada função            │
├──────────────────────────────────────┤
│  BLOCK B (espelho de A, invertido)   │
│   - mesmos bytes em ordem reversa    │
└──────────────────────────────────────┘
```

18.3 Verificação de Integridade

Ao carregar um .uzb, a UVLM realiza:

13. Lê o magic e versão; aborta se incompatíveis.
14. Lê o tamanho do Block A.
15. Carrega Block A para memória.
16. Carrega Block B para memória.
17. Inverte Block B byte a byte.
18. Compara byte-a-byte com Block A.
19. Se idênticos, prossegue; caso contrário, aborta com erro UZ-9001 (arquivo corrompido).

18.4 Tamanho dos Arquivos

Como o GLP duplica os dados (A + espelho de A), os arquivos .uzb são aproximadamente 2× maiores que seriam em um formato compacto. Este overhead é o custo da garantia de integridade autocontida:

Programa	Tamanho aproximado .uzb
Hello World mínimo	~1 KB
Calculadora simples	~3-5 KB
Aplicação média	~50-200 KB
Programa grande	~500 KB - 2 MB

> **NOTA**
> Para distribuir versões compactas de programas, sele para .uzs (que aplica AES + compressão implícita), ou compacte com gzip externamente.

19. Selar e Desselar Arquivos (.uzs)

O formato `.uzs` é uma camada de proteção construída sobre `.uz` ou `.uzb`. Aplica criptografia AES-256 com derivação de chave a partir de senha (PBKDF2) e, opcionalmente, vinculação ao dispositivo (DEVICE KEY). É a forma recomendada de distribuir software UpperZetta proprietário.

19.1 Casos de Uso

- **Distribuição comercial:** produto vendido onde o código não pode ser inspecionado livremente.
- **Compartilhamento sob NDA:** enviar código para cliente ou parceiro com proteção contra leitura por terceiros.
- **Arquivamento sensível:** guardar versões legadas ou snapshots com proteção em repouso.
- **Trabalhos acadêmicos:** preservar autenticidade de submissões em concursos ou avaliações com prazo.

19.2 Selar via CLI

```shell
uz seal meuarquivo.uz
# Saída:
# Senha: ********
# Confirmar: ********
# Selado em: meuarquivo.uzs
```

```
# Pode selar bytecode também
uz seal meuarquivo.uzb
```

- A senha é solicitada interativamente (não exibida no terminal).
- Confirme digitando duas vezes.
- O arquivo .uzs é gerado no mesmo diretório do original.
- O arquivo original NÃO é deletado — remova manualmente se quiser apenas a versão selada.

19.3 Desselar via CLI

```shell
uz unseal meuarquivo.uzs
# Saída:
# Senha: ********
# Restaurado em: meuarquivo.uz
```

> **ATENÇÃO**
> A senha digitada incorretamente apenas resulta em "decryption failed" — não há mecanismo de "recovery hint" ou recuperação de senha. Se você esquecer a senha, o arquivo é irrecuperável.

19.4 Selar via ZettaSource IDE

- **Selar arquivo aberto:** menu `Arquivo → Selar Código-Fonte (.uzs)...` ou atalho .
- **Abrir arquivo .uzs:** `Arquivo → Abrir Arquivo...`, selecione um `.uzs` e insira a senha quando solicitada.
- **Editar e re-selar:** após abrir um `.uzs`, edite normalmente. Use `Salvar` para sobrescrever o arquivo selado mantendo a mesma senha.

19.5 Boas Práticas de Senha

- Use senhas longas (mínimo 16 caracteres).
- Combine maiúsculas, minúsculas, números e símbolos.
- Não reutilize senhas de outros sistemas.
- Armazene em gerenciador de senhas confiável (1Password, Bitwarden, KeePass).
- Para distribuição comercial, gere uma senha por cliente/release.

19.6 Formatos de Criptografia

Magic	Algoritmo	Origem
UZS!	AES-256-GCM, PBKDF2-SHA512 (600.000 iterações)	ZettaSource IDE (Export)
UZS1	AES-256-CBC, PBKDF2-SHA512 (100.000 iter.) + DEVICE KEY	CLI seal

19.6.1 UZS! (modo IDE Export)

- Não vincula ao dispositivo: pode ser aberto em qualquer máquina com a senha.
- Usa GCM (Galois/Counter Mode): autenticação integrada, detecta adulteração.
- 600 mil iterações de PBKDF2: lento de derivar a chave (boa proteção contra brute force).
- Recomendado para distribuição multi-máquina ou backup pessoal.

19.6.2 UZS1 (modo CLI seal)

- Vincula ao dispositivo via DEVICE KEY (ver Seção 20).
- Usa CBC: padrão clássico, sem autenticação integrada.
- 100 mil iterações de PBKDF2: derivação mais rápida.
- Recomendado para arquivamento local ou licenciamento por máquina.

20. DEVICE KEY e Migração entre Máquinas

O esquema UZS1 (CLI seal) vincula arquivos selados ao dispositivo onde foram criados, usando uma chave gerada na primeira execução da UVLM. Esta seção cobre como funciona, quando usar e como migrar entre máquinas.

20.1 O que é a DEVICE KEY

A DEVICE KEY é uma chave criptográfica única gerada aleatoriamente na primeira vez que a UVLM é executada em um computador. Fica armazenada em `~/.uvlm/.uvlm_dk` (Unix/macOS) ou no equivalente do Windows (`%USERPROFILE%\.uvlm\.uvlm_dk`).

20.2 Como Afeta a Selagem

Arquivos selados via CLI (formato UZS1) usam tanto a senha quanto a DEVICE KEY para derivar a chave AES final. Isso significa que:

- Mesmo que alguém obtenha a senha correta, sem a DEVICE KEY do dispositivo original o arquivo permanece criptografado.

- Mover o .uzs para outra máquina sem migrar a DEVICE KEY torna o arquivo inacessível.
- Adiciona uma segunda camada de defesa: o invasor precisa de senha + arquivo de chave.

PERIGO
Se você reformatar o computador sem fazer backup da DEVICE KEY, todos os arquivos UZS1 selados naquela máquina ficam permanentemente irrecuperáveis. Ver Seção 20.4 sobre backup.

20.3 Visualizando a DEVICE KEY

```shell
uz key-show
# Saída (exemplo):
# DEVICE KEY: a3f7c92e4b1d8a05f3e9c7b2d1a4e8f5
# Localização: /home/usuario/.uvlm/.uvlm_dk
# Criada em: 2025-03-15 14:23:11
```

ATENÇÃO
A DEVICE KEY exibida é o identificador derivado, não os bytes brutos da chave. O arquivo .uvlm_dk contém os bytes binários da chave real, que NUNCA deve ser exibido ou compartilhado em texto puro.

20.4 Backup: Exportar a DEVICE KEY

Antes de qualquer operação de risco (formatação, troca de máquina, atualização de SO), exporte a DEVICE KEY para um arquivo seguro:

```shell
uz key-export backup.uvlmkey
# Saída:
# Senha de proteção do backup: ********
# Confirmar: ********
# Backup gerado em: backup.uvlmkey
```

- O arquivo backup.uvlmkey contém a DEVICE KEY criptografada com a senha que você forneceu.
- Guarde este arquivo em local seguro (pen drive offline, gerenciador de senhas, cofre).
- Considere fazer múltiplos backups em locais diferentes.

20.5 Restauração: Importar a DEVICE KEY

Em uma nova máquina (ou após reinstalação), importe o backup para restaurar acesso aos arquivos UZS1:

```shell
uz key-import backup.uvlmkey
# Saída:
# Senha do backup: ********
# AVISO: a DEVICE KEY atual será sobrescrita.
# Confirmar (s/n)? s
# DEVICE KEY restaurada com sucesso.
```

PERIGO
Importar uma DEVICE KEY sobrescreve a atual. Arquivos UZS1 que foram selados após a geração da DEVICE KEY atual (mas antes da importação) ficarão inacessíveis. Ordem importa.

20.6 Cenários Comuns de Uso

20.6.1 Cenário 1: Trabalhar em Múltiplos Computadores

20. No computador principal, exporte a DEVICE KEY: uz key-export minha-chave.uvlmkey
21. Transfira o arquivo para o segundo computador (pen drive, transferência segura).
22. No segundo computador, importe: uz key-import minha-chave.uvlmkey
23. Agora arquivos UZS1 funcionam em ambos os computadores.

20.6.2 Cenário 2: Distribuir Software para Cliente

24. Use formato UZS! (via IDE Export) em vez de UZS1, pois UZS! não depende de DEVICE KEY.
25. Compartilhe o arquivo .uzs e a senha através de canais separados (email + SMS, por exemplo).
26. O cliente pode abrir em qualquer máquina apenas com a senha correta.

20.6.3 Cenário 3: Migração para Nova Máquina

27. Antes de descartar a máquina antiga: uz key-export backup.uvlmkey
28. Instale UpperZetta na máquina nova (a primeira execução gera uma nova DEVICE KEY automaticamente).
29. Importe o backup: uz key-import backup.uvlmkey (sobrescreve a chave nova).
30. Verifique abrindo um .uzs antigo: uz unseal arquivo.uzs.

20.7 Quando NÃO Usar UZS1

- Quando precisar abrir o arquivo em qualquer máquina sem migração de chave → use UZS! (IDE Export).
- Para distribuição massiva (muitos clientes em máquinas diferentes) → UZS! é mais prático.
- Para colaboração em equipe → UZS! evita que cada membro precise importar a mesma DEVICE KEY.

•

PARTE IV

Ferramentas

Linha de comando e ZettaSource IDE

21. CLI Reference

A interface de linha de comando agrupa compilador, runtime, utilitários de selagem e gerenciamento de DEVICE KEY em um único entry point. Esta seção é um guia de referência completo de todos os comandos disponíveis.

21.1 Forma Geral

```
java -cp Main.jar Main <comando> [argumentos] [opções]

# Com alias 'uz' configurado (ver Seção 2.4):
uz <comando> [argumentos] [opções]
```

21.2 Tabela de Comandos

Comando	Propósito
<arquivo.uz>	Compila e executa o arquivo fonte
<arquivo.uzb>	Executa bytecode pré-compilado
build	Compila .uz para .uzb sem executar
seal	Cria arquivo .uzs criptografado
unseal	Restaura .uz ou .uzb a partir de .uzs
key-show	Exibe a DEVICE KEY desta máquina
key-export	Faz backup da DEVICE KEY em arquivo
key-import	Restaura DEVICE KEY a partir de backup
--disasm	Disassembly de bytecode (usado com .uzb)
--version	Exibe versão da UVLM e do compilador
--help	Exibe ajuda resumida

21.3 Comando: Executar Diretamente (.uz)

```
uso

uz programa.uz
```

Este é o comando mais comum durante desenvolvimento: o compilador transforma .uz em .uzb temporário e a UVLM o executa imediatamente. O .uzb não é persistido em disco — fica apenas em memória durante a execução.

21.3.1 Exemplo

```shell
# arquivo: hello.uz
# componente Home { fun render() { System.print("Olá!"); } }

uz hello.uz
# Saída: Olá!
```

21.4 Comando: Executar Bytecode (.uzb)

```uso
uz programa.uzb
```

Quando o programa já foi compilado anteriormente, executar diretamente o .uzb evita o tempo de compilação e é mais rápido. Útil para programas distribuídos a usuários finais ou para rodadas repetidas em produção:

```shell
uz programa.uzb
# Saída do programa, sem mensagens de compilação
```

21.5 Comando: Compilar Sem Executar (build)

```uso
uz build programa.uz [-o saida.uzb]
```

Gera o arquivo .uzb e termina, sem executar. Útil em pipelines de CI/CD ou para testar se o código compila com sucesso:

```shell
uz build app.uz
# Saída:
# Compilando app.uz...
# Bytecode gerado: app.uzb (3.241 bytes)

# Especificando nome do arquivo de saída
uz build app.uz -o build/app.uzb
```

> **NOTA**
> Por padrão, o arquivo .uzb é gerado no mesmo diretório do .uz, com o mesmo nome base. Use -o para personalizar.

21.6 Comando: Disassembly (--disasm)

uso

```
uz programa.uzb --disasm
```

Produz uma representação textual legível do bytecode. Útil para entender o código gerado, depurar problemas de baixo nível ou aprender sobre o modelo de execução da UVLM:

shell

```
uz hello.uzb --disasm
# Saída:
# === COMPONENT Home ===
#   FUNCTION render (locals=0)
#     0000  PUSH_STR     "Olá!"
#     0001  CALL_BUILTIN print
#     0002  RETURN
```

21.7 Comando: Selar (seal)

uso

```
uz seal <arquivo.uz | arquivo.uzb>
```

Aplica criptografia AES-256 e gera um arquivo .uzs. A senha é solicitada interativamente:

shell

```
uz seal app.uz
# Senha: ********
# Confirmar: ********
# Selado em: app.uzs

# Pode-se selar bytecode também:
uz seal app.uzb
# Selado em: app.uzs (a partir do bytecode)
```

ATENÇÃO

Selar via CLI usa o formato UZS1, que vincula o arquivo à DEVICE KEY desta máquina. Para criar um .uzs portável entre computadores, use a opção "Exportar Selado" da ZettaSource IDE (formato UZS!).

21.8 Comando: Desselar (unseal)

```
uso

uz unseal <arquivo.uzs>
```

Solicita a senha e, se correta, restaura o arquivo .uz ou .uzb original:

```shell
uz unseal app.uzs
# Senha: ********
# Restaurado em: app.uz   (ou app.uzb, conforme o caso)
```

21.9 Comandos de DEVICE KEY

21.9.1 key-show

```shell
uz key-show
# DEVICE KEY: a3f7c92e4b1d8a05f3e9c7b2d1a4e8f5
# Localização: /home/usuario/.uvlm/.uvlm_dk
# Criada em: 2025-03-15 14:23:11
```

21.9.2 key-export

```shell
uz key-export backup.uvlmkey
# Senha de proteção: ********
# Confirmar: ********
# Backup gerado: backup.uvlmkey
```

21.9.3 key-import

```shell
uz key-import backup.uvlmkey
# Senha do backup: ********
# AVISO: a DEVICE KEY atual será sobrescrita.
# Confirmar (s/n)? s
# DEVICE KEY restaurada.
```

21.10 Códigos de Saída

A CLI segue convenções Unix de códigos de saída:

Código	Significado
0	Sucesso
1	Erro genérico (mensagem em stderr)
2	Argumentos inválidos ou comando desconhecido
3	Erro de compilação (código UZ-1xxx ou UZ-2xxx)
4	Erro de runtime (código UZ-3xxx)
5	Senha incorreta ao desselar (apenas seal/unseal)
6	DEVICE KEY ausente ou inválida
9	Bytecode corrompido (Block A ≠ espelho de Block B)

21.11 Uso em Scripts e CI/CD

A CLI é facilmente integrável a pipelines de build automatizados. Exemplo de script shell de build com testes simples:

build.sh

```
#!/bin/bash
set -e

echo "[1/3] Compilando..."
uz build src/main.uz -o build/app.uzb

echo "[2/3] Testando execução..."
output=$(uz build/app.uzb)
if [[ "$output" != *"OK"* ]]; then
    echo "Falha: saída não contém OK"
    exit 1
fi

echo "[3/3] Selando para distribuição..."
uz seal build/app.uzb
mv build/app.uzs dist/

echo "Build concluído."
```

22. ZettaSource IDE

A ZettaSource é a IDE oficial para UpperZetta. Construída sobre Electron (interface) e CodeMirror 6 (editor), oferece syntax highlighting nativo da linguagem, compilação one-click, painel de saída integrado e suporte completo aos formatos selados (.uzs).

22.1 Layout da Interface

```
┌──────────────────────────────────────────────────────────┐
│  Menu (Arquivo, Editar, Ver, Executar, Ajuda)            │
├──────────┬───────────────────────────────────────────────┤
│          │                                               │
│ Sidebar  │            Editor (CodeMirror 6)              │
│          │                                               │
│ Arquivos │    componente Home {                          │
│ Símbolos │        fun render() {                         │
│          │            System.print("Olá!");              │
│          │        }                                      │
│          │    }                                          │
│          │                                               │
├──────────┴───────────────────────────────────────────────┤
│  Painel Output UVLM                                      │
│  > Compilando hello.uz...                                │
│  > Bytecode gerado.                                      │
│  > Olá!                                                  │
└──────────────────────────────────────────────────────────┘
```

22.2 Atalhos de Teclado

22.2.1 Arquivo

Atalho	Ação
Ctrl+N	Novo arquivo
Ctrl+O	Abrir arquivo (suporta .uz, .uzb, .uzs)
Ctrl+S	Salvar arquivo atual
Ctrl+Shift+S	Salvar como (escolher novo caminho/nome)
Ctrl+W	Fechar aba atual
Ctrl+Shift+E	Selar arquivo atual como .uzs

22.2.2 Edição

Atalho	Ação
Ctrl+Z	Desfazer

Atalho	Ação
Ctrl+Y	Refazer
Ctrl+X	Recortar linha (sem seleção) ou seleção
Ctrl+C	Copiar
Ctrl+V	Colar
Ctrl+F	Buscar no arquivo
Ctrl+H	Buscar e substituir
Ctrl+/	Comentar/descomentar linha
Ctrl+D	Selecionar próxima ocorrência da palavra
Alt+Up/Down	Mover linha para cima/baixo

22.2.3 Compilação e Execução

Atalho	Ação
F5	Compilar e executar
Ctrl+Shift+B	Apenas compilar (gera .uzb)
Ctrl+F5	Executar sem recompilar (.uzb existente)
Esc	Interromper execução em andamento

22.2.4 Interface

Atalho	Ação
Ctrl+B	Mostrar/ocultar sidebar
Ctrl+	Mostrar/ocultar terminal integrado
Ctrl+,	Abrir configurações
Ctrl+Shift+P	Command Palette (busca de comandos)
Ctrl++ / Ctrl+-	Aumentar/diminuir tamanho da fonte
F11	Modo tela cheia

22.3 Syntax Highlighting

A ZettaSource colore tokens UpperZetta segundo o seguinte esquema (no tema escuro padrão):

Elemento	Cor padrão
Palavras-chave (fun, let, if, while, for)	Vermelho coral
Tipos primitivos (int, str, float, ...)	Azul claro
Strings literais	Verde
Números (int e float)	Laranja
Comentários (// e /* */)	Cinza itálico
Nomes de função (em declaração e chamada)	Laranja dourado
Nomes de classe e componente	Roxo
Builtins (System.print, Math.*)	Ciano
Operadores (+, -, *, /, ==, ...)	Branco
Identificadores comuns	Cinza claro

> **DICA**
> O esquema de cores pode ser customizado em Configurações → Aparência → Tema. Temas claros e variantes alternativas estão disponíveis na galeria.

22.4 Painel Output UVLM

O painel inferior exibe três tipos de saída:

- **Mensagens do compilador:** progresso de compilação, avisos e erros com formato `UZ-XXXX:linha:coluna`.
- **Saída do programa:** tudo que `System.print` gera durante a execução.
- **Mensagens da UVLM:** status de carregamento de bytecode, verificação GLP, término de execução.

22.4.1 Erros Clicáveis

Erros do compilador aparecem como links: clicar em um erro UZ-XXXX:linha:coluna leva o cursor diretamente para a posição do problema no editor. Isso acelera consideravelmente o ciclo de correção:

```
exemplo de saída
```

```
> Compilando app.uz...
> UZ-1023:7:14 expected ';' after expression
> UZ-2105:12:5 'e.const' não permitido fora do escopo global
> Compilação falhou (2 erros)
```

22.5 Recursos do Editor

22.5.1 Auto-complete

A ZettaSource oferece sugestões contextuais para palavras-chave, tipos primitivos, builtins e nomes definidos no arquivo atual (variáveis, funções, classes). Trigger automático ao começar a digitar; manual com Ctrl+Espaço.

22.5.2 Indentação Inteligente

- Ao pressionar Enter dentro de chaves, o editor indenta automaticamente.
- Ao digitar }, o editor recua para alinhar com o { correspondente.
- Tab insere 4 espaços (padrão UpperZetta) — configurável.

22.5.3 Dobra de Código (Code Folding)

Blocos de função, classe e componente podem ser colapsados clicando na seta na margem esquerda. Útil para navegar arquivos grandes:

```
▼ componente Home {        ← clique para colapsar
    fun render() { ... }
}

▶ componente Home { ... }   ← colapsado
```

22.5.4 Multi-cursor

- Alt+Click adiciona cursor adicional na posição clicada.
- Ctrl+D adiciona cursor na próxima ocorrência da palavra atual.
- Permite editar múltiplas linhas simultaneamente.

22.6 Trabalhando com Arquivos .uzs

22.6.1 Abrindo um .uzs

31. `Arquivo → Abrir Arquivo` ().
32. Selecione o arquivo .uzs no diálogo.
33. Insira a senha quando solicitado.
34. O conteúdo descriptografado aparece no editor.

22.6.2 Editando e Salvando

- **Salvar (Ctrl+S):** sobrescreve o .uzs com o conteúdo atualizado, mantendo a mesma senha.
- **Salvar como (Ctrl+Shift+S):** permite escolher novo nome e/ou nova senha.
- **Exportar Selado (Ctrl+Shift+E):** converte qualquer .uz aberto para .uzs no formato UZS! (portável entre máquinas).

22.7 Configurações

Acessível via Ctrl+, ou pelo menu Editar → Preferências. As principais categorias:

Categoria	Configurações principais
Editor	Tamanho de fonte, família, indentação, word wrap
Aparência	Tema (escuro/claro/customizado), cores
Compilação	Diretório de saída padrão, flags do compilador
Atalhos	Remapear comandos de teclado
Linguagem	Verificação ortográfica em comentários, locale
Avançado	Caminho do Main.jar, opções da JVM

22.8 Command Palette

Acionada com Ctrl+Shift+P, a Command Palette é uma busca textual de todos os comandos disponíveis. Em vez de procurar pelos menus, basta digitar o início do comando:

```
Ctrl+Shift+P → digite "selar"
  → Selar Código-Fonte (.uzs)...
  → Selar como Bytecode (.uzs)...
  → Importar arquivo selado

Ctrl+Shift+P → digite "tema"
  → Aparência: Mudar Tema...
  → Aparência: Tema Claro
  → Aparência: Tema Escuro
```

DICA
A Command Palette é geralmente o caminho mais rápido para qualquer ação na IDE. Familiarize-se com ela e você dependerá muito menos do mouse.

22.9 Terminal Integrado

Pressione `Ctrl+` para abrir um terminal acoplado à parte inferior da janela. Útil para invocar comandos da CLI sem alternar para outra janela:

terminal integrado

```
$ uz build src/main.uz
Compilando src/main.uz...
Bytecode gerado: src/main.uzb (4.123 bytes)

$ uz src/main.uzb
Saída do programa...
```

P A R T E V

Prática e Referência

Boas práticas, padrões, performance, FAQ e referência completa

23. Convenções e Style Guide

Esta seção consolida convenções de estilo recomendadas para código UpperZetta. Não são imposições da linguagem — o compilador aceita qualquer formatação válida — mas seguir estas diretrizes torna o código mais legível e facilita colaboração entre desenvolvedores.

23.1 Princípios Gerais

- **Clareza acima de concisão:** código que pode ser entendido em 5 segundos é melhor que código compacto que exige 30 segundos para decifrar.
- **Consistência:** siga o mesmo estilo em todo o projeto. Mistura de convenções é mais confusa que qualquer convenção isolada.
- **Explicitude:** UpperZetta valoriza tipagem explícita. Estenda essa filosofia para nomenclatura, comentários e estrutura do código.
- **Linhas curtas:** mantenha linhas com até ~80 caracteres. Linhas longas dificultam revisão lado-a-lado e leitura em telas estreitas.

23.2 Indentação e Espaçamento

23.2.1 Indentação

- Use 4 espaços por nível.
- Nunca misture tabs e espaços no mesmo arquivo.
- Continue indentação consistente em estruturas aninhadas.

```
// ☑ correto

componente Home {
    fun render() {
        if (condicao) {
            for (let i.int = 0; i < 10; i = i + 1) {
                System.print(i);
            }
        }
    }
}
```

23.2.2 Espaçamento

- Espaço único após vírgulas: `soma(a, b, c)`, não `soma(a,b,c)`.
- Espaço em torno de operadores binários: `a + b`, `x == y`.
- Sem espaço imediatamente após (ou antes de): `fun f(x.int)`, não `fun f( x.int )`.
- Sem espaço entre nome de função e parêntese de chamada: `soma(2, 3)`, não `soma (2, 3)`.

- Linha em branco entre métodos para legibilidade.
- Duas linhas em branco entre classes/componentes para separação visual.

23.3 Posicionamento de Chaves

UpperZetta segue o estilo K&R (chave de abertura na mesma linha):

```
// ☑ recomendado

if (x > 0) {
    System.print("positivo");
} else {
    System.print("não positivo");
}

fun calcular(a.int, b.int) >> int {
    return a + b;
}
```

```
// ✕ evitar

if (x > 0)
{
    System.print("positivo");
}
else
{
    System.print("não positivo");
}
```

23.4 Nomenclatura

Construção	Convenção	Exemplo positivo	Exemplo negativo
Variável local	camelCase	precoTotal, idadeUsuario	preco_total, idadeusuario
Parâmetro	camelCase	a, b, valorMaximo	A, valor_maximo
Constante	UPPER_SNAKE	PI, MAX_TENTATIVAS	pi, MaxTentativas
Função	camelCase	calcularMedia, salvar	CalcularMedia, salvar_dados
Predicado (retorna boolean)	eh/tem + verbo	ehAdulto, temPermissao	adulto, permissao
Acessor	obter + nome	obterNome, obterIdade	pegarNome, n
Classe	PascalCase	Usuario, ContaCorrente	usuario, conta_corrente

Construção	Convenção	Exemplo positivo	Exemplo negativo
Componente	PascalCase	Home, DashboardPrincipal	home, dashboard
Pacote	lowercase.dot	app.util, br.com.empresa	App.Util, BR_COM

23.5 Diretrizes de Tamanho

- Funções com mais de 30-40 linhas geralmente fazem demais. Considere extrair partes.
- Classes/componentes com mais de 200 linhas indicam responsabilidades demais. Refatore separando em classes menores.
- Arquivos com mais de 500 linhas são candidatos a serem divididos (mesmo que UZ v1.0 não suporte imports cross-arquivo, dividir é boa preparação para o futuro).
- Aninhamento profundo (> 3 níveis) é sinal para refatorar com early returns ou extração de método.

23.6 Comentários: Quando e Como

23.6.1 Bons Comentários

- Explicam decisões de design não óbvias do código ("por que escolhi este algoritmo").
- Documentam suposições do código ("este parâmetro deve ser positivo").
- Marcam workarounds ou limitações conhecidas ("// HACK: ", "// TODO: ").
- Cabeçalhos de funções públicas: descrevem propósito, parâmetros e retorno.

```
// ☑ comentário útil

// Usamos busca linear em vez de binária porque
// o array nunca passa de 10 elementos na prática.
fun encontrar(arr.array, n.int, alvo.int) >> int {
    for (let i.int = 0; i < n; i = i + 1) {
        if (arr[i] == alvo) { return i; }
    }
    return -1;
}
```

23.6.2 Comentários a Evitar

- Comentários óbvios que apenas repetem o código.
- Comentários desatualizados que não refletem o código atual.
- Código comentado sem justificativa (use controle de versão para isso).
- "Decoração" excessiva (// ========== etc. em todo lugar).

```
// ❌ ruído inútil
```

```
// Variável i com valor inicial 0
let i.int = 0;

// Adiciona 1 a i
i = i + 1;

// Imprime i
System.print(i);
```

23.7 Cabeçalhos de Função

Para funções públicas e não triviais, considere documentação no estilo:

```
/*
 * Calcula o valor com juros compostos aplicados.
 *
 * Parâmetros:
 *   principal — valor inicial (deve ser > 0)
 *   taxa      — taxa por período (ex: 0.05 = 5%)
 *   meses     — número de períodos (deve ser >= 0)
 *
 * Retorno:
 *   Valor final acumulado
 */
fun calcularJurosCompostos(principal.float, taxa.float, meses.int) >> float {
    let resultado.float = principal;
    for (let i.int = 0; i < meses; i = i + 1) {
        resultado = resultado * (1.0 + taxa);
    }
    return resultado;
}
```

23.8 Organização Interna do Arquivo

Quando um arquivo tem várias construções, organize-as nesta ordem para facilitar leitura:

35. Comentário de cabeçalho (descrição do arquivo, autor, data).
36. Declaração de pacote.
37. Constantes globais (e.const), agrupadas por tema.
38. Funções globais auxiliares (puras antes de impuras).
39. Classes (class.public), uma por região, com métodos públicos antes dos privados.
40. Componentes (Home por último, geralmente).

24. Padrões e Anti-padrões

Esta seção catálogo padrões úteis (idiomas que valem a pena adotar) e anti-padrões (práticas comuns mas problemáticas) específicos do contexto UpperZetta.

24.1 Padrões Úteis

24.1.1 Função Fábrica para Inicialização

Como UpperZetta não tem construtores, usar funções fábrica isola a inicialização e torna o código de criação reutilizável:

```
fun criarUsuario(nome.str, idade.int) >> Usuario {
    let u.Usuario = new Usuario();
    u.nome = nome;
    u.idade = idade;
    u.ativo = true;
    return u;
}

componente Home {
    fun render() {
        let u1.Usuario = criarUsuario("Ana", 30);
        let u2.Usuario = criarUsuario("Bruno", 25);
        // sem repetição de código de inicialização
    }
}
```

24.1.2 Early Return para Reduzir Aninhamento

Em vez de aninhar várias condições, retorne cedo para casos especiais:

```
// ❌ aninhado
```

```
fun calcularDesconto(preco.float, ehCliente.boolean) >> float {
    if (preco > 0.0) {
        if (ehCliente == true) {
            return preco * 0.9;
        } else {
            return preco;
        }
    } else {
        return 0.0;
    }
}
```

```
// ☑ early return
```

```
fun calcularDesconto(preco.float, ehCliente.boolean) >> float {
    if (preco <= 0.0) { return 0.0; }
    if (ehCliente == false) { return preco; }
    return preco * 0.9;
}
```

24.1.3 Componente como Orquestrador

Mantenha o componente Home enxuto: ele coordena, mas não implementa lógica de domínio. Toda lógica vive em classes ou funções globais reutilizáveis.

24.1.4 Constantes para Limites e Configurações

```
e.const TAMANHO_MAX_NOME.int = 50;
e.const TENTATIVAS_LOGIN.int = 3;
e.const TIMEOUT_SEGUNDOS.int = 30;
e.const TAXA_IMPOSTO.float = 0.18;
```

DICA
Constantes nomeadas tornam o código autodocumentado. "TAXA_IMPOSTO" diz mais que "0.18" hard-coded em três lugares diferentes.

24.1.5 Predicados Compostos como Funções

Quando uma condição combina muitas verificações, extraia para uma função com nome descritivo:

```
// Em vez de espalhar a lógica:
if (usuario.ativo == true && usuario.idade >= 18 && usuario.confirmado == true) {
    // ...
}

// Extraia:
fun podeAcessar(u.Usuario) >> boolean {
    return u.ativo == true && u.idade >= 18 && u.confirmado == true;
}

if (podeAcessar(usuario)) {
    // ...
}
```

24.1.6 Iteração com Constante de Tamanho

Como UZ não tem operador de tamanho de array, declare o tamanho explicitamente como constante quando possível:

```
e.const N_NOTAS.int = 5;

componente Home {
    fun render() {
        let notas.array = [8.5, 7.0, 9.5, 6.5, 8.0];
        let soma.float = 0.0;

        for (let i.int = 0; i < N_NOTAS; i = i + 1) {
            soma = soma + notas[i];
        }

        let media.float = soma / 5.0;
        System.print("Média: " + media);
    }
}
```

24.2 Anti-padrões a Evitar

24.2.1 Variáveis Mutáveis Excessivas

Reatribuir a mesma variável muitas vezes torna o fluxo difícil de seguir. Prefira variáveis locais imutáveis (declaradas e nunca reatribuídas) sempre que possível:

```
// ❌ confuso

fun processar(x.int) >> int {
    let r.int = x;
    r = r + 10;
    r = r * 2;
    r = r - 5;
    r = r / 2;
    return r;
}
```

```
// ☑ mais claro

fun processar(x.int) >> int {
    let comAjuste.int = x + 10;
    let dobrado.int = comAjuste * 2;
    let comDesconto.int = dobrado - 5;
    return comDesconto / 2;
}
```

24.2.2 Comparação Booleana Redundante

```
// ❌ verboso

if (ativo == true) { ... }
if (admin == false) { ... }
```

```
// ☑ idiomático

if (ativo) { ... }
if (admin == false) { ... }   // OK manter quando enfatiza negação
```

> **NOTA**
> Em UpperZetta, ambas as formas funcionam, mas if (booleanVar) é mais idiomático para o caso positivo. Para negação, == false ou ! (se houver no futuro) é uma escolha estilística.

24.2.3 Magic Numbers

```
// ❌ valores misteriosos

if (idade >= 18) { ... }
if (tentativas > 3) { ... }
let preco.float = total * 0.18;
```

```
// ☑ nomes descritivos

e.const IDADE_ADULTO.int = 18;
e.const MAX_TENTATIVAS.int = 3;
e.const TAXA_IMPOSTO.float = 0.18;

if (idade >= IDADE_ADULTO) { ... }
if (tentativas > MAX_TENTATIVAS) { ... }
let preco.float = total * TAXA_IMPOSTO;
```

24.2.4 Funções Que Fazem Demais

Uma função que valida, calcula, formata, imprime e salva simultaneamente é difícil de testar e reutilizar. Separe responsabilidades:

```
// Em vez de uma função monstra, divida:
fun validar(dados.Pedido) >> boolean { ... }
fun calcular(dados.Pedido) >> float { ... }
fun formatar(valor.float) >> str { ... }
fun salvar(dados.Pedido) { ... }
```

```
// Compor é direto:
componente Home {
    fun render() {
        let p.Pedido = ...;
        if (validar(p)) {
            let valor.float = calcular(p);
            System.print(formatar(valor));
            salvar(p);
        }
    }
}
```

24.2.5 Profundidade de Aninhamento Excessiva

4 ou mais níveis de aninhamento são sinal de alerta. Refatore com early returns, funções auxiliares ou inversão de condições.

24.2.6 Nomes Cripticos

- x, y, z, t, tmp em escopo amplo: aceitável apenas em loops curtos.
- Abreviações ambíguas: pls (please? plus? plans?).
- Sufixos numéricos sem semântica: usuario1, usuario2 (use array ou nomes descritivos).

25. Performance e Otimização

UpperZetta v1.0 prioriza simplicidade e correção sobre velocidade extrema. A UVLM não realiza otimizações sofisticadas como JIT ou inlining agressivo. Esta seção descreve como o desempenho típico se compara, quando se preocupar com performance e quais práticas ajudam.

25.1 Características de Desempenho

- **Inicialização:** partida da JVM + carregamento UVLM custa ~200-500 ms. Não é adequado para programas que precisam responder em microssegundos.
- **Compilação:** programas pequenos (<1000 linhas) compilam em poucos ms. Programas maiores escalam linearmente.
- **Execução:** interpretação em pilha sem JIT — esperar performance entre Python e código JVM compilado.
- **Memória:** overhead da JVM + heap UVLM. Programas mínimos consomem ~30-50 MB; cresce conforme estruturas alocadas.

25.2 Quando Otimizar

41. Apenas após medir: faça um programa funcionar primeiro, otimize depois com dados.
42. Se o programa é interativo (usuário aguarda resposta): perceba latências > 100 ms.
43. Se processa volumes grandes (milhares de iterações em loops aninhados): atente para complexidade algorítmica.
44. Para scripts pequenos ou tarefas em batch, otimização raramente compensa o esforço.

> **DICA**
> "Otimização prematura é a raiz de todo o mal" (Donald Knuth). UpperZetta v1.0 é mais lenta que C ou Java compilados — aceite isso e foque em escrever código correto e claro.

25.3 Boas Práticas de Performance

25.3.1 Evite Recriar Objetos em Loops

```
// ❌ cria 1000 objetos
```

```
for (let i.int = 0; i < 1000; i = i + 1) {
    let buf.StringBuilder = new StringBuilder();
    buf.append(i);
    System.print(buf.toString());
}
```

```
// ☑ cria um, reutiliza
```

```
let buf.StringBuilder = new StringBuilder();
for (let i.int = 0; i < 1000; i = i + 1) {
    buf.clear();   // (suposição: método reset)
    buf.append(i);
    System.print(buf.toString());
}
```

> **NOTA**
> StringBuilder é hipotético — UZ v1.0 não traz biblioteca padrão de utilitários. Use o exemplo conceitualmente.

25.3.2 Cuide da Concatenação Em Loops

Concatenação repetida com + cria várias strings intermediárias. Para textos pequenos é irrelevante; para milhares de iterações faz diferença mensurável:

```
// Para muitas concatenações, considere acumular em variáveis
```

```
// e fazer uma só concatenação final.

// Em vez disso (cria string nova a cada iteração):
let resultado.str = "";
for (let i.int = 0; i < 100; i = i + 1) {
    resultado = resultado + "linha " + i + " ";
}

// Pode ser que a UVLM otimize internamente, mas se medir
// pior performance, mantenha o resultado parcial e imprima
// dentro do loop:
for (let i.int = 0; i < 100; i = i + 1) {
    System.print("linha " + i);
}
```

25.3.3 Recursão Profunda Evite

Como UVLM não otimiza tail calls, recursão profunda estoura a pilha. Para profundidades > ~5000, prefira loop:

```
// ❌ overflow para n grande

fun fatorial(n.int) >> int {
    if (n <= 1) { return 1; }
    return n * fatorial(n - 1);
}
```

```
// ☑ sem risco de overflow

fun fatorial(n.int) >> int {
    let resultado.int = 1;
    for (let i.int = 2; i <= n; i = i + 1) {
        resultado = resultado * i;
    }
    return resultado;
}
```

25.3.4 Cuidado com Algoritmos Quadráticos

Loops aninhados com mesmo array ($O(n^2)$) ficam dolorosos rápido:

Tamanho n	Operações em O(n)	Operações em $O(n^2)$
100	100	10.000
1.000	1.000	1.000.000
10.000	10.000	100.000.000
100.000	100.000	10.000.000.000 (impraticável)

25.3.5 Carregue Dados Uma Vez

Se um cálculo é caro e o resultado não muda, calcule uma vez e armazene em variável local:

```
// ❌ recalcula em todo loop

for (let i.int = 0; i < calcularLimite(); i = i + 1) {
    // calcularLimite() chamado 1000x se retornar 1000
}
```

```
// ☑ calcula uma vez

let limite.int = calcularLimite();
for (let i.int = 0; i < limite; i = i + 1) {
    // calcularLimite() chamado uma só vez
}
```

25.4 Medindo Performance

A versão atual de UpperZetta não tem profiler nativo. Para medir tempo de execução em alto nível, use a CLI do sistema:

```shell
# Linux/macOS
time uz programa.uz

# Saída:
# real    0m0.421s
# user    0m0.398s
# sys     0m0.022s
```

- "real" é o tempo total observado (wall clock).
- "user" é tempo de CPU em modo usuário (código do programa).
- "sys" é tempo gasto em chamadas de sistema.

25.5 Tradeoffs do Formato GLP

O formato GLP (com Block A + espelho B) duplica o tamanho do bytecode em disco. Isso afeta:

- Espaço em disco: arquivos .uzb são ~2x maiores que seriam em formatos compactos.
- Tempo de carregamento: leitura e verificação adicionam alguns ms ao startup.
- Memória: durante carregamento, ambos os blocos coexistem temporariamente.

> **NOTA**
> Para programas grandes onde startup importa, considere selar para .uzs (que aplica AES e tem efeito de compressão), ou compactar com gzip externo se proteção não é necessária.

26. Limitações Conhecidas

Esta seção lista, de forma honesta e completa, o que UpperZetta v1.0 NÃO suporta. Conhecer as limitações antecipadamente evita frustração e facilita planejar arquiteturas viáveis dentro das capacidades da linguagem.

26.1 Linguagem

- **Sem break/continue:** loops não têm comandos de salto. Use flags ou condições compostas.
- **Sem switch/match:** use cadeias de if/else if.
- **Sem operadores compostos:** não há `+=`, `-=`, `++`, `--`. Use `x = x + 1`.
- **Sem operador ternário:** não há `cond ? a : b`. Use if/else explícito.
- **Sem operador unário !:** negação booleana via `== false`.
- **Sem inferência de tipo:** toda variável e parâmetro precisa de anotação explícita.
- **Sem genéricos:** arrays e classes não são parametrizados por tipo.
- **Sem herança:** classes não herdam de outras classes. Use composição.
- **Sem interfaces:** não há contratos abstratos formais entre classes.
- **Sem visibilidade granular:** não há `private`/`protected`. Tudo em uma classe é acessível.
- **Sem construtores:** inicialização via atribuições manuais ou função fábrica.
- **Sem destructors/finalizers:** liberação de recursos é por GC apenas.
- **Sem exceções:** não há try/catch. Erros de runtime abortam o programa.
- **Sem closures ou funções de primeira classe:** funções não são valores, não podem ser passadas como argumento.
- **Sem operações com strings (split, indexOf, etc.):** strings são opacas, suportam apenas concatenação com +.
- **Sem parsing nativo de string para número:** não há `Integer.parse` embutido.
- **Sem operador de tamanho de array:** você precisa rastrear o tamanho separadamente.

26.2 Bibliotecas e I/O

- **Sem leitura do teclado:** programas não interagem com input do usuário.

- **Sem leitura/escrita de arquivos:** programas operam apenas com dados literais ou hard-coded.
- **Sem rede:** nada de HTTP, sockets, conexões.
- **Sem random/data/hora:** não há acesso a recursos não-determinísticos do sistema.
- **Sem biblioteca matemática avançada:** apenas operadores aritméticos básicos. Sem `sqrt`, `sin`, `log`.
- **Sem variáveis de ambiente:** programas não conseguem ler `$HOME`, `$PATH` etc.

26.3 Modularidade

- **Sem imports:** não há mecanismo para usar código de outro arquivo .uz.
- **Um arquivo por compilação:** projetos multi-arquivo precisam ser concatenados ou mantidos como arquivos isolados.
- **Sem gerenciamento de pacotes:** não há repositório oficial de bibliotecas, nem npm/pip/maven equivalente.

26.4 Compilação e Runtime

- **Sem JIT:** UVLM interpreta bytecode sem compilação just-in-time, limitando velocidade.
- **Sem otimização de tail call:** recursão profunda estoura a pilha.
- **Sem multithreading:** programas são single-threaded.
- **GC simples:** coleta de lixo da JVM subjacente, sem controle fino.
- **Limite de 256 slots por frame:** funções com excesso de variáveis locais falham na compilação.
- **Compatibilidade unidirecional:** bytecode gerado pela versão N pode não rodar em versões N-1.

26.5 Ferramentas

- **Sem debugger interativo:** depuração é por `System.print` (printf debugging).
- **Sem framework de testes nativo:** testes são funções comuns que invocam asserts manuais.
- **Sem profiler:** medições de performance são feitas externamente (`time` do shell).
- **Sem linter formal:** a IDE oferece syntax highlighting, mas não análise estática profunda.
- **Sem formatter automático:** formatação é responsabilidade do desenvolvedor seguindo o style guide.

> **NOTA**
> Esta lista pode parecer extensa, mas é normal para uma linguagem em v1.0. Versões futuras devem endereçar as limitações mais críticas progressivamente. Para projetos sérios em UZ atual, planeje sua arquitetura considerando estas restrições desde o início.

27. Erros Comuns e Troubleshooting

Esta seção cataloga os erros mais frequentes encontrados ao escrever UpperZetta, com diagnóstico e correção. Use como primeiro recurso quando um erro de compilação ou runtime aparecer.

27.1 Erros Léxicos / Sintáticos

27.1.1 Ponto-e-vírgula faltando

```
UZ-1023:5:14 expected ';' after expression
```

```
let x.int = 10
let y.int = 20;
```

Causa: faltou ; ao final da linha 1.

Correção: adicione ; em "let x.int = 10;".

27.1.2 Caractere inválido

```
UZ-1003:3:9 invalid character
```

```
let nome.str = 'Maria';   // aspas simples não são aceitas
```

Causa: UZ aceita apenas aspas duplas para strings.

Correção: troque por aspas duplas: "Maria".

27.1.3 Identificador com acento

```
UZ-1004:2:5 invalid identifier
```

```
let usuário.str = "João";
```

Causa: identificadores devem ser ASCII (sem á, é, ç, etc.).

Correção: use "usuario" sem acento. (O valor "João" como string permanece válido.)

27.1.4 String não fechada

```
UZ-1005:7:13 unterminated string literal
```

```
let msg.str = "Olá, mundo;
```

Causa: faltou aspa de fechamento.

Correção: feche a string: "Olá, mundo";

27.2 Erros Semânticos

27.2.1 Tipo não anotado

```
UZ-2001:2:5 type annotation required
```

```
let x = 10;             // ERRO
fun soma(a, b) >> int {  // ERRO em a e b
    return a + b;
}
```

Causa: variáveis e parâmetros precisam de .tipo.

```
let x.int = 10;
fun soma(a.int, b.int) >> int {
    return a + b;
}
```

27.2.2 Classe sem .public

```
UZ-2010:1:1 expected 'class.public'
```

```
class Pessoa {
    let nome.str;
}
```

Correção: use class.public Pessoa { ... }.

27.2.3 e.const dentro de função

```
UZ-2105:3:5 'e.const' só permitido no escopo global
```

```
fun calcular() {
    e.const PI.float = 3.14;   // ERRO
    return PI;
}
```

Correção: declare PI antes da função, no escopo global do arquivo.

27.2.4 Inicialização em class.public

```
UZ-2210:2:5 inicialização não permitida em class
```

```
class.public Produto {
```

```
    let preco.float = 9.99;   // ERRO
}
```

Correção: declare sem inicializar, ou mova para um componente:

```
class.public Produto {
    let preco.float;            // ☑ sem init
}

componente Home {
    let preco.float = 9.99;     // ☑ permitido em componente
}
```

27.2.5 Tipos incompatíveis em atribuição

```
UZ-2050:3:5 type mismatch: int = str
```

```
let n.int = 10;
n = "texto";    // ERRO
```

27.2.6 Função sem return em todos os caminhos

```
UZ-2060: function may not return on all paths
```

```
fun classificar(x.int) >> str {
    if (x > 0) { return "positivo"; }
    // se x <= 0, não retorna
}
```

Correção: garanta um return em todos os caminhos:

```
fun classificar(x.int) >> str {
    if (x > 0) { return "positivo"; }
    return "não-positivo";
}
```

27.3 Erros de Runtime

27.3.1 Índice de array fora dos limites

```
UZ-3001:4:12 array index 10 out of bounds (size 5)
```

```
let arr.array = [1, 2, 3, 4, 5];
let x.int = arr[10];   // ERRO em runtime
```

Correção: valide o índice antes de acessar:

```
if (i >= 0 && i < 5) {
```

```
    let x.int = arr[i];
}
```

27.3.2 Divisão por zero (int)

```
UZ-3010: integer division by zero
```

```
let a.int = 10;
let b.int = 0;
let c.int = a / b;   // ERRO em runtime
```

27.3.3 Acesso a campo de objeto null

```
UZ-3020: null pointer access
```

```
class.public Caixa {
    let valor.int;
}

componente Home {
    fun render() {
        let c.Caixa;          // c é null
        c.valor = 10;          // ERRO em runtime
    }
}
```

Correção: instancie antes de usar: let c.Caixa = new Caixa();

27.3.4 Stack overflow por recursão profunda

```
UZ-3030: stack overflow
```

```
fun loop(n.int) >> int {
    return loop(n + 1);    // recursão sem caso base
}
```

27.4 Problemas com .uzs

27.4.1 Senha incorreta

```
decryption failed: wrong password or DEVICE KEY
```

```
uz unseal arquivo.uzs
# Senha: ********
# decryption failed.
```

- Verifique se a senha está exatamente correta (atenção a caps lock).
- Se o arquivo foi criado em outra máquina via CLI seal (UZS1), você precisa também da DEVICE KEY de origem.

27.4.2 DEVICE KEY ausente

```
UZ-9020: DEVICE KEY not found or corrupted
```

```
uz unseal arquivo.uzs
# UZ-9020: DEVICE KEY not found or corrupted
```

Causa: o arquivo .uvlm/.uvlm_dk não existe ou está corrompido.

- Se você tem backup: uz key-import backup.uvlmkey.
- Se não tem backup e o arquivo é UZS1: o conteúdo é irrecuperável.

27.5 Problemas Genéricos

27.5.1 "java: command not found"

- Verifique instalação: java -version.
- Adicione Java ao PATH conforme instruções do seu sistema operacional.
- Recarregue o terminal após alterar o PATH.

27.5.2 "Bytecode version mismatch"

Você tentou executar um .uzb gerado em uma versão diferente da UVLM:

- Recompile o .uz na versão atual: uz build programa.uz.
- Atualize a UVLM para a versão que gerou o .uzb.

27.5.3 Programa "trava" sem saída

- Possível loop infinito: revise condições de while/for.
- Use Ctrl+C no terminal para interromper.
- Adicione System.print de debug em pontos críticos do fluxo.

28. Códigos de Erro

Tabela de referência completa dos códigos UZ-XXXX. Útil para localizar rapidamente a causa de um erro e a correção apropriada.

28.1 Convenção de Numeração

- **UZ-1xxx:** erros léxicos e sintáticos (problemas de gramática).
- **UZ-2xxx:** erros semânticos (compile-time, depois do parsing).
- **UZ-3xxx:** erros de runtime (durante execução na UVLM).
- **UZ-9xxx:** erros de sistema, arquivo ou criptografia.

28.2 UZ-1xxx — Léxicos / Sintáticos

Código	Significado	Causa típica
UZ-1001	unexpected token	Token fora de lugar
UZ-1003	invalid character	Caractere não-ASCII em código
UZ-1004	invalid identifier	Nome com caractere ilegal
UZ-1005	unterminated string literal	String sem aspa de fechamento
UZ-1010	expected expression	Operador sem operando
UZ-1023	expected ';' after expression	Ponto-e-vírgula ausente
UZ-1024	expected '{'	Bloco não iniciado
UZ-1025	expected '}'	Bloco não fechado
UZ-1030	expected ')'	Parêntese não fechado
UZ-1045	expected type annotation '.tipo'	Variável/parâmetro sem tipo
UZ-1050	expected '>>' before return type	Função sem separador de retorno

28.3 UZ-2xxx — Semânticos

Código	Significado	Causa típica
UZ-2001	type annotation required	Falta .tipo em declaração
UZ-2010	expected 'class.public'	Sem .public após class
UZ-2020	undefined identifier	Variável/função não declarada
UZ-2030	duplicate declaration	Mesmo nome declarado duas vezes
UZ-2050	type mismatch	Atribuição de tipo incompatível
UZ-2060	function may not return on all paths	Função sem return em algum if/else
UZ-2070	method called on non-object	Chamada .metodo() em primitivo
UZ-2105	e.const fora do escopo global	Constante dentro de função
UZ-2210	campo inicializado em class.public	Init de campo em class (use componente)
UZ-2300	wrong number of arguments	Chamada com n errado de argumentos

Código	Significado	Causa típica
UZ-2310	wrong argument type	Tipo de argumento incompatível

28.4 UZ-3xxx — Runtime

Código	Significado	Causa típica
UZ-3001	array index out of bounds	Índice >= tamanho ou < 0
UZ-3010	integer division by zero	a / b com b == 0
UZ-3020	null pointer access	Acesso a campo de objeto não instanciado
UZ-3030	stack overflow	Recursão muito profunda
UZ-3040	out of memory	Heap esgotado (alocação massiva)
UZ-3050	invalid cast	Conversão entre tipos incompatíveis

28.5 UZ-9xxx — Sistema

Código	Significado	Causa típica
UZ-9001	corrupt bytecode (GLP mismatch)	Block A ≠ espelho de B
UZ-9002	unsupported bytecode version	UVLM antiga lendo bytecode novo
UZ-9010	file not found	Caminho inválido
UZ-9011	permission denied	Sem permissão de leitura/escrita
UZ-9020	DEVICE KEY not found or corrupted	.uvlm_dk ausente ou inválido
UZ-9030	decryption failed	Senha incorreta no .uzs
UZ-9031	invalid .uzs format	Magic byte não reconhecido

NOTA
Códigos exatos podem variar entre versões. Esta tabela cobre os mais comuns; a referência canônica é a saída de "uz --help-errors" no compilador instalado.

29. Migrando de Outras Linguagens

Esta seção é um guia rápido para desenvolvedores vindos de outras linguagens. Mostra os "tradutores mentais" mais úteis para acelerar a aprendizagem.

29.1 De JavaScript / TypeScript

JavaScript / TypeScript	UpperZetta
let x = 10;	let x.int = 10;
let nome: string = "Ana";	let nome.str = "Ana";
const PI = 3.14;	e.const PI.float = 3.14;
function soma(a, b) { return a+b; }	fun soma(a.int, b.int) >> int { return a + b; }
class Pessoa { ... }	class.public Pessoa { ... }
new Pessoa()	new Pessoa() (igual)
console.log(x)	System.print(x)
for (let i = 0; i < 10; i++)	for (let i.int = 0; i < 10; i = i + 1)
arr.length	(não existe — rastreie tamanho separadamente)
arr.push(x)	(não existe — arrays são tamanho fixo na prática)
const obj = { a: 1, b: 2 }	(não há literais de objeto — use class.public)

29.1.1 Diferenças que Pegam de Surpresa

- **Sem typeof:** tipos são checados em compilação, não há introspecção em runtime.
- **Sem === vs ==:** UZ tem apenas `==` (semântica de comparação por valor para primitivos, por referência para objetos).
- **Sem null/undefined distinção:** apenas null para objetos não-instanciados.
- **Sem callbacks/funções como valores:** arquitete sem first-class functions.
- **Sem JSON nativo:** programas não consomem nem produzem JSON.

29.2 De Python

Python	UpperZetta
x = 10	let x.int = 10;
PI = 3.14	e.const PI.float = 3.14;

Python	UpperZetta
def soma(a, b): return a + b	fun soma(a.int, b.int) >> int { return a + b; }
class Pessoa: ...	class.public Pessoa { ... }
print(x)	System.print(x);
if x > 0: ... else: ...	if (x > 0) { ... } else { ... }
for i in range(10):	for (let i.int = 0; i < 10; i = i + 1)
lista = [1, 2, 3]	let lista.array = [1, 2, 3];
len(lista)	(rastreie tamanho manualmente)
lista.append(x)	(arrays são tamanho fixo)
"texto".split(",")	(sem operações de string)

29.2.1 Diferenças Importantes vs. Python

- Tipagem estática obrigatória (Python tem hints opcionais).
- Chaves em vez de indentação semântica (a indentação é convenção, não exige).
- Ponto-e-vírgula obrigatório no final de instruções.
- Sem list comprehensions, generators, decorators.
- Sem dunder methods (__init__, __str__, etc.) — composição manual.

29.3 De Java

Java	UpperZetta
int x = 10;	let x.int = 10;
final double PI = 3.14;	e.const PI.float = 3.14;
public class Pessoa { ... }	class.public Pessoa { ... }
new Pessoa()	new Pessoa() (igual)
System.out.println(x)	System.print(x);
public static void main(...)	componente Home { fun render() { ... } }
for (int i = 0; i < 10; i++)	for (let i.int = 0; i < 10; i = i + 1)
String[] nomes = {"a", "b"}	let nomes.array = ["a", "b"];
nomes.length	(rastreie manualmente)
try { ... } catch { ... }	(sem exceções)

Java	UpperZetta
interface Imprimivel { ... }	(sem interfaces)
extends Animal	(sem herança — use composição)

29.3.1 Diferenças vs. Java

- Sem main() — uso de componente Home com método render().
- Sem visibilidade granular (public/protected/private).
- Anotação de tipo após o nome com ponto, não antes do nome.
- Sem sobrecarga de métodos (cada nome é único na classe).
- Tipos primitivos minúsculos (int, str), não maiúsculos (Integer, String).

29.4 De C / C++

C / C++	UpperZetta
int x = 10;	let x.int = 10;
const float PI = 3.14;	e.const PI.float = 3.14;
int soma(int a, int b) { ... }	fun soma(a.int, b.int) >> int { ... }
printf("%d", x)	System.print(x);
int arr[10];	let arr.array = [0, 0, 0, ..., 0];
arr[5] = 42;	arr[5] = 42; (igual)
struct Pessoa { ... };	class.public Pessoa { ... }
Pessoa* p = malloc(...)	let p.Pessoa = new Pessoa();
free(p)	(GC automático)
#include <math.h>	(sem bibliotecas)

29.4.1 Diferenças vs. C/C++

- Sem ponteiros explícitos (objetos são referências automáticas).
- Sem aritmética de ponteiros.
- GC automático, sem free/delete.
- Sem pré-processador (#define, #include).
- Sem header files — tudo em um arquivo .uz.

29.5 Estratégia Geral de Migração

45. Comece com programas pequenos (Hello World, calculadora) para internalizar a sintaxe.
46. Foque na disciplina de tipagem explícita — é o que mais difere de linguagens dinâmicas.
47. Aceite que muitas funcionalidades familiares não existem em UZ v1.0 — refatore para usar apenas o que existe.
48. Use a separação class.public vs. componente como fator organizador principal.
49. Como UZ não tem coleções avançadas, crie suas próprias estruturas (lista encadeada, fila) com class.public quando necessário.

30. Perguntas Frequentes (FAQ)

Esta seção responde dúvidas recorrentes sobre UpperZetta, organizadas por tema.

30.1 Sobre a Linguagem

P: Posso usar UpperZetta em produção?

R: UpperZetta v1.0 é estável para projetos pequenos a médios, especialmente os que se beneficiam da proteção .uzs. Para sistemas críticos com alta concorrência, integração extensa com APIs externas ou requisitos rigorosos de performance, considere se as limitações da Seção 26 são aceitáveis para seu caso.

P: Por que tipagem estática obrigatória?

R: Tipagem estática captura erros em compilação, torna código autodocumentado e melhora performance da máquina virtual. UpperZetta deliberadamente não oferece inferência para forçar a explicitude — o trade-off é mais verbosidade em troca de mais clareza.

P: Quando usar class.public vs componente?

R: class.public para modelos de dados e lógica de domínio reutilizável (Usuario, Pedido, Produto). componente para o ponto de entrada da aplicação (sempre Home) e unidades de UI/orquestração com estado próprio. Em programas pequenos, você pode ter apenas o componente Home.

P: Há plano para herança/interfaces nas próximas versões?

R: O design atual prioriza composição. Versões futuras podem introduzir interfaces ou traits, mas herança clássica não está nos planos imediatos. Se você sente falta de OO clássico, modele com composição: um campo do tipo "Animal" dentro de "Cachorro" replica boa parte do que herança ofereceria.

30.2 Sobre Compilação e Execução

P: Por que o .uzb é tão grande?

R: Pelo formato GLP: o arquivo contém o bytecode mais seu espelho para verificação de integridade autocontida. Aceite o overhead em troca da garantia de detecção de corrupção sem checksums externos.

P: Posso compilar em uma máquina e executar em outra?

R: Sim para .uzb (puramente bytecode UVLM, portável). Para .uzs no formato UZS! (gerado pela IDE, exportação) também sim. Para .uzs no formato UZS1 (CLI seal) você precisa migrar a DEVICE KEY (ver Seção 20).

P: O bytecode .uzb é compatível entre versões da UVLM?

R: Dentro da mesma linha (1.x), sim — versões maiores (UVLM 1.5) executam bytecode de versões menores (1.0). Não há garantia de compatibilidade entre linhas (1.x → 2.x). Para distribuição de longo prazo, mantenha o .uz original e recompile quando atualizar.

P: Posso descompilar um .uzb para recuperar o .uz?

R: Não diretamente. O comando --disasm produz bytecode legível, mas não reconstrói o código fonte original (perde-se nomes locais, comentários, formatação). Mantenha o .uz original em controle de versão.

P: O programa é multi-threaded?

R: Não. UpperZetta v1.0 é single-threaded. A UVLM executa instruções sequencialmente em uma única thread.

30.3 Sobre Selagem e Segurança

P: Quão segura é a criptografia .uzs?

R: Usa AES-256 com PBKDF2-SHA512 e milhares de iterações de derivação de chave. Resistente a ataques de força bruta com senhas razoavelmente fortes. Não é à prova de senhas fracas — use senhas longas e únicas.

P: Esqueci a senha do meu .uzs. Como recupero?

R: Não há mecanismo de recuperação. A senha é a única forma de descriptografar; sem ela, o conteúdo é matematicamente irrecuperável. Mantenha senhas em gerenciador confiável.

P: Posso usar o mesmo .uzs em vários computadores?

R: Depende do formato: UZS! (IDE Export) sim, qualquer máquina com a senha pode abrir. UZS1 (CLI seal) requer também a DEVICE KEY do computador onde foi selado — você precisa exportar e importar a chave nas máquinas alvo.

P: A DEVICE KEY pode vazar?

R: É um arquivo binário em ~/.uvlm/.uvlm_dk com permissões restritas. Em sistemas multi-usuário ou expostos a malware, é vulnerável como qualquer outro arquivo de credencial. Faça backup criptografado em local seguro.

30.4 Sobre Ferramentas e Workflow

P: É possível debugar passo-a-passo?

R: Não há debugger interativo na v1.0. Use System.print estrategicamente nos pontos críticos do fluxo (printf debugging). Para testes, escreva funções que invocam asserts manuais.

P: Como organizo um projeto com muitos arquivos?

R: A v1.0 não suporta imports cross-arquivo. Você pode (a) manter tudo em um único .uz (viável até alguns milhares de linhas), (b) usar um script externo de build que concatena vários .uz antes de compilar, (c) aguardar futuras versões com sistema de módulos.

P: Posso integrar UpperZetta com bibliotecas Java?

R: Não diretamente. A UVLM roda sobre JVM mas não expõe interop com classes Java arbitrárias. UZ programas são autocontidos.

P: Existe um repositório público de bibliotecas UZ?

R: A v1.0 não tem ecossistema de pacotes. A comunidade pode compartilhar arquivos .uz por meios convencionais (Git, fórum oficial), mas não há instalador automatizado.

30.5 Sobre Comparações

P: UpperZetta é mais lenta que Java?

R: Sim, em geral. UVLM é interpretada sem JIT. Para benchmarks computacionais intensos, Java compilado é tipicamente 5-20x mais rápido. Para programas comuns (lógica de negócio, fluxo de dados pequenos), a diferença raramente importa.

P: Posso usar UZ para web ou mobile?

R: A v1.0 não tem APIs para isso. UZ é usável para utilitários de linha de comando, scripts batch, programas didáticos e ferramentas internas. Aplicações web/mobile requerem outras pilhas.

P: Por que escolheria UZ em vez de Python?

R: Por proteção de código (.uzs), tipagem estática mais rigorosa, ou para projetos onde a sintaxe explícita ajuda no ensino/avaliação. Python ganha em ecossistema, bibliotecas e flexibilidade.

31. Glossário

Termos técnicos usados nesta documentação, em ordem alfabética.

AES-256
Advanced Encryption Standard com chave de 256 bits. Algoritmo de criptografia simétrica usado para proteger arquivos .uzs.

AST (Abstract Syntax Tree)
Árvore de sintaxe abstrata. Representação intermediária produzida pelo parser, que reflete a estrutura gramatical do código fonte.

Block A / Block B
No formato GLP, Block A contém o bytecode propriamente dito; Block B é o espelho byte-a-byte de A, usado para verificação de integridade.

Bytecode
Código binário de baixo nível executado por uma máquina virtual. Em UpperZetta, refere-se ao formato gerado pelo compilador e armazenado em arquivos .uzb.

CodeMirror
Biblioteca de editor de texto baseada em web, usada como núcleo do editor da ZettaSource IDE.

Compilador
Programa que transforma código fonte em outra forma (geralmente bytecode ou código de máquina). Em UZ, transforma .uz em .uzb.

Componente
Construção declarada com a palavra-chave "componente". Diferente de class.public, suporta inicialização de campos. O componente Home é o ponto de entrada do programa.

Constant Pool
Tabela armazenada no bytecode que contém literais (strings, números) referenciados pelas instruções. Permite deduplicação e referência por índice.

CodeGen
Fase do compilador responsável por gerar bytecode a partir da AST. Sigla de Code Generator.

DEVICE KEY
Chave criptográfica única gerada automaticamente na primeira execução da UVLM em um computador. Armazenada em ~/.uvlm/.uvlm_dk e usada no esquema UZS1 de selagem.

Disassembly
Processo inverso à compilação: traduzir bytecode de volta para uma forma legível humanamente. Em UZ, ativado com --disasm.

Frame de chamada
Estrutura criada pela UVLM ao invocar uma função/método. Contém slots locais (256 por frame), pilha de operandos e endereço de retorno.

GLP (Generative Logic Palindrome)

Formato binário do bytecode UpperZetta. Característica distintiva: arquivo é palindrômico (Block A + espelho B), permitindo verificação de integridade sem checksum externo.

Heap

Região de memória onde objetos e arrays são alocados dinamicamente. Em UVLM, gerenciado pelo coletor de lixo da JVM subjacente.

Lexer

Componente do compilador que transforma código fonte em sequência de tokens. Também chamado de scanner ou tokenizer.

Magic byte

Bytes iniciais de um arquivo binário que identificam seu formato. Em UZ, "UZB\0" para bytecode, "UZS!" e "UZS1" para arquivos selados.

Máquina virtual de pilha (Stack VM)

Modelo de máquina virtual onde instruções operam sobre uma pilha de operandos. UVLM, JVM e CLR são exemplos. Diferente de máquinas baseadas em registradores.

Parser

Componente do compilador que valida a estrutura sintática do código e produz a AST a partir do fluxo de tokens do lexer.

PBKDF2

Password-Based Key Derivation Function 2. Algoritmo que deriva chaves criptográficas a partir de senhas, com muitas iterações para resistir a ataques de força bruta.

Pilha de operandos

Estrutura LIFO usada por máquinas de pilha para armazenar valores temporários durante a execução de instruções. Distinta da pilha de chamadas.

Selagem

Processo de criptografar um arquivo .uz ou .uzb com senha, produzindo um .uzs. Inverso: desselagem.

Slot local

Posição numerada (0 a 255) em um frame de chamada que armazena uma variável local ou parâmetro. Em métodos, slot 0 contém a referência ao objeto.

Tipagem estática

Modelo onde tipos são verificados em compilação. UpperZetta exige anotação explícita; tipos não são inferidos.

UVLM

UpperZetta Virtual Language Machine. Máquina virtual de pilha que executa bytecode .uzb. Implementada em Java, distribuída em Main.jar.

UZS! e UZS1

Variantes do formato selado. UZS! usa AES-256-GCM e independe de DEVICE KEY (gerado pela IDE). UZS1 usa AES-256-CBC e depende de DEVICE KEY (gerado pela CLI seal).

ZettaSource
IDE oficial para UpperZetta. Construída em Electron + CodeMirror 6.

32. Referência Rápida

Esta seção é uma "cola" para consulta rápida durante o desenvolvimento. Use como cheat sheet.

32.1 Palavras-chave Reservadas

A linguagem reserva os seguintes identificadores. Não podem ser usados como nomes de variáveis, funções, classes ou componentes:

`package`	`in`	`e.const`	`let`
`fun`	`return`	`class.public`	`componente`
`if`	`else`	`while`	`for`
`new`	`true`	`false`	`null`
`System.print`	`>>`	`&&`	`||`

32.2 Sintaxe de Tipos

Todas as anotações de tipo seguem o padrão nome.tipo (ponto, não dois-pontos):

`nome.int`	`nome.float`	`nome.str`	`nome.boolean`
`nome.array`	`nome.void`	`nome.NomeClasse`	`nome.Componente`

32.3 Sintaxes Canônicas

`e.const NOME.tipo = valor;`	Constante global (única forma)
`let nome.tipo;`	Variável sem inicialização
`let nome.tipo = valor;`	Variável com inicialização
`fun nome(p.tipo) { ... }`	Função sem retorno (void)
`fun nome(p.tipo) >> tipoRetorno { ... }`	Função com retorno tipado
`class.public Nome { ... }`	Classe (sem init de campos)

`componente Nome { ... }`	Componente (campos podem ter init)
`new NomeClasse()`	Instanciação de objeto
`obj.campo`	Acesso a campo
`obj.metodo(args)`	Invocação de método
`arr[indice]`	Acesso a elemento de array
`package in nome.do.pacote;`	Declaração de pacote
`return expressao;`	Retorno de função
`System.print(expressao);`	Saída para terminal

32.4 Controle de Fluxo

`if (cond) { ... }`	Condicional simples
`if (cond) { ... } else { ... }`	Condicional com alternativa
`if (a) { } else if (b) { } else { }`	Cadeia de condicionais
`while (cond) { ... }`	Loop com pré-condição
`for (let i.int = 0; i < n; i = i + 1)`	Loop indexado

32.5 Operadores

32.5.1 Aritméticos

`+`	`-`	`*`	`/`

32.5.2 Relacionais

`==`	`!=`	`<`	`>`	`<=`	`>=`

32.5.3 Lógicos e Atribuição

`&&`	`||`	`=`

32.6 Tipos e Valores Padrão

Tipo	Valor padrão	Exemplos de literais
int	0	42, -7, 0

Tipo	Valor padrão	Exemplos de literais
float	0.0	3.14, -0.5, 1.0
str	""	"Olá", "texto"
boolean	false	true, false
array	[]	[1, 2, 3], ["a", "b"]
Classe	null	new Pessoa()

32.7 Atalhos da IDE

Ação	Atalho
Novo arquivo	Ctrl+N
Abrir	Ctrl+O
Salvar	Ctrl+S
Compilar e executar	F5
Apenas compilar	Ctrl+Shift+B
Selar (.uzs)	Ctrl+Shift+E
Command Palette	Ctrl+Shift+P
Buscar	Ctrl+F
Substituir	Ctrl+H
Comentar linha	Ctrl+/

32.8 CLI Resumida

`uz arquivo.uz`	Compila e executa
`uz arquivo.uzb`	Executa bytecode
`uz build arquivo.uz`	Apenas compila
`uz arquivo.uzb --disasm`	Disassembly do bytecode
`uz seal arquivo.uz`	Cria .uzs criptografado
`uz unseal arquivo.uzs`	Restaura a partir de .uzs
`uz key-show`	Exibe DEVICE KEY
`uz key-export backup.key`	Backup da DEVICE KEY

`uz key-import backup.key`	Restaura DEVICE KEY
`uz --version`	Versão da UVLM
`uz --help`	Ajuda resumida

32.9 Códigos de Erro Mais Comuns

Código	Significado curto
UZ-1023	Ponto-e-vírgula faltando
UZ-1045	Tipo não anotado
UZ-2001	Anotação de tipo obrigatória
UZ-2105	e.const fora do escopo global
UZ-2210	Init de campo em class.public
UZ-3001	Índice fora dos limites do array
UZ-3010	Divisão inteira por zero
UZ-3030	Stack overflow (recursão profunda)
UZ-9020	DEVICE KEY ausente
UZ-9030	Senha incorreta no .uzs

32.9 Gramática Resumida

```
programa     → (pacote)? (const | fun | class | componente)*
pacote       → "package in" IDENT ("." IDENT)* ";"
const        → "e.const" IDENT "." IDENT "=" expr ";"
variavel     → "let" IDENT "." IDENT ("=" expr)? ";"
fun          → "fun" IDENT "(" params ")" (">>" IDENT)? "{" stmt* "}"
params       → (IDENT "." IDENT ("," IDENT "." IDENT)*)?
class        → "class.public" IDENT "{" (variavel | fun)* "}"
componente   → "componente" IDENT "{" (variavel | fun)* "}"
stmt         → if | while | for | print | return | let | expr ";"
if           → "if" "(" expr ")" bloco ("else" bloco)?
while        → "while" "(" expr ")" bloco
for          → "for" "(" (let | expr ";") expr ";" expr ")" bloco
print        → "System.print" "(" expr ")" ";"
return       → "return" expr? ";"
bloco        → "{" stmt* "}" | stmt
expr         → assign
assign       → log ("=" assign)?
log          → eq (("&&" | "||") eq)*
```

```
eq          → rel (("==" | "!=") rel)*
rel         → add (("<"|">"|"<="|">=") add)*
add         → mul (("+" | "-") mul)*
mul         → call (("*" | "/") call)*
call        → prim (("(" args ")" | "." IDENT | "[" expr "]"))*
prim        → NUMBER | STRING | "true" | "false"
            | "new" IDENT "()"
            | IDENT | "(" expr ")"
            | "[" (expr ("," expr)*)? "]"
```

33. Exemplos Completos

Esta seção apresenta exemplos completos e funcionais que ilustram diferentes facetas da linguagem. Comece pelos mais simples e progrida para os mais elaborados.

33.1 Hello World

O programa mais simples possível:

hello.uz

```
package in exemplo.hello;

componente Home {
    fun render() {
        System.print("Olá, Mundo!");
    }
}
```

33.2 Calculadora Simples

Demonstra funções globais com parâmetros e retorno:

calculadora.uz

```
package in exemplo.calc;

fun somar(a.int, b.int) >> int { return a + b; }
fun subtrair(a.int, b.int) >> int { return a - b; }
fun multiplicar(a.int, b.int) >> int { return a * b; }
fun dividir(a.float, b.float) >> float { return a / b; }

componente Home {
    fun render() {
        System.print("=== Calculadora ===");
        System.print("Soma:          " + somar(10, 5));
```

```
        System.print("Subtração:     " + subtrair(10, 5));
        System.print("Multiplicação: " + multiplicar(10, 5));
        System.print("Divisão:       " + dividir(10.0, 3.0));
    }
}
```

33.3 Classificação com if/else

Demonstra cadeias de condicionais e funções com retorno de string:

classificacao.uz

```
package in exemplo.classif;

fun classificarNota(nota.float) >> str {
    if (nota >= 9.0) { return "Excelente"; }
    else if (nota >= 7.0) { return "Bom"; }
    else if (nota >= 5.0) { return "Regular"; }
    else { return "Insuficiente"; }
}

componente Home {
    fun render() {
        let notas.array = [9.5, 7.2, 5.0, 3.8];

        for (let i.int = 0; i < 4; i = i + 1) {
            let n.float = notas[i];
            System.print("Nota " + n + " - " + classificarNota(n));
        }
    }
}
```

33.4 Loop e Acumulador

Demonstra for, while e cálculo iterativo:

soma_progressiva.uz

```
package in exemplo.loop;

fun somarAte(n.int) >> int {
    let soma.int = 0;
    for (let i.int = 1; i <= n; i = i + 1) {
        soma = soma + i;
    }
    return soma;
}

componente Home {
```

```
    fun render() {
        System.print("Soma de 1 a 10:  " + somarAte(10));
        System.print("Soma de 1 a 100: " + somarAte(100));

        // Demonstrando while também
        System.print("--- Contagem regressiva ---");
        let x.int = 5;
        while (x > 0) {
            System.print("x = " + x);
            x = x - 1;
        }
    }
}
```

33.5 Classe com Métodos

Demonstra encapsulamento de dados e comportamento em uma classe:

retangulo.uz

```
package in exemplo.classes;

class.public Retangulo {
    let largura.float;
    let altura.float;

    fun area() >> float {
        return largura * altura;
    }

    fun perimetro() >> float {
        return 2.0 * (largura + altura);
    }

    fun ehQuadrado() >> boolean {
        return largura == altura;
    }
}

componente Home {
    fun render() {
        let r.Retangulo = new Retangulo();
        r.largura = 4.0;
        r.altura = 6.0;

        System.print("Largura:    " + r.largura);
        System.print("Altura:     " + r.altura);
        System.print("Área:       " + r.area());
        System.print("Perímetro:  " + r.perimetro());
        System.print("É quadrado: " + r.ehQuadrado());
```

```
    }
}
```

33.6 Recursão: Fibonacci

Demonstra recursão dupla com caso base claro:

fibonacci.uz

```
package in exemplo.fibonacci;

fun fib(n.int) >> int {
    if (n <= 1) { return n; }
    return fib(n - 1) + fib(n - 2);
}

componente Home {
    fun render() {
        for (let i.int = 0; i <= 10; i = i + 1) {
            System.print("fib(" + i + ") = " + fib(i));
        }
    }
}
```

33.7 Trabalhando com Arrays

Demonstra criação, iteração e processamento de arrays com função utilitária:

arrays.uz

```
package in exemplo.arrays;

e.const TAMANHO.int = 5;

fun maior(arr.array, n.int) >> float {
    let m.float = arr[0];
    for (let i.int = 1; i < n; i = i + 1) {
        if (arr[i] > m) {
            m = arr[i];
        }
    }
    return m;
}

fun menor(arr.array, n.int) >> float {
    let m.float = arr[0];
    for (let i.int = 1; i < n; i = i + 1) {
        if (arr[i] < m) {
            m = arr[i];
```

```
        }
    }
    return m;
}

fun media(arr.array, n.int) >> float {
    let soma.float = 0.0;
    for (let i.int = 0; i < n; i = i + 1) {
        soma = soma + arr[i];
    }
    return soma / 5.0;
}

componente Home {
    fun render() {
        let valores.array = [3.5, 8.1, 2.9, 7.4, 5.0];

        System.print("Valores armazenados:");
        for (let i.int = 0; i < TAMANHO; i = i + 1) {
            System.print("  [" + i + "] = " + valores[i]);
        }

        System.print("---");
        System.print("Maior: " + maior(valores, TAMANHO));
        System.print("Menor: " + menor(valores, TAMANHO));
        System.print("Média: " + media(valores, TAMANHO));
    }
}
```

33.8 Componente com Estado

Demonstra inicialização de campos e métodos auxiliares dentro do componente:

contador.uz

```
package in exemplo.estado;

componente Home {
    let valor.int = 0;
    let nome.str = "Contador Principal";

    fun render() {
        System.print(nome);
        incrementar();
        incrementar();
        incrementar();
        System.print("Valor após 3 incrementos: " + valor);

        decrementar();
        System.print("Valor após decremento: " + valor);
```

```
        resetar();
        System.print("Valor após reset: " + valor);
    }

    fun incrementar() {
        valor = valor + 1;
    }

    fun decrementar() {
        valor = valor - 1;
    }

    fun resetar() {
        valor = 0;
    }
}
```

33.9 Composição de Classes

Demonstra como uma classe pode conter outra como campo:

endereco_cliente.uz

```
package in exemplo.composicao;

class.public Endereco {
    let rua.str;
    let numero.int;
    let cidade.str;
    let cep.str;

    fun formatado() >> str {
        return rua + ", " + numero + " - " + cidade + " (" + cep + ")";
    }
}

class.public Cliente {
    let nome.str;
    let idade.int;
    let endereco.Endereco;

    fun resumo() >> str {
        return nome + " (" + idade + " anos) — " + endereco.formatado();
    }
}

fun criarCliente(nome.str, idade.int) >> Cliente {
    let c.Cliente = new Cliente();
```

```
    c.nome = nome;
    c.idade = idade;
    c.endereco = new Endereco();
    return c;
}

componente Home {
    fun render() {
        let c.Cliente = criarCliente("Felipe", 21);
        c.endereco.rua = "Rua das Acácias";
        c.endereco.numero = 123;
        c.endereco.cidade = "Franca";
        c.endereco.cep = "14400-000";

        System.print(c.resumo());
    }
}
```

33.10 Sistema de Notas Acadêmicas

Programa mais elaborado integrando classes, funções globais, arrays e lógica de negócio:

notas_academicas.uz

```
package in exemplo.notas;

e.const N_PROVAS.int = 4;
e.const NOTA_APROVACAO.float = 6.0;

class.public Aluno {
    let nome.str;
    let provas.array;
    let aprovado.boolean;

    fun calcularMedia() >> float {
        let soma.float = 0.0;
        for (let i.int = 0; i < N_PROVAS; i = i + 1) {
            soma = soma + provas[i];
        }
        return soma / 4.0;
    }

    fun avaliar() {
        let m.float = calcularMedia();
        if (m >= NOTA_APROVACAO) {
            aprovado = true;
        } else {
            aprovado = false;
        }
    }
```

```
    fun relatorio() >> str {
        let media.float = calcularMedia();
        let status.str = "";
        if (aprovado) {
            status = "APROVADO";
        } else {
            status = "REPROVADO";
        }
        return nome + " | Média: " + media + " | " + status;
    }
}

fun criarAluno(nome.str, p1.float, p2.float, p3.float, p4.float) >> Aluno {
    let a.Aluno = new Aluno();
    a.nome = nome;
    a.provas = [p1, p2, p3, p4];
    a.avaliar();
    return a;
}

componente Home {
    fun render() {
        System.print("=== Sistema de Avaliação ===");
        System.print("Critério: média >= " + NOTA_APROVACAO);
        System.print("");

        let a1.Aluno = criarAluno("Ana",     8.5, 7.0, 9.0, 8.0);
        let a2.Aluno = criarAluno("Bruno",   5.0, 4.5, 5.5, 6.0);
        let a3.Aluno = criarAluno("Carlos",  9.0, 9.5, 8.5, 9.0);

        System.print(a1.relatorio());
        System.print(a2.relatorio());
        System.print(a3.relatorio());
    }
}
```

33.11 Mini-Banco com Conta Corrente

Exemplo final integrando vários conceitos: encapsulamento, validações, mutação de estado, função fábrica:

conta_corrente.uz

```
package in exemplo.banco;

e.const LIMITE_NEGATIVO.float = -500.0;

class.public Conta {
```

```
    let titular.str;
    let saldo.float;
    let numero.int;

    fun depositar(valor.float) {
        if (valor <= 0.0) {
            System.print("[" + titular + "] Depósito inválido: " + valor);
            return;
        }
        saldo = saldo + valor;
        System.print("[" + titular + "] Depósito de R$ " + valor + ". Saldo: R$ " +
saldo);
    }

    fun sacar(valor.float) {
        if (valor <= 0.0) {
            System.print("[" + titular + "] Saque inválido: " + valor);
            return;
        }
        let novoSaldo.float = saldo - valor;
        if (novoSaldo < LIMITE_NEGATIVO) {
            System.print("[" + titular + "] Limite excedido. Saque negado.");
            return;
        }
        saldo = novoSaldo;
        System.print("[" + titular + "] Saque de R$ " + valor + ". Saldo: R$ " +
saldo);
    }

    fun consultarSaldo() {
        System.print("[" + titular + "] Saldo atual: R$ " + saldo);
    }
}

fun abrirConta(titular.str, numero.int, saldoInicial.float) >> Conta {
    let c.Conta = new Conta();
    c.titular = titular;
    c.numero = numero;
    c.saldo = saldoInicial;
    System.print("Conta " + numero + " aberta para " + titular + " com R$ " +
saldoInicial);
    return c;
}

componente Home {
    fun render() {
        System.print("=== Sistema Bancário UZ ===");
        System.print("");

        let c1.Conta = abrirConta("Felipe", 1001, 1000.0);
        let c2.Conta = abrirConta("Joana",  1002, 250.0);
```

```
        System.print("");
        c1.depositar(500.0);
        c1.sacar(200.0);
        c1.consultarSaldo();

        System.print("");
        c2.sacar(300.0);   // entra no negativo, mas dentro do limite
        c2.sacar(700.0);   // excede limite, será negado
        c2.consultarSaldo();
    }
}
```

33.12 Próximos Passos

Após dominar os exemplos acima, recomenda-se:

50. Recriar os exemplos sem olhar, testando seu domínio dos padrões.
51. Modificar os exemplos para adicionar funcionalidades (ex: extrato com array de transações na conta).
52. Combinar conceitos: ex: sistema bancário que classifica clientes em categorias, ou sistema de notas que ordena por média.
53. Selar (.uzs) seus exemplos para testar a CLI completa de selagem/desselagem.
54. Examinar o bytecode com --disasm para entender o modelo de execução em baixo nível.

Fim da Documentação

UpperZetta v1.0 · UVLM Runtime · ZettaSource IDE v2.0.0

Para suporte, consulte a Seção 27 (Troubleshooting) e a Seção 30 (FAQ).

PARTE VI

Apêndices

Tutoriais, cookbook, estruturas de dados, algoritmos e mais

Apêndice A · Tutorial Completo: Sistema de Biblioteca

Este apêndice constrói um sistema completo de gerenciamento de biblioteca, do zero, em etapas progressivas. Cada etapa adiciona uma funcionalidade nova, demonstrando como UpperZetta evolui de programas simples para aplicações de domínio real. É o tutorial mais longo desta documentação e cobre quase todos os recursos da linguagem em contexto prático.

> **NOTA**
> Cada etapa do tutorial mostra o código completo do arquivo. Você pode acompanhar copiando o código a cada passo e executando para ver o comportamento.

A.1 Etapa 1 — Modelo Básico de Livro

Começamos com a entidade central: um livro com título, autor e disponibilidade. Apenas um livro, sem coleção ainda:

biblioteca.uz — etapa 1

```
package in tutorial.biblioteca;

class.public Livro {
    let titulo.str;
    let autor.str;
    let disponivel.boolean;

    fun resumo() >> str {
        let status.str = "";
        if (disponivel) {
            status = "[disponível]";
        } else {
            status = "[emprestado]";
        }
        return titulo + " — " + autor + " " + status;
    }
}

componente Home {
    fun render() {
        let l.Livro = new Livro();
        l.titulo = "O Cortiço";
        l.autor = "Aluísio Azevedo";
        l.disponivel = true;

        System.print(l.resumo());
    }
```

```
}
```

Saída esperada: O Cortiço — Aluísio Azevedo [disponível]

A.2 Etapa 2 — Função Fábrica e Múltiplos Livros

Criar livros manualmente é repetitivo. Vamos extrair uma função fábrica e armazenar vários livros em um array:

biblioteca.uz — etapa 2

```
package in tutorial.biblioteca;

e.const N_LIVROS.int = 4;

class.public Livro {
    let titulo.str;
    let autor.str;
    let disponivel.boolean;

    fun resumo() >> str {
        let status.str = "";
        if (disponivel) { status = "[disponível]"; }
        else { status = "[emprestado]"; }
        return titulo + " — " + autor + " " + status;
    }
}

fun criarLivro(titulo.str, autor.str) >> Livro {
    let l.Livro = new Livro();
    l.titulo = titulo;
    l.autor = autor;
    l.disponivel = true;
    return l;
}

componente Home {
    fun render() {
        let acervo.array = [
            criarLivro("O Cortiço", "Aluísio Azevedo"),
            criarLivro("Dom Casmurro", "Machado de Assis"),
            criarLivro("Memórias Póstumas", "Machado de Assis"),
            criarLivro("Vidas Secas", "Graciliano Ramos")
        ];

        System.print("=== Acervo da Biblioteca ===");
        for (let i.int = 0; i < N_LIVROS; i = i + 1) {
            let l.Livro = acervo[i];
            System.print((i + 1) + ". " + l.resumo());
```

```
        }
    }
}
```

A.3 Etapa 3 — Operações: Emprestar e Devolver

Adicionamos métodos para alterar o estado de disponibilidade. Notem o cuidado para não emprestar livros já emprestados:

biblioteca.uz — etapa 3 (apenas a classe Livro modificada)

```
class.public Livro {
    let titulo.str;
    let autor.str;
    let disponivel.boolean;

    fun resumo() >> str {
        let status.str = "";
        if (disponivel) { status = "[disponível]"; }
        else { status = "[emprestado]"; }
        return titulo + " — " + autor + " " + status;
    }

    fun emprestar() >> boolean {
        if (disponivel == false) {
            System.print("[!] " + titulo + " já está emprestado.");
            return false;
        }
        disponivel = false;
        System.print("[ok] " + titulo + " emprestado.");
        return true;
    }

    fun devolver() >> boolean {
        if (disponivel) {
            System.print("[!] " + titulo + " já estava disponível.");
            return false;
        }
        disponivel = true;
        System.print("[ok] " + titulo + " devolvido.");
        return true;
    }
}
```

Use no render():

```
fun render() {
    let acervo.array = [/* ... */];

    let l1.Livro = acervo[0];
    l1.emprestar();        // [ok] O Cortiço emprestado.
    l1.emprestar();        // [!] O Cortiço já está emprestado.
    l1.devolver();         // [ok] O Cortiço devolvido.
    l1.devolver();         // [!] O Cortiço já estava disponível.
}
```

A.4 Etapa 4 — Modelo de Usuário

Adicionamos a entidade Usuario. Cada usuário tem nome, ID e um array fixo de livros emprestados:

biblioteca.uz — etapa 4 (adições)

```
e.const MAX_EMPRESTIMOS.int = 3;

class.public Usuario {
    let nome.str;
    let id.int;
    let emprestimos.array;
    let nEmprestimos.int;

    fun pegarLivro(l.Livro) >> boolean {
        if (nEmprestimos >= MAX_EMPRESTIMOS) {
            System.print("[!] " + nome + " atingiu limite de empréstimos.");
            return false;
        }
        if (l.emprestar() == false) {
            return false;
        }
        emprestimos[nEmprestimos] = l;
        nEmprestimos = nEmprestimos + 1;
        return true;
    }

    fun listarEmprestimos() {
        System.print(nome + " (#" + id + ") tem " + nEmprestimos + " livro(s):");
        for (let i.int = 0; i < nEmprestimos; i = i + 1) {
            let l.Livro = emprestimos[i];
            System.print("  - " + l.titulo);
        }
    }
}

fun criarUsuario(nome.str, id.int) >> Usuario {
```

```
    let u.Usuario = new Usuario();
    u.nome = nome;
    u.id = id;
    u.emprestimos = [null, null, null];   // capacidade fixa
    u.nEmprestimos = 0;
    return u;
}
```

A.5 Etapa 5 — Componente Como Orquestrador

O componente Home agora orquestra acervo + usuários. Note como ele permanece enxuto, deixando lógica nas classes:

biblioteca.uz — etapa 5 (Home reescrito)

```
componente Home {
    fun render() {
        // ===== ACERVO =====
        let acervo.array = [
            criarLivro("O Cortiço", "Aluísio Azevedo"),
            criarLivro("Dom Casmurro", "Machado de Assis"),
            criarLivro("Memórias Póstumas", "Machado de Assis"),
            criarLivro("Vidas Secas", "Graciliano Ramos")
        ];

        // ===== USUÁRIOS =====
        let ana.Usuario = criarUsuario("Ana", 1001);
        let bruno.Usuario = criarUsuario("Bruno", 1002);

        // ===== OPERAÇÕES =====
        System.print("=== Empréstimos ===");
        ana.pegarLivro(acervo[0]);
        ana.pegarLivro(acervo[1]);
        bruno.pegarLivro(acervo[1]);   // já emprestado
        bruno.pegarLivro(acervo[2]);
        bruno.pegarLivro(acervo[3]);

        System.print("");
        System.print("=== Estado dos Usuários ===");
        ana.listarEmprestimos();
        bruno.listarEmprestimos();

        System.print("");
        System.print("=== Estado do Acervo ===");
        for (let i.int = 0; i < N_LIVROS; i = i + 1) {
            let l.Livro = acervo[i];
            System.print(l.resumo());
        }
    }
}
```

A.6 Etapa 6 — Devolução com Remoção do Array

Quando um livro é devolvido, precisamos removê-lo da lista de empréstimos do usuário. Como UZ não tem função de remoção, implementamos manualmente compactando o array:

Método devolverLivro em Usuario

```
fun devolverLivro(l.Livro) >> boolean {
    // Procura o livro no array de empréstimos
    let pos.int = -1;
    for (let i.int = 0; i < nEmprestimos; i = i + 1) {
        let lv.Livro = emprestimos[i];
        if (lv.titulo == l.titulo) {
            pos = i;
        }
    }

    if (pos == -1) {
        System.print("[!] " + nome + " não tem " + l.titulo);
        return false;
    }

    if (l.devolver() == false) {
        return false;
    }

    // Compacta: shift dos elementos posteriores
    for (let i.int = pos; i < nEmprestimos - 1; i = i + 1) {
        emprestimos[i] = emprestimos[i + 1];
    }
    emprestimos[nEmprestimos - 1] = null;
    nEmprestimos = nEmprestimos - 1;

    return true;
}
```

A.7 Etapa 7 — Estatísticas e Relatórios

Adicionamos funções globais que computam estatísticas sobre o acervo:

```
fun contarDisponiveis(acervo.array, n.int) >> int {
    let contador.int = 0;
    for (let i.int = 0; i < n; i = i + 1) {
        let l.Livro = acervo[i];
        if (l.disponivel) {
            contador = contador + 1;
        }
    }
    return contador;
}
```

```
fun contarEmprestados(acervo.array, n.int) >> int {
    return n - contarDisponiveis(acervo, n);
}

fun relatorioAcervo(acervo.array, n.int) {
    let disp.int = contarDisponiveis(acervo, n);
    let emp.int = n - disp;
    System.print("=== RELATÓRIO ===");
    System.print("Total no acervo:  " + n);
    System.print("Disponíveis:      " + disp);
    System.print("Emprestados:      " + emp);
    let pct.float = (disp * 100) / n;
    System.print("Taxa disponível:  " + pct + "%");
}
```

A.8 Versão Final Completa

Juntando tudo, a versão final do arquivo (consolidada para você copiar e testar):

biblioteca.uz — VERSÃO FINAL

```
package in tutorial.biblioteca;

e.const N_LIVROS.int = 4;
e.const MAX_EMPRESTIMOS.int = 3;

class.public Livro {
    let titulo.str;
    let autor.str;
    let disponivel.boolean;

    fun resumo() >> str {
        let s.str = "[disponível]";
        if (disponivel == false) { s = "[emprestado]"; }
        return titulo + " — " + autor + " " + s;
    }

    fun emprestar() >> boolean {
        if (disponivel == false) {
            System.print("[!] " + titulo + " já está emprestado.");
            return false;
        }
        disponivel = false;
        return true;
    }

    fun devolver() >> boolean {
        if (disponivel) {
            System.print("[!] " + titulo + " já estava disponível.");
```

```
            return false;
        }
        disponivel = true;
        return true;
    }
}

class.public Usuario {
    let nome.str;
    let id.int;
    let emprestimos.array;
    let nEmprestimos.int;

    fun pegarLivro(l.Livro) >> boolean {
        if (nEmprestimos >= MAX_EMPRESTIMOS) {
            System.print("[!] " + nome + " no limite.");
            return false;
        }
        if (l.emprestar() == false) { return false; }
        emprestimos[nEmprestimos] = l;
        nEmprestimos = nEmprestimos + 1;
        System.print("[ok] " + nome + " pegou " + l.titulo);
        return true;
    }

    fun listarEmprestimos() {
        System.print(nome + " (#" + id + "): " + nEmprestimos + " livros");
        for (let i.int = 0; i < nEmprestimos; i = i + 1) {
            let l.Livro = emprestimos[i];
            System.print("  - " + l.titulo);
        }
    }
}

fun criarLivro(t.str, a.str) >> Livro {
    let l.Livro = new Livro();
    l.titulo = t;
    l.autor = a;
    l.disponivel = true;
    return l;
}

fun criarUsuario(n.str, id.int) >> Usuario {
    let u.Usuario = new Usuario();
    u.nome = n;
    u.id = id;
    u.emprestimos = [null, null, null];
    u.nEmprestimos = 0;
    return u;
}
```

```
fun contarDisponiveis(acervo.array, n.int) >> int {
    let c.int = 0;
    for (let i.int = 0; i < n; i = i + 1) {
        let l.Livro = acervo[i];
        if (l.disponivel) { c = c + 1; }
    }
    return c;
}

componente Home {
    fun render() {
        let acervo.array = [
            criarLivro("O Cortiço", "Aluísio Azevedo"),
            criarLivro("Dom Casmurro", "Machado de Assis"),
            criarLivro("Memórias Póstumas", "Machado de Assis"),
            criarLivro("Vidas Secas", "Graciliano Ramos")
        ];

        let ana.Usuario = criarUsuario("Ana", 1001);
        let bruno.Usuario = criarUsuario("Bruno", 1002);

        System.print("=== Empréstimos ===");
        ana.pegarLivro(acervo[0]);
        ana.pegarLivro(acervo[1]);
        bruno.pegarLivro(acervo[1]);
        bruno.pegarLivro(acervo[2]);

        System.print("");
        ana.listarEmprestimos();
        bruno.listarEmprestimos();

        System.print("");
        let d.int = contarDisponiveis(acervo, N_LIVROS);
        System.print("Disponíveis: " + d + " de " + N_LIVROS);
    }
}
```

A.9 Próximos Passos

A partir desta base, você pode estender o sistema de várias formas como exercício:

55. Adicione data de empréstimo e cálculo de multa por atraso (use int para representar dias).
56. Implemente um sistema de reservas (livro indisponível pode ter um usuário "fila de espera").
57. Crie categorias de livros (romance, técnico, infantil) e relatórios por categoria.
58. Adicione um sistema de pontuação para usuários (mais devoluções no prazo = mais pontos).
59. Sele a versão final como .uzs para distribuir aos colegas.

Apêndice B · Cookbook: Receitas Comuns

Coleção de soluções rápidas para tarefas típicas do dia a dia em UpperZetta. Cada receita tem objetivo claro, código pronto e cabe em poucas linhas. Use como ponto de partida ou para descobrir idioms da linguagem.

B.1 Strings e Texto

◆ Concatenar strings com formatação

Combine textos com valores e separadores explícitos:

```
let nome.str = "Felipe";
let idade.int = 21;
let mensagem.str = "Olá, " + nome + "! Você tem " + idade + " anos.";
System.print(mensagem);
```

◆ Construir uma linha separadora

Crie linhas decorativas concatenando o mesmo caractere:

```
fun linha(n.int) >> str {
    let r.str = "";
    for (let i.int = 0; i < n; i = i + 1) {
        r = r + "-";
    }
    return r;
}

System.print(linha(40));
```

◆ Centralizar texto entre delimitadores

Útil para títulos de relatório:

```
fun titulo(texto.str, largura.int) >> str {
    let espacos.int = (largura - 8) / 2;   // 8 = "== " + " ==" + texto pad
    let resultado.str = "==";
    for (let i.int = 0; i < espacos; i = i + 1) {
        resultado = resultado + " ";
    }
    return resultado + " " + texto + " ==";
}

System.print(titulo("RELATÓRIO", 40));
```

B.2 Números

◆ Valor absoluto de um inteiro

UZ não tem Math.abs, então implementamos:

```
fun absInt(n.int) >> int {
    if (n < 0) { return -n; }
    return n;
}

System.print(absInt(-42));   // 42
System.print(absInt(7));     // 7
```

◆ Mínimo e máximo de dois números

Funções utilitárias clássicas:

```
fun min(a.int, b.int) >> int {
    if (a < b) { return a; }
    return b;
}

fun max(a.int, b.int) >> int {
    if (a > b) { return a; }
    return b;
}
```

◆ Verificar se um número é par

Sem operador de módulo, usamos divisão e multiplicação:

```
fun ehPar(n.int) >> boolean {
    let metade.int = n / 2;
    return (metade * 2) == n;
}

System.print(ehPar(10));    // true
System.print(ehPar(7));     // false
```

◆ Potência inteira (base elevada a expoente)

Multiplicação repetida:

```
fun potencia(base.int, exp.int) >> int {
    let r.int = 1;
```

```
    for (let i.int = 0; i < exp; i = i + 1) {
        r = r * base;
    }
    return r;
}

System.print(potencia(2, 10));   // 1024
```

B.3 Arrays

◆ Inicializar array com mesmo valor

UZ não tem array fill, então use literal explícito ou loop:

```
// Para arrays pequenos, literal é mais simples:
let zeros.array = [0, 0, 0, 0, 0];

// Para arrays maiores, loop preenche:
let arr.array = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0];
let valor.int = 7;
for (let i.int = 0; i < 10; i = i + 1) {
    arr[i] = valor;
}
```

◆ Contar ocorrências de um valor

Itere e incremente um contador:

```
fun contar(arr.array, n.int, alvo.int) >> int {
    let c.int = 0;
    for (let i.int = 0; i < n; i = i + 1) {
        if (arr[i] == alvo) { c = c + 1; }
    }
    return c;
}

let dados.array = [1, 3, 5, 3, 7, 3, 9];
System.print(contar(dados, 7, 3));   // 3
```

◆ Copiar array para outro

Cria array novo com os mesmos valores:

```
fun copiar(origem.array, destino.array, n.int) {
    for (let i.int = 0; i < n; i = i + 1) {
```

```
        destino[i] = origem[i];
    }
}

let a.array = [1, 2, 3, 4, 5];
let b.array = [0, 0, 0, 0, 0];
copiar(a, b, 5);
// b agora é [1, 2, 3, 4, 5]
```

◆ Reverter ordem de um array

Troca elementos do início com os do fim:

```
fun reverter(arr.array, n.int) {
    for (let i.int = 0; i < n / 2; i = i + 1) {
        let temp.int = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }
}

let v.array = [1, 2, 3, 4, 5];
reverter(v, 5);
// v agora é [5, 4, 3, 2, 1]
```

◆ Soma e média de array numérico

Idioma clássico de acumulação:

```
fun soma(arr.array, n.int) >> int {
    let total.int = 0;
    for (let i.int = 0; i < n; i = i + 1) {
        total = total + arr[i];
    }
    return total;
}

fun media(arr.array, n.int) >> float {
    let s.int = soma(arr, n);
    return s / 1.0 / n;   // divisão float
}
```

B.4 Validações

◆ Verificar se string está vazia

Compare com literal vazio:

```
fun ehVazia(s.str) >> boolean {
    return s == "";
}

System.print(ehVazia(""));      // true
System.print(ehVazia("a"));     // false
```

◆ Validar idade dentro de faixa

Composição de comparações:

```
fun idadeValida(idade.int) >> boolean {
    return idade >= 0 && idade <= 150;
}
```

◆ Verificar se objeto não é nulo antes de usar

Curto-circuito do && evita acesso a campo nulo:

```
if (usuario != null && usuario.ativo) {
    System.print("usuário ativo");
}
```

B.5 Saída e Formatação

◆ Imprimir tabela simples

Cabeçalho + dados em linhas:

```
fun imprimirCabecalho() {
    System.print("+----------+--------+");
    System.print("| Nome     | Idade  |");
    System.print("+----------+--------+");
}

fun imprimirLinha(nome.str, idade.int) {
    System.print("| " + nome + "      | " + idade + "     |");
}

imprimirCabecalho();
imprimirLinha("Ana", 30);
imprimirLinha("Bia", 25);
System.print("+----------+--------+");
```

◆ Formatador de moeda brasileira

Concatenação simples (sem casas decimais por enquanto):

```
fun formatarReais(valor.float) >> str {
    return "R$ " + valor;
}

System.print(formatarReais(99.90));   // R$ 99.9
```

◆ Linhas em branco e separadores visuais

Para espaçamento e seções na saída:

```
System.print("");                        // linha em branco
System.print("---");                     // separador simples
System.print("=== TÍTULO ===");          // título
System.print("###");                     // outro estilo
```

B.6 Estado e Lógica

◆ Toggle (alternar boolean)

Inverte o valor de uma flag:

```
let ativo.boolean = true;
if (ativo) { ativo = false; } else { ativo = true; }
// agora ativo é false
```

◆ Contador com limite máximo

Incrementa até um teto:

```
let contador.int = 0;
let limite.int = 5;

while (contador < limite) {
    System.print("tick " + contador);
    contador = contador + 1;
}
```

◆ Map de "chave string → valor int" simples

Sem hash maps nativos, use dois arrays paralelos:

```
e.const N_PARES.int = 3;

let chaves.array = ["um", "dois", "tres"];
let valores.array = [1, 2, 3];

fun buscar(chave.str) >> int {
    for (let i.int = 0; i < N_PARES; i = i + 1) {
        let k.str = chaves[i];
        if (k == chave) { return valores[i]; }
    }
    return -1;   // não encontrado
}

System.print(buscar("dois"));    // 2
System.print(buscar("quatro"));  // -1
```

◆ Switch via cadeia de if/else if

Como UZ não tem switch, simule com else if:

```
fun classificarDia(d.int) >> str {
    if (d == 1) { return "domingo"; }
    else if (d == 2) { return "segunda"; }
    else if (d == 3) { return "terça"; }
    else if (d == 4) { return "quarta"; }
    else if (d == 5) { return "quinta"; }
    else if (d == 6) { return "sexta"; }
    else if (d == 7) { return "sábado"; }
    return "inválido";
}
```

B.7 Padrões de Loop

◆ Loop com condição de parada via flag

Substituto para break:

```
fun encontrarPositivo(arr.array, n.int) >> int {
    let achou.boolean = false;
    let pos.int = -1;
    for (let i.int = 0; i < n && achou == false; i = i + 1) {
        if (arr[i] > 0) {
            achou = true;
            pos = i;
        }
    }
    return pos;
```

```
}
```

◆ Loop reverso (de N até 0)

Decremento explícito:

```
for (let i.int = 10; i >= 0; i = i - 1) {
    System.print(i);
}
```

◆ Loop pulando elementos (de 2 em 2)

Incremento personalizado:

```
for (let i.int = 0; i < 100; i = i + 2) {
    System.print(i);   // pares: 0, 2, 4, ...
}
```

Apêndice C · Padrões de Design Adaptados

Padrões de design clássicos foram catalogados originalmente para linguagens orientadas a objeto com recursos como herança, interfaces e funções de primeira classe. UpperZetta v1.0 não tem alguns desses recursos — então adaptamos os padrões para usar apenas os mecanismos disponíveis: classes, composição, funções globais e arrays.

> **NOTA**
> Os padrões descritos aqui podem parecer "diluídos" em comparação com suas formas canônicas. Essa simplicidade é intencional: padrões servem ao código, não o contrário. Use-os onde fazem sentido, não como ritual obrigatório.

C.1 Padrão Factory (Fábrica)

Substitui a ausência de construtores. Encapsula a lógica de criação de objetos em uma função dedicada, garantindo que instâncias sejam sempre inicializadas corretamente.

Quando usar

- Quando uma classe requer múltiplos campos para estar em estado válido.
- Quando há lógica de validação ou cálculo durante a criação.
- Para evitar repetição de código de inicialização espalhado pela base.

Implementação

```
class.public Pessoa {
    let nome.str;
    let idade.int;
    let email.str;
    let ativo.boolean;
}

fun criarPessoa(nome.str, idade.int, email.str) >> Pessoa {
    let p.Pessoa = new Pessoa();
    p.nome = nome;
    p.idade = idade;
    p.email = email;
    p.ativo = true;            // padrão: ativo na criação
    return p;
}

// Uso simples e consistente:
let ana.Pessoa = criarPessoa("Ana Silva", 30, "ana@exemplo.com");
let bruno.Pessoa = criarPessoa("Bruno Lima", 25, "bruno@exemplo.com");
```

C.2 Padrão Strategy (Estratégia)

Permite trocar algoritmos em tempo de execução. Como UZ não tem funções de primeira classe, simulamos com classes que encapsulam comportamento, escolhidas por um seletor.

Implementação

```
class.public Calculadora {
    let modo.str;     // "soma", "subtrai", "multiplica"

    fun executar(a.int, b.int) >> int {
        if (modo == "soma") { return a + b; }
        else if (modo == "subtrai") { return a - b; }
        else if (modo == "multiplica") { return a * b; }
        return 0;
    }
}

fun criarCalculadora(modo.str) >> Calculadora {
    let c.Calculadora = new Calculadora();
    c.modo = modo;
    return c;
}

componente Home {
    fun render() {
        let somador.Calculadora = criarCalculadora("soma");
        let multiplicador.Calculadora = criarCalculadora("multiplica");

        System.print(somador.executar(3, 4));         // 7
        System.print(multiplicador.executar(3, 4));   // 12
    }
}
```

C.3 Padrão Singleton (Instância Única)

Em UZ, o componente Home já é um singleton implícito da aplicação. Para outros singletons, encapsule no próprio componente Home como campo:

```
class.public Configuracao {
    let tema.str;
    let idioma.str;
    let nivelLog.int;
}

fun configPadrao() >> Configuracao {
    let c.Configuracao = new Configuracao();
    c.tema = "escuro";
    c.idioma = "pt-BR";
```

```
    c.nivelLog = 2;
    return c;
}

componente Home {
    let config.Configuracao;   // singleton da aplicação

    fun render() {
        config = configPadrao();

        System.print("Tema: " + config.tema);
        System.print("Idioma: " + config.idioma);
    }
}
```

C.4 Padrão State (Máquina de Estados)

Representa um objeto cujo comportamento muda conforme seu estado interno. Use uma string ou int para identificar o estado e despache nos métodos:

```
class.public Pedido {
    let id.int;
    let estado.str;   // "rascunho", "submetido", "aprovado", "cancelado"

    fun submeter() >> boolean {
        if (estado != "rascunho") {
            System.print("[!] só pode submeter rascunhos.");
            return false;
        }
        estado = "submetido";
        System.print("[ok] pedido " + id + " submetido.");
        return true;
    }

    fun aprovar() >> boolean {
        if (estado != "submetido") {
            System.print("[!] só pode aprovar pedidos submetidos.");
            return false;
        }
        estado = "aprovado";
        return true;
    }

    fun cancelar() >> boolean {
        if (estado == "aprovado") {
            System.print("[!] pedido aprovado não pode ser cancelado.");
            return false;
        }
        estado = "cancelado";
```

```
        return true;
    }
}
```

C.5 Padrão Composite (Composição)

Trata um conjunto de objetos como um objeto único. Útil para estruturas hierárquicas:

```
e.const MAX_ITENS.int = 10;

class.public ItemPedido {
    let nome.str;
    let preco.float;
    let quantidade.int;

    fun subtotal() >> float {
        return preco * quantidade;
    }
}

class.public Pedido {
    let itens.array;
    let nItens.int;

    fun adicionar(it.ItemPedido) {
        if (nItens < MAX_ITENS) {
            itens[nItens] = it;
            nItens = nItens + 1;
        }
    }

    fun total() >> float {
        let t.float = 0.0;
        for (let i.int = 0; i < nItens; i = i + 1) {
            let it.ItemPedido = itens[i];
            t = t + it.subtotal();
        }
        return t;
    }
}
```

C.6 Padrão Builder (Construtor Fluente)

Constrói objetos complexos passo a passo. Útil quando há muitos campos opcionais:

```
class.public Email {
    let de.str;
    let para.str;
```

```
    let assunto.str;
    let corpo.str;
    let prioridade.str;
}

class.public EmailBuilder {
    let email.Email;

    fun init() >> EmailBuilder {
        email = new Email();
        email.prioridade = "normal";   // padrão
        return this_self();
    }

    fun this_self() >> EmailBuilder {
        // truque: retorna a si mesmo via campo (limitação UZ)
        return null;   // simplificação: cada método retorna void
    }

    fun setRemetente(de.str) {
        email.de = de;
    }

    fun setDestinatario(para.str) {
        email.para = para;
    }

    fun setAssunto(assunto.str) {
        email.assunto = assunto;
    }

    fun setCorpo(corpo.str) {
        email.corpo = corpo;
    }

    fun setPrioridade(p.str) {
        email.prioridade = p;
    }

    fun build() >> Email {
        return email;
    }
}

// Uso:
let b.EmailBuilder = new EmailBuilder();
b.email = new Email();
b.email.prioridade = "normal";
b.setRemetente("a@b.com");
b.setDestinatario("c@d.com");
```

```
b.setAssunto("Oi");
b.setCorpo("Tudo bem?");
let e.Email = b.build();
```

ATENÇÃO
UZ não suporta retorno de "this", então o builder fluente clássico (a.b().c().d()) não funciona. Adaptamos para chamadas separadas no mesmo objeto.

C.7 Padrão Repository (Repositório)

Centraliza acesso a coleções de objetos, isolando a lógica de busca e filtragem:

```
e.const TAM_REPO.int = 100;

class.public RepositorioPessoas {
    let pessoas.array;
    let n.int;

    fun adicionar(p.Pessoa) {
        if (n < TAM_REPO) {
            pessoas[n] = p;
            n = n + 1;
        }
    }

    fun buscarPorNome(nome.str) >> Pessoa {
        for (let i.int = 0; i < n; i = i + 1) {
            let p.Pessoa = pessoas[i];
            if (p.nome == nome) { return p; }
        }
        return null;
    }

    fun contarAtivos() >> int {
        let c.int = 0;
        for (let i.int = 0; i < n; i = i + 1) {
            let p.Pessoa = pessoas[i];
            if (p.ativo) { c = c + 1; }
        }
        return c;
    }
}
```

Apêndice D · Estruturas de Dados Implementadas

UpperZetta oferece nativamente apenas arrays de tamanho fixo. Para muitas tarefas reais, precisamos de estruturas mais sofisticadas: pilhas, filas, listas encadeadas, dicionários, conjuntos. Esta seção mostra implementações funcionais dessas estruturas usando apenas os recursos da v1.0.

> **NOTA**
> As implementações priorizam clareza didática, não performance máxima. Para coleções com milhares de elementos, considere o impacto das limitações da linguagem (sem hashmaps nativos, sem array dinâmico).

D.1 Pilha (Stack — LIFO)

Estrutura LIFO (Last In, First Out): o último elemento adicionado é o primeiro a sair. Operações principais: push (adicionar) e pop (remover do topo).

```
e.const TAM_PILHA.int = 100;

class.public Pilha {
    let dados.array;
    let topo.int;

    fun push(valor.int) >> boolean {
        if (topo >= TAM_PILHA) {
            System.print("[!] pilha cheia");
            return false;
        }
        dados[topo] = valor;
        topo = topo + 1;
        return true;
    }

    fun pop() >> int {
        if (topo == 0) {
            System.print("[!] pilha vazia");
            return -1;
        }
        topo = topo - 1;
        return dados[topo];
    }

    fun peek() >> int {
        if (topo == 0) { return -1; }
        return dados[topo - 1];
```

```
    }

    fun ehVazia() >> boolean {
        return topo == 0;
    }

    fun tamanho() >> int {
        return topo;
    }
}

fun criarPilha() >> Pilha {
    let p.Pilha = new Pilha();
    // inicializa array com 100 zeros (use script para gerar literal completo)
    p.dados = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
               0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
               0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
               0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
               0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0];
    p.topo = 0;
    return p;
}

componente Home {
    fun render() {
        let p.Pilha = criarPilha();
        p.push(10);
        p.push(20);
        p.push(30);

        System.print("topo: " + p.peek());      // 30
        System.print("pop:  " + p.pop());       // 30
        System.print("pop:  " + p.pop());       // 20
        System.print("tam:  " + p.tamanho());   // 1
    }
}
```

D.2 Fila (Queue — FIFO)

Estrutura FIFO (First In, First Out): o primeiro elemento adicionado é o primeiro a sair. Implementação simples (não-circular):

```
e.const TAM_FILA.int = 100;

class.public Fila {
    let dados.array;
    let inicio.int;
    let fim.int;
```

```
    fun enqueue(valor.int) >> boolean {
        if (fim >= TAM_FILA) {
            System.print("[!] fila cheia");
            return false;
        }
        dados[fim] = valor;
        fim = fim + 1;
        return true;
    }

    fun dequeue() >> int {
        if (inicio == fim) {
            System.print("[!] fila vazia");
            return -1;
        }
        let v.int = dados[inicio];
        inicio = inicio + 1;
        return v;
    }

    fun ehVazia() >> boolean {
        return inicio == fim;
    }

    fun tamanho() >> int {
        return fim - inicio;
    }
}
```

D.3 Lista Encadeada Simples

Coleção dinâmica de elementos onde cada nó aponta para o próximo. Vantagem sobre array: tamanho cresce conforme necessário (não há limite fixo, exceto memória):

```
class.public No {
    let valor.int;
    let proximo.No;
}

class.public ListaEncadeada {
    let cabeca.No;
    let tamanho.int;

    fun adicionarInicio(v.int) {
        let novo.No = new No();
        novo.valor = v;
        novo.proximo = cabeca;
        cabeca = novo;
        tamanho = tamanho + 1;
```

```
    }

    fun adicionarFim(v.int) {
        let novo.No = new No();
        novo.valor = v;
        novo.proximo = null;

        if (cabeca == null) {
            cabeca = novo;
        } else {
            let atual.No = cabeca;
            while (atual.proximo != null) {
                atual = atual.proximo;
            }
            atual.proximo = novo;
        }
        tamanho = tamanho + 1;
    }

    fun imprimir() {
        let atual.No = cabeca;
        while (atual != null) {
            System.print(atual.valor);
            atual = atual.proximo;
        }
    }

    fun buscar(v.int) >> boolean {
        let atual.No = cabeca;
        while (atual != null) {
            if (atual.valor == v) { return true; }
            atual = atual.proximo;
        }
        return false;
    }
}

fun criarLista() >> ListaEncadeada {
    let l.ListaEncadeada = new ListaEncadeada();
    l.cabeca = null;
    l.tamanho = 0;
    return l;
}
```

D.4 Dicionário (Map) Linear

Sem hashmaps nativos, implementamos com dois arrays paralelos. Performance O(n) na busca, mas funcional para coleções pequenas (< 100 entradas):

```
e.const TAM_DICT.int = 50;

class.public Dicionario {
    let chaves.array;
    let valores.array;
    let n.int;

    fun definir(k.str, v.int) {
        // Atualiza se chave já existe
        for (let i.int = 0; i < n; i = i + 1) {
            let ka.str = chaves[i];
            if (ka == k) {
                valores[i] = v;
                return;
            }
        }
        // Adiciona nova
        if (n < TAM_DICT) {
            chaves[n] = k;
            valores[n] = v;
            n = n + 1;
        }
    }

    fun obter(k.str) >> int {
        for (let i.int = 0; i < n; i = i + 1) {
            let ka.str = chaves[i];
            if (ka == k) { return valores[i]; }
        }
        return -1;   // sentinela "não encontrado"
    }

    fun contem(k.str) >> boolean {
        for (let i.int = 0; i < n; i = i + 1) {
            let ka.str = chaves[i];
            if (ka == k) { return true; }
        }
        return false;
    }

    fun tamanho() >> int { return n; }
}
```

D.5 Conjunto (Set)

Coleção sem duplicatas. Implementação direta com array de inteiros e verificação de existência:

```
e.const TAM_SET.int = 50;

class.public Conjunto {
    let elementos.array;
    let n.int;

    fun adicionar(v.int) >> boolean {
        if (contem(v)) { return false; }   // já existe
        if (n >= TAM_SET) { return false; }
        elementos[n] = v;
        n = n + 1;
        return true;
    }

    fun contem(v.int) >> boolean {
        for (let i.int = 0; i < n; i = i + 1) {
            if (elementos[i] == v) { return true; }
        }
        return false;
    }

    fun remover(v.int) >> boolean {
        let pos.int = -1;
        for (let i.int = 0; i < n; i = i + 1) {
            if (elementos[i] == v) { pos = i; }
        }
        if (pos == -1) { return false; }

        // Compacta
        for (let i.int = pos; i < n - 1; i = i + 1) {
            elementos[i] = elementos[i + 1];
        }
        n = n - 1;
        return true;
    }
}
```

D.6 Árvore Binária de Busca

Estrutura hierárquica onde cada nó tem até dois filhos: esquerda (valores menores) e direita (valores maiores). Permite busca eficiente em O(log n) na média:

```
class.public NoArvore {
    let valor.int;
```

```
    let esquerda.NoArvore;
    let direita.NoArvore;
}

class.public Arvore {
    let raiz.NoArvore;

    fun inserir(v.int) {
        raiz = inserirRec(raiz, v);
    }

    fun inserirRec(no.NoArvore, v.int) >> NoArvore {
        if (no == null) {
            let novo.NoArvore = new NoArvore();
            novo.valor = v;
            novo.esquerda = null;
            novo.direita = null;
            return novo;
        }
        if (v < no.valor) {
            no.esquerda = inserirRec(no.esquerda, v);
        } else if (v > no.valor) {
            no.direita = inserirRec(no.direita, v);
        }
        return no;
    }

    fun contem(v.int) >> boolean {
        return buscarRec(raiz, v);
    }

    fun buscarRec(no.NoArvore, v.int) >> boolean {
        if (no == null) { return false; }
        if (v == no.valor) { return true; }
        if (v < no.valor) { return buscarRec(no.esquerda, v); }
        return buscarRec(no.direita, v);
    }

    fun imprimirEmOrdem(no.NoArvore) {
        if (no == null) { return; }
        imprimirEmOrdem(no.esquerda);
        System.print(no.valor);
        imprimirEmOrdem(no.direita);
    }
}
```

DICA

A travessia em ordem (esquerda → raiz → direita) imprime os valores em ordem crescente — uma forma elegante de ordenar usando árvore.

Apêndice E · Algoritmos Clássicos

Implementações em UpperZetta de algoritmos fundamentais que todo desenvolvedor deveria conhecer. São úteis tanto como referência quanto como exercício para internalizar a sintaxe da linguagem.

E.1 Algoritmos de Ordenação

E.1.1 Bubble Sort

O algoritmo mais simples de ordenação. Compara pares adjacentes e troca quando estão fora de ordem. Complexidade: O(n²). Inadequado para arrays grandes, mas didático:

```
fun bubbleSort(arr.array, n.int) {
    for (let i.int = 0; i < n - 1; i = i + 1) {
        for (let j.int = 0; j < n - 1 - i; j = j + 1) {
            if (arr[j] > arr[j + 1]) {
                let temp.int = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

componente Home {
    fun render() {
        let v.array = [5, 2, 8, 1, 9, 3, 7, 4, 6];
        bubbleSort(v, 9);
        for (let i.int = 0; i < 9; i = i + 1) {
            System.print(v[i]);
        }
        // Saída: 1 2 3 4 5 6 7 8 9
    }
}
```

E.1.2 Selection Sort

A cada iteração, encontra o menor elemento do restante e o coloca na posição correta. Mais eficiente que Bubble em alguns casos. Complexidade: O(n²):

```
fun selectionSort(arr.array, n.int) {
    for (let i.int = 0; i < n - 1; i = i + 1) {
        let minIdx.int = i;
        for (let j.int = i + 1; j < n; j = j + 1) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
```

```
        }
        if (minIdx != i) {
            let temp.int = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = temp;
        }
    }
}
```

E.1.3 Insertion Sort

Constrói a ordenação um elemento por vez, inserindo cada novo elemento na posição correta entre os anteriores. Excelente para arrays pequenos ou quase ordenados:

```
fun insertionSort(arr.array, n.int) {
    for (let i.int = 1; i < n; i = i + 1) {
        let chave.int = arr[i];
        let j.int = i - 1;

        // Move elementos maiores que 'chave' uma posição à frente
        while (j >= 0 && arr[j] > chave) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = chave;
    }
}
```

E.2 Algoritmos de Busca

E.2.1 Busca Linear

Percorre o array do início ao fim procurando o alvo. Funciona em qualquer array, ordenado ou não. Complexidade: O(n):

```
fun buscaLinear(arr.array, n.int, alvo.int) >> int {
    for (let i.int = 0; i < n; i = i + 1) {
        if (arr[i] == alvo) { return i; }
    }
    return -1;   // não encontrado
}
```

E.2.2 Busca Binária

Requer array ORDENADO. Divide o intervalo de busca pela metade a cada iteração. Drasticamente mais rápida: O(log n):

```
fun buscaBinaria(arr.array, n.int, alvo.int) >> int {
    let inicio.int = 0;
    let fim.int = n - 1;

    while (inicio <= fim) {
        let meio.int = (inicio + fim) / 2;
        if (arr[meio] == alvo) { return meio; }
        if (arr[meio] < alvo) {
            inicio = meio + 1;
        } else {
            fim = meio - 1;
        }
    }
    return -1;
}

// Uso (array deve estar ORDENADO):
componente Home {
    fun render() {
        let v.array = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19];
        System.print("posição de 7:  " + buscaBinaria(v, 10, 7));    // 3
        System.print("posição de 20: " + buscaBinaria(v, 10, 20));   // -1
    }
}
```

E.3 Recursão Clássica

E.3.1 Fatorial

```
fun fatorial(n.int) >> int {
    if (n <= 1) { return 1; }
    return n * fatorial(n - 1);
}

// Versão iterativa (sem risco de stack overflow):
fun fatorialIter(n.int) >> int {
    let r.int = 1;
    for (let i.int = 2; i <= n; i = i + 1) {
        r = r * i;
    }
    return r;
}
```

E.3.2 Fibonacci

```
// Recursivo (lento para n grande, O(2^n)):
fun fibRec(n.int) >> int {
    if (n <= 1) { return n; }
```

```
    return fibRec(n - 1) + fibRec(n - 2);
}

// Iterativo (eficiente, O(n)):
fun fibIter(n.int) >> int {
    if (n <= 1) { return n; }
    let a.int = 0;
    let b.int = 1;
    for (let i.int = 2; i <= n; i = i + 1) {
        let c.int = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

E.3.3 Torre de Hanói

Problema clássico de recursão: mover N discos da torre A para C, usando B como auxiliar:

```
fun hanoi(n.int, origem.str, destino.str, aux.str) {
    if (n == 1) {
        System.print("Mova disco 1 de " + origem + " para " + destino);
        return;
    }
    hanoi(n - 1, origem, aux, destino);
    System.print("Mova disco " + n + " de " + origem + " para " + destino);
    hanoi(n - 1, aux, destino, origem);
}

componente Home {
    fun render() {
        hanoi(3, "A", "C", "B");
    }
}
```

E.4 Algoritmos Numéricos

E.4.1 Máximo Divisor Comum (MDC) — Algoritmo de Euclides

```
fun mdc(a.int, b.int) >> int {
    while (b != 0) {
        let temp.int = b;
        // Resto de a por b: a - (a/b)*b
        b = a - (a / b) * b;
        a = temp;
    }
    return a;
}
```

```
System.print(mdc(48, 36));   // 12
System.print(mdc(100, 75));  // 25
```

E.4.2 Verificar se Número é Primo

```
fun ehPrimo(n.int) >> boolean {
    if (n < 2) { return false; }
    if (n == 2) { return true; }
    let par.int = (n / 2) * 2;
    if (par == n) { return false; }   // par > 2 não é primo

    let i.int = 3;
    while (i * i <= n) {
        let resto.int = n - (n / i) * i;
        if (resto == 0) { return false; }
        i = i + 2;
    }
    return true;
}

// Listar primos até N:
fun listarPrimos(limite.int) {
    for (let n.int = 2; n <= limite; n = n + 1) {
        if (ehPrimo(n)) { System.print(n); }
    }
}
```

E.4.3 Reverter um Número

```
fun reverter(n.int) >> int {
    let r.int = 0;
    let restante.int = n;
    while (restante > 0) {
        let digito.int = restante - (restante / 10) * 10;
        r = r * 10 + digito;
        restante = restante / 10;
    }
    return r;
}

System.print(reverter(12345));   // 54321
```

E.4.4 Verificar se é Palíndromo Numérico

```
fun ehPalindromoNum(n.int) >> boolean {
    return n == reverter(n);
}
```

```
System.print(ehPalindromoNum(12321));   // true
System.print(ehPalindromoNum(12345));   // false
```

E.5 Manipulação de Arrays

E.5.1 Encontrar Mínimo e Máximo Simultaneamente

```
fun minMax(arr.array, n.int, resultado.array) {
    // resultado[0] = min, resultado[1] = max
    let mn.int = arr[0];
    let mx.int = arr[0];
    for (let i.int = 1; i < n; i = i + 1) {
        if (arr[i] < mn) { mn = arr[i]; }
        if (arr[i] > mx) { mx = arr[i]; }
    }
    resultado[0] = mn;
    resultado[1] = mx;
}

let v.array = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3];
let r.array = [0, 0];
minMax(v, 10, r);
System.print("Min: " + r[0] + ", Max: " + r[1]);
```

E.5.2 Rotacionar Array

```
fun rotacionarEsquerda(arr.array, n.int, k.int) {
    // Rotaciona k posições à esquerda
    for (let r.int = 0; r < k; r = r + 1) {
        let primeiro.int = arr[0];
        for (let i.int = 0; i < n - 1; i = i + 1) {
            arr[i] = arr[i + 1];
        }
        arr[n - 1] = primeiro;
    }
}

let v.array = [1, 2, 3, 4, 5];
rotacionarEsquerda(v, 5, 2);
// v agora é [3, 4, 5, 1, 2]
```

E.5.3 Verificar se Arrays São Iguais

```
fun arraysIguais(a.array, b.array, n.int) >> boolean {
    for (let i.int = 0; i < n; i = i + 1) {
        if (a[i] != b[i]) { return false; }
```

```
    }
    return true;
}
```

Apêndice F · Receitas Matemáticas

UpperZetta v1.0 não inclui biblioteca matemática (Math.sqrt, Math.sin, Math.log, etc.). Esta seção apresenta implementações próprias de funções matemáticas comuns, úteis para suprir essa lacuna em projetos práticos.

> **ATENÇÃO**
> As implementações abaixo priorizam clareza e correção; não são otimizadas para performance ou precisão extrema. Para cálculos científicos críticos, considere se UZ é a ferramenta certa.

F.1 Funções Básicas

F.1.1 Valor Absoluto

```
fun absInt(n.int) >> int {
    if (n < 0) { return -n; }
    return n;
}

fun absFloat(n.float) >> float {
    if (n < 0.0) { return -n; }
    return n;
}
```

F.1.2 Mínimo e Máximo

```
fun minInt(a.int, b.int) >> int {
    if (a < b) { return a; }
    return b;
}

fun maxInt(a.int, b.int) >> int {
    if (a > b) { return a; }
    return b;
}

fun minFloat(a.float, b.float) >> float {
    if (a < b) { return a; }
    return b;
```

```
}

fun maxFloat(a.float, b.float) >> float {
    if (a > b) { return a; }
    return b;
}
```

F.1.3 Sinal de um Número

```
fun sinal(n.float) >> int {
    if (n > 0.0) { return 1; }
    if (n < 0.0) { return -1; }
    return 0;
}
```

F.2 Potenciação

F.2.1 Potência Inteira

```
fun potInt(base.int, exp.int) >> int {
    if (exp < 0) { return 0; }   // não suporta expoentes negativos
    let r.int = 1;
    for (let i.int = 0; i < exp; i = i + 1) {
        r = r * base;
    }
    return r;
}

System.print(potInt(2, 10));   // 1024
System.print(potInt(3, 4));    // 81
```

F.2.2 Potência com Base Float

```
fun potFloat(base.float, exp.int) >> float {
    let r.float = 1.0;
    let positivo.boolean = exp >= 0;
    let n.int = exp;
    if (positivo == false) { n = -exp; }

    for (let i.int = 0; i < n; i = i + 1) {
        r = r * base;
    }

    if (positivo == false) { return 1.0 / r; }
    return r;
}
```

```
System.print(potFloat(2.0, -3));   // 0.125
System.print(potFloat(1.5, 4));    // 5.0625
```

F.3 Raízes

F.3.1 Raiz Quadrada — Método de Newton

Método iterativo que converge rapidamente para a raiz quadrada. Cada iteração refina a estimativa: x_{n+1} = (x_n + a/x_n) / 2:

```
fun sqrt(a.float) >> float {
    if (a < 0.0) { return 0.0; }
    if (a == 0.0) { return 0.0; }

    let x.float = a;
    let i.int = 0;
    while (i < 20) {   // 20 iterações são mais que suficientes
        x = (x + a / x) / 2.0;
        i = i + 1;
    }
    return x;
}

System.print(sqrt(2.0));      // ~1.4142135623730951
System.print(sqrt(9.0));      // ~3.0
System.print(sqrt(100.0));    // ~10.0
System.print(sqrt(1234.0));   // ~35.128...
```

F.3.2 Raiz Cúbica

```
fun cbrt(a.float) >> float {
    let positivo.boolean = a >= 0.0;
    let n.float = a;
    if (positivo == false) { n = -a; }

    let x.float = n;
    let i.int = 0;
    while (i < 30) {
        x = (2.0 * x + n / (x * x)) / 3.0;
        i = i + 1;
    }

    if (positivo == false) { return -x; }
    return x;
}

System.print(cbrt(27.0));    // ~3.0
System.print(cbrt(1000.0));  // ~10.0
```

F.4 Funções Trigonométricas (Aproximações por Série de Taylor)

F.4.1 Seno

Usa a expansão em série de Taylor: $\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$

```
e.const PI.float = 3.14159265358979;
e.const DOIS_PI.float = 6.28318530717959;

fun sin(x.float) >> float {
    // Reduz x ao intervalo [-π, π] para melhor precisão
    let xr.float = x;
    while (xr > PI) { xr = xr - DOIS_PI; }
    while (xr < -PI) { xr = xr + DOIS_PI; }

    let resultado.float = 0.0;
    let termo.float = xr;
    let xQuad.float = xr * xr;
    let i.int = 1;

    // Soma 10 termos da série
    while (i <= 19) {
        resultado = resultado + termo;
        termo = -termo * xQuad / ((i + 1) * (i + 2));
        i = i + 2;
    }
    return resultado;
}

System.print(sin(0.0));         // ~0
System.print(sin(PI / 2.0));    // ~1
System.print(sin(PI));          // ~0
```

F.4.2 Cosseno

```
fun cos(x.float) >> float {
    return sin(x + PI / 2.0);
}

System.print(cos(0.0));        // ~1
System.print(cos(PI));         // ~-1
```

F.4.3 Tangente

```
fun tan(x.float) >> float {
    let s.float = sin(x);
    let c.float = cos(x);
    return s / c;
```

```
}
```

F.5 Logaritmos e Exponenciais

F.5.1 Exponencial e^x — Série de Taylor

```
fun exp(x.float) >> float {
    let resultado.float = 1.0;
    let termo.float = 1.0;
    let i.int = 1;

    while (i < 20) {
        termo = termo * x / i;
        resultado = resultado + termo;
        i = i + 1;
    }
    return resultado;
}

System.print(exp(0.0));   // 1
System.print(exp(1.0));   // ~2.71828 (e)
System.print(exp(2.0));   // ~7.389
```

F.6 Conversões

F.6.1 Graus para Radianos e Vice-versa

```
fun grausParaRad(graus.float) >> float {
    return graus * PI / 180.0;
}

fun radParaGraus(rad.float) >> float {
    return rad * 180.0 / PI;
}
```

F.6.2 Celsius / Fahrenheit

```
fun celsiusParaFahrenheit(c.float) >> float {
    return c * 9.0 / 5.0 + 32.0;
}

fun fahrenheitParaCelsius(f.float) >> float {
    return (f - 32.0) * 5.0 / 9.0;
}
```

F.7 Estatística

F.7.1 Média Aritmética

```
fun media(arr.array, n.int) >> float {
    let soma.float = 0.0;
    for (let i.int = 0; i < n; i = i + 1) {
        soma = soma + arr[i];
    }
    return soma / n;
}
```

F.7.2 Variância e Desvio Padrão

```
fun variancia(arr.array, n.int) >> float {
    let m.float = media(arr, n);
    let somaQuad.float = 0.0;
    for (let i.int = 0; i < n; i = i + 1) {
        let diff.float = arr[i] - m;
        somaQuad = somaQuad + diff * diff;
    }
    return somaQuad / n;
}

fun desvioPadrao(arr.array, n.int) >> float {
    return sqrt(variancia(arr, n));
}
```

F.8 Constantes Úteis

```
e.const PI.float = 3.14159265358979;
e.const DOIS_PI.float = 6.28318530717959;
e.const PI_SOBRE_2.float = 1.57079632679490;
e.const E.float = 2.71828182845905;
e.const SQRT2.float = 1.41421356237310;
e.const SQRT3.float = 1.73205080756888;
e.const LN2.float = 0.69314718055995;
e.const LN10.float = 2.30258509299405;
```

Apêndice G · Estratégias de Teste

UpperZetta v1.0 não inclui framework de testes. Esta seção mostra como organizar testes de forma disciplinada usando apenas os recursos da linguagem, garantindo confiança no código sem dependências externas.

G.1 Asserts Customizados

Implemente uma função simples de asserção que verifica uma condição e imprime o resultado:

```
let totalTestes.int = 0;
let testesPassaram.int = 0;

fun assertEquals(esperado.int, atual.int, nome.str) {
    totalTestes = totalTestes + 1;
    if (esperado == atual) {
        System.print("[PASS] " + nome);
        testesPassaram = testesPassaram + 1;
    } else {
        System.print("[FAIL] " + nome + " esperado=" + esperado + " atual=" + atual);
    }
}

fun assertTrue(condicao.boolean, nome.str) {
    totalTestes = totalTestes + 1;
    if (condicao) {
        System.print("[PASS] " + nome);
        testesPassaram = testesPassaram + 1;
    } else {
        System.print("[FAIL] " + nome);
    }
}

fun resumoTestes() {
    System.print("---");
    System.print("Resultados: " + testesPassaram + "/" + totalTestes);
}
```

ATENÇÃO

Note: como UZ não suporta variáveis "globais mutáveis" tradicionais (e.const não pode ser reatribuída), use um componente como container de estado de teste.

G.2 Padrão Test Runner

Um componente dedicado a executar testes evita misturar lógica de teste com lógica de produção:

```
componente Home {
    let total.int = 0;
    let passou.int = 0;

    fun render() {
        System.print("=== Executando testes ===");

        testarSoma();
        testarMultiplicacao();
        testarStringConcat();
```

```
        testarValidacao();

        System.print("");
        System.print("Resultado final: " + passou + "/" + total);
    }

    fun assertEq(esperado.int, atual.int, nome.str) {
        total = total + 1;
        if (esperado == atual) {
            passou = passou + 1;
            System.print("[ok] " + nome);
        } else {
            System.print("[FAIL] " + nome + " (esperado=" + esperado + ", atual=" +
atual + ")");
        }
    }

    fun testarSoma() {
        assertEq(5, 2 + 3, "soma simples");
        assertEq(0, -5 + 5, "soma com negativo");
    }

    fun testarMultiplicacao() {
        assertEq(12, 3 * 4, "mult básica");
        assertEq(0, 100 * 0, "mult por zero");
    }

    fun testarStringConcat() {
        let s.str = "abc" + "def";
        // não há comparação direta de strings retornando int, mas podemos:
        if (s == "abcdef") { passou = passou + 1; }
        total = total + 1;
    }

    fun testarValidacao() {
        // testes da nossa lógica de domínio
    }
}
```

G.3 Testes de Borda

Sempre teste casos extremos: zero, valores negativos, arrays vazios, valores máximos:

```
fun testarBorda() {
    // Inteiros nas bordas
    assertEq(-2147483648, -2147483648, "min int");
    assertEq(2147483647, 2147483647, "max int");

    // Divisão por números pequenos
```

```
    let r.float = 1.0 / 0.0001;
    // r é finito mas grande

    // Strings vazias
    let vazio.str = "";
    let combinado.str = "x" + vazio + "y";
    // combinado == "xy"
}
```

G.4 Testes de Regressão

Quando você corrige um bug, escreva um teste que falha antes da correção e passa depois. Esse teste impede que o bug volte:

```
// Bug encontrado em 2026-04-15: divisão inteira retornava errado para negativos
fun testarRegressaoBug001() {
    assertEq(-3, -10 / 3, "divisão inteira de negativo (bug-001)");
}
```

G.5 Estrutura Recomendada

Para projetos que crescem, mantenha testes em arquivo separado:

```
projeto/
├── src/
│   └── biblioteca.uz       # código de produção
└── testes/
    └── biblioteca_test.uz  # testes (executável separado)
```

> **DICA**
> Como UZ não tem imports, copie a lógica relevante para o arquivo de teste, ou mantenha tudo em um arquivo único usando uma flag para ativar o modo de teste.

Apêndice H · Deploy e Distribuição

Este apêndice cobre o ciclo completo de levar um programa UpperZetta de desenvolvimento à entrega para usuários finais. Foca em práticas que evitam dor de cabeça posterior.

H.1 Versionamento Semântico

Adote o padrão SemVer (Semantic Versioning) para suas aplicações. Versões têm formato MAJOR.MINOR.PATCH:

- **MAJOR** — incrementa quando há mudanças incompatíveis (quebra programas que dependem da versão anterior).
- **MINOR** — incrementa quando adiciona funcionalidades sem quebrar nada existente.
- **PATCH** — incrementa para correções de bug que mantêm compatibilidade.

H.2 Release Notes

Para cada release, mantenha um arquivo CHANGELOG.md descrevendo as mudanças. Exemplo de estrutura:

CHANGELOG.md

```
# Changelog

## [2.1.0] - 2026-04-25
### Adicionado
- Sistema de empréstimos com fila de espera
- Comando "uz arquivo --debug" para logging detalhado

### Corrigido
- Travamento quando array vazio é passado para mediaCalc()

### Mudado
- Formato do relatório financeiro agora inclui taxas

## [2.0.0] - 2026-03-12
### MUDANÇA INCOMPATÍVEL
- API de Pedido alterada: agora exige campo "id" obrigatório

## [1.0.0] - 2026-01-15
- Versão inicial
```

H.3 Build Reproduzível

Garanta que qualquer pessoa possa gerar o mesmo .uzb a partir do mesmo .uz. Isso requer:

60. Versionar o .uz no Git (nunca apenas o .uzb).
61. Documentar a versão exata da UVLM usada (no CHANGELOG ou README).
62. Usar um script de build (build.sh ou Makefile) que sempre executa os mesmos comandos.
63. Não depender de variáveis de ambiente que mudam entre máquinas.

H.4 Empacotamento para Distribuição

Cinco formatos típicos de entrega:

H.4.1 Distribuição como Bytecode (.uzb)

- Vantagens: usuário não vê código-fonte; não precisa recompilar.
- Desvantagens: usuário precisa ter UVLM instalada.
- Use quando: o público é técnico ou tem ambiente UVLM padronizado.

H.4.2 Distribuição como Selado (.uzs)

- Vantagens: proteção criptográfica do código + bytecode.
- Desvantagens: requer senha (logística de comunicação com cliente).
- Use quando: software comercial sob NDA ou licenciamento.

H.4.3 Distribuição com Loader Empacotado

Crie um script wrapper que invoca a UVLM:

rodar.sh

```
#!/bin/bash
# Loader de programa UpperZetta
DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && pwd )"
java -cp "$DIR/Main.jar" Main "$DIR/programa.uzb" "$@"
```

H.4.4 Distribuição como Imagem Docker

Para servidores ou ambientes uniformes:

Dockerfile

```
FROM openjdk:17-slim
COPY Main.jar /app/Main.jar
COPY programa.uzb /app/programa.uzb
WORKDIR /app
CMD ["java", "-cp", "Main.jar", "Main", "programa.uzb"]
```

H.4.5 Distribuição como Pacote Auto-contido

Diretório com tudo que o usuário precisa:

```
distribuicao/
├── Main.jar          # UVLM empacotada
├── programa.uzb      # bytecode da aplicação
├── README.md         # instruções de uso
├── LICENSE
├── CHANGELOG.md
├── rodar.sh          # script de execução (Unix)
└── rodar.bat         # script de execução (Windows)
```

H.5 Checklist Pré-Release

64. Todos os testes passam (Apêndice G).

65. CHANGELOG atualizado com as mudanças desta versão.
66. Versão incrementada conforme SemVer.
67. Compilação limpa: uz build src/main.uz -o dist/main.uzb sem warnings.
68. Smoke test: rode o .uzb manualmente e verifique funcionalidade básica.
69. Documentação revisada (README, exemplos atualizados).
70. Tag de versão criada no Git: git tag v2.1.0.
71. Backup do .uz e da DEVICE KEY em local seguro.

H.6 Pipeline CI/CD Simples

Exemplo de pipeline em GitHub Actions para automatizar build e teste:

.github/workflows/build.yml

```
name: Build UpperZetta

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Java
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
      - name: Build
        run: java -cp Main.jar Main build src/main.uz -o build/main.uzb
      - name: Test
        run: java -cp Main.jar Main testes/runner.uz
      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: uzb-bytecode
          path: build/main.uzb
```

Apêndice I · Comparações com Outras Linguagens

Esta seção amplia a Seção 29, comparando UpperZetta com mais linguagens modernas. Útil para desenvolvedores migrando de stacks específicas.

I.1 vs. Go

Go	UpperZetta
var x int = 10	let x.int = 10;
const PI = 3.14	e.const PI.float = 3.14;
func soma(a, b int) int	fun soma(a.int, b.int) >> int
type Pessoa struct { ... }	class.public Pessoa { ... }
fmt.Println(x)	System.print(x);
for i := 0; i < n; i++ { }	for (let i.int = 0; i < n; i = i + 1) { }
if err != nil { ... }	(sem erros como valores)
interface { ... }	(sem interfaces)
goroutines	(single-threaded)

- **Diferenças filosóficas:** Go enfatiza concorrência (goroutines, channels), UZ é estritamente single-threaded. Go usa interfaces estruturais; UZ não tem interfaces.
- **Sintaxe:** Go põe tipo após nome com espaço (`x int`); UZ usa ponto (`x.int`).

I.2 vs. Rust

Rust	UpperZetta
let x: i32 = 10;	let x.int = 10;
const PI: f64 = 3.14;	e.const PI.float = 3.14;
fn soma(a: i32, b: i32) -> i32	fun soma(a.int, b.int) >> int
struct Pessoa { ... }	class.public Pessoa { ... }
println!("{}", x)	System.print(x);
for i in 0..n { }	for (let i.int = 0; i < n; i = i + 1) { }
Result<T, E>	(sem erro como tipo)
ownership / borrow	(referências livres, GC)
trait	(sem traits)

- **Memória:** Rust usa ownership/borrow para garantir segurança em compilação. UZ usa GC da JVM, sem ownership.
- **Erro:** Rust trata erros como valores (Result/Option). UZ aborta no erro de runtime.

I.3 vs. Kotlin

Kotlin	UpperZetta
val x: Int = 10	let x.int = 10;
const val PI = 3.14	e.const PI.float = 3.14;
fun soma(a: Int, b: Int): Int	fun soma(a.int, b.int) >> int
class Pessoa { ... }	class.public Pessoa { ... }
println(x)	System.print(x);
for (i in 0 until n) { }	for (let i.int = 0; i < n; i = i + 1) { }
x?.metodo()	(verifique != null manualmente)
data class	(sem data class)
extension functions	(sem extensões)

I.4 vs. C#

C#	UpperZetta
int x = 10;	let x.int = 10;
const double PI = 3.14;	e.const PI.float = 3.14;
int Soma(int a, int b)	fun soma(a.int, b.int) >> int
public class Pessoa { ... }	class.public Pessoa { ... }
Console.WriteLine(x)	System.print(x);
for (int i = 0; i < n; i++)	for (let i.int = 0; i < n; i = i + 1)
try { } catch (Exception)	(sem exceções)
async/await	(sem async)
LINQ	(sem LINQ)

I.5 vs. Swift

Swift	UpperZetta
var x: Int = 10	let x.int = 10;
let PI: Double = 3.14	e.const PI.float = 3.14;

Swift	UpperZetta
func soma(a: Int, b: Int) -> Int	fun soma(a.int, b.int) >> int
class Pessoa { ... }	class.public Pessoa { ... }
print(x)	System.print(x);
for i in 0..<n { }	for (let i.int = 0; i < n; i = i + 1)
Optional<T>	(use null com checagem manual)
protocol	(sem protocols)

I.6 Resumo Comparativo Geral

Aspecto	UZ	Java	JS	Python	Go	Rust
Tipagem estática	Sim	Sim	Não	Opcional	Sim	Sim
Inferência de tipo	Não	Parc.	—	Parcial	Parc.	Sim
Garbage Collector	Sim	Sim	Sim	Sim	Sim	Não
Compilada	Sim	Sim	JIT	Interp.	Sim	Sim
Multithreading	Não	Sim	Não	Limit.	Sim	Sim
Genéricos	Não	Sim	—	Parc.	Sim	Sim
Herança	Não	Sim	Sim	Sim	Não	Não
Tratamento erro	Abort	Excep.	Excep.	Excep.	Valor	Result
Ecossistema bibl.	—	Vasto	Vasto	Vasto	Vasto	Cresc.

Apêndice J · Roadmap e Histórico de Versões

Visão geral das mudanças entre versões e direções planejadas para futuro.

J.1 Histórico de Versões

Versão 1.0.0 (atual) — Janeiro 2026

- Sintaxe básica: variáveis, constantes, funções, classes, componentes.
- Tipos primitivos: int, float, str, boolean, array.
- Operadores aritméticos, relacionais e lógicos.

- Controle de fluxo: if/else, while, for.
- Compilação para bytecode UVLM (formato GLP).
- CLI com comandos: build, seal, unseal, key-show, key-export, key-import.
- ZettaSource IDE v2.0.0 com syntax highlighting e atalhos.

J.2 Direções Possíveis para Versões Futuras

> **ATENÇÃO**
> Esta seção é especulativa. Reflete necessidades comuns identificadas em uso real, mas não constitui compromisso oficial. Versões futuras podem priorizar features diferentes.

J.2.1 Planejado para 1.x

- Operadores de atribuição compostos: += -= *= /=.
- Operadores de incremento/decremento: ++ e --.
- Operador unário ! (negação booleana).
- Operador ternário condicional.
- break e continue em loops.
- Operador de tamanho de array (algo como arr.size ou len(arr)).
- Funções nativas básicas para strings (length, substring, indexOf).
- Função de parsing string → número.
- Random e tempo do sistema.

J.2.2 Considerado para 2.x

- Sistema de imports cross-arquivo (impacta compatibilidade).
- Construtores em classes.
- Interfaces ou traits.
- Visibilidade granular (private, protected).
- Genéricos (parametrização de tipos).
- Try/catch para tratamento de erros.
- Funções de primeira classe (callbacks, closures).
- Switch/match expressivo.
- Biblioteca padrão expandida (Math, Strings, Collections).

J.2.3 Possível para 3.x

- Multithreading básico.
- JIT na UVLM (otimização em runtime).
- Sistema de pacotes com gerenciamento de dependências.

- FFI (foreign function interface) para integração com Java/JVM.
- I/O de arquivo e rede.
- Suporte oficial a IDEs alternativas (VS Code, IntelliJ).

J.3 Compromissos de Compatibilidade

- Programas escritos em UZ 1.0 devem continuar compilando em todas as versões 1.x.
- Bytecode .uzb gerado em UZ 1.x deve executar em UVLMs 1.y onde y >= x.
- Migrações entre 1.x → 2.x podem requerer ajustes; ferramentas de migração serão fornecidas.

Apêndice K · Tipos em Profundidade

Detalhes técnicos sobre como os tipos são representados internamente pela UVLM. Útil para depurar problemas de overflow, precisão ou comportamento inesperado.

K.1 Inteiros (int)

K.1.1 Representação Binária

int em UpperZetta é armazenado como inteiro de 32 bits com sinal, usando complemento de dois. Isso significa:

- Bit mais significativo (bit 31) indica sinal: 0 = positivo, 1 = negativo.
- Bits 0-30 representam a magnitude.
- Total: 32 bits ocupados na memória.

K.1.2 Faixa Real

Limite	Valor decimal	Hexadecimal
Mínimo	-2.147.483.648	0x80000000
Máximo	2.147.483.647	0x7FFFFFFF
Zero	0	0x00000000
-1	-1	0xFFFFFFFF

K.1.3 Comportamento de Overflow

Quando uma operação ultrapassa os limites, ocorre wraparound (envolvimento) silencioso:

```
let max.int = 2147483647;
let resultado.int = max + 1;
System.print(resultado);   // -2147483648 (não é erro!)

let min.int = -2147483648;
let resultado2.int = min - 1;
System.print(resultado2);  // 2147483647
```

ATENÇÃO
Overflow não é detectado nem reportado em UZ v1.0. Para cálculos próximos dos limites, considere usar float para maior amplitude (com perda de precisão exata).

K.2 Floats

K.2.1 Padrão IEEE 754 Dupla Precisão

float em UZ corresponde a double-precision IEEE 754 (64 bits):

- 1 bit de sinal.
- 11 bits de expoente.
- 52 bits de mantissa (significando).

K.2.2 Faixa e Precisão

Característica	Valor
Mínimo positivo	~5.0×10^{-324}
Máximo	~1.7×10^{308}
Precisão	~15-17 dígitos decimais
Epsilon	~2.22×10^{-16}

K.2.3 Valores Especiais

IEEE 754 inclui valores especiais que podem aparecer:

- **+Infinity / -Infinity:** resultado de divisão por zero ou overflow em float.
- **NaN (Not a Number):** resultado de operações inválidas como 0.0/0.0 ou sqrt(-1).

K.2.4 Cuidados com Comparação

ATENÇÃO
NUNCA compare floats com == para verificar igualdade exata. Use tolerância:

```
// ❌ Pode falhar por erro de arredondamento
```

```
if (x == 0.1) { ... }

// ☑ Comparação com tolerância
e.const EPSILON.float = 0.000001;

fun aproxIgual(a.float, b.float) >> boolean {
    let diff.float = a - b;
    if (diff < 0.0) { diff = -diff; }
    return diff < EPSILON;
}

if (aproxIgual(x, 0.1)) { ... }
```

K.3 Strings (str)

K.3.1 Encoding

Strings em UpperZetta são armazenadas como UTF-8 internamente. Suportam todo o conjunto Unicode:

```
let portugues.str = "Olá, ção, ñ";
let chines.str = "你好";
let arabe.str = "مرحبا";
let emoji.str = "😀";

System.print(portugues);
System.print(chines);
System.print(arabe);
System.print(emoji);
```

K.3.2 Imutabilidade

Strings são imutáveis. Operações que parecem modificar uma string (como concatenação) na verdade criam strings novas:

```
let s1.str = "Olá";
let s2.str = s1 + ", mundo!";
// s1 ainda é "Olá"
// s2 é "Olá, mundo!"
```

K.3.3 Comparação Lexicográfica

Strings são comparadas lexicograficamente (ordem de Unicode code points):

```
System.print("a" < "b");          // true
System.print("abc" < "abd");      // true
System.print("Z" < "a");          // true (Z=90, a=97)
```

K.4 Booleanos

Internamente representado como int de 32 bits, mas conceitualmente apenas dois valores: true (1) e false (0).

> **ATENÇÃO**
> Em UZ, ints e booleanos NÃO são intercambiáveis: if (1) { ... } causa erro de tipo. Sempre use comparações explícitas.

K.5 Arrays

K.5.1 Representação Interna

Arrays são objetos alocados em heap. A variável "array" guarda apenas uma referência (ponteiro):

```
let a.array = [1, 2, 3];
let b.array = a;     // b e a apontam para o MESMO array
b[0] = 99;
System.print(a[0]);  // 99 (a foi afetado)
```

K.5.2 Cópia Profunda vs. Superficial

Para criar uma cópia independente, use uma função de cópia explícita:

```
fun copiar(origem.array, destino.array, n.int) {
    for (let i.int = 0; i < n; i = i + 1) {
        destino[i] = origem[i];
    }
}
```

K.6 Objetos (Instâncias de Classes)

Como arrays, objetos são alocados em heap e variáveis guardam referências. Atribuição copia a referência, não o objeto:

```
let a.Pessoa = new Pessoa();
a.nome = "Ana";

let b.Pessoa = a;    // b aponta para o MESMO objeto
b.nome = "Bruno";

System.print(a.nome);   // "Bruno" (mudou!)
```

Apêndice L · Organização de Projetos

Recomendações para estruturar projetos UpperZetta de qualquer porte. Foca em práticas que sobrevivem ao crescimento do projeto e facilitam manutenção a longo prazo.

L.1 Estrutura para Projeto Pequeno

Para scripts e utilitários simples (< 500 linhas), uma estrutura mínima é suficiente:

```
projeto-pequeno/
├── main.uz
├── README.md
└── .gitignore
```

L.2 Estrutura para Projeto Médio

Para projetos com múltiplas responsabilidades (~500-2000 linhas):

```
projeto-medio/
├── src/
│   └── main.uz             # arquivo principal (componente Home)
├── docs/
│   ├── README.md
│   ├── ARQUITETURA.md
│   └── EXEMPLOS.md
├── exemplos/
│   ├── exemplo1.uz
│   └── exemplo2.uz
├── testes/
│   └── runner.uz
├── build/                   # gerado, não versionado
│   └── main.uzb
├── dist/                    # releases
│   └── main-v1.0.uzs
├── scripts/
│   ├── build.sh
│   └── release.sh
├── CHANGELOG.md
├── LICENSE
└── .gitignore
```

L.3 Estrutura para Projeto Grande

Para sistemas complexos onde o arquivo único deixa de fazer sentido (estratégia de "pré-processamento"):

```
projeto-grande/
├── src/
│   ├── modelos/
│   │   ├── usuario.uz       # apenas a class.public Usuario
│   │   ├── pedido.uz        # apenas a class.public Pedido
│   │   └── produto.uz
│   ├── servicos/
│   │   ├── auth.uz          # funções globais de autenticação
│   │   ├── pagamento.uz
│   │   └── relatorio.uz
│   ├── util/
│   │   ├── strings.uz
│   │   ├── numeros.uz
│   │   └── matematica.uz
│   └── main.uz              # componente Home + concatenação
├── scripts/
│   ├── concatenar.sh        # junta todos os .uz em build/main.uz
│   └── build.sh             # concatena + compila
├── docs/
├── testes/
├── build/
└── dist/
```

ATENÇÃO
Como UZ v1.0 não suporta imports, projetos multi-arquivo dependem de pré-processamento (concatenação ou copy-paste). Versões futuras podem oferecer imports nativos.

L.4 Script de Concatenação

Para projetos que separam arquivos por preocupação:

scripts/concatenar.sh

```
#!/bin/bash
set -e

OUT="build/main.uz"
mkdir -p build

# Cabeçalho
cat > "$OUT" << EOF
// AUTO-GERADO em $(date) — não edite manualmente
package in app.principal;

EOF

# Constantes globais
```

```
cat src/constantes.uz >> "$OUT"

# Modelos
for arquivo in src/modelos/*.uz; do
    echo "// === $arquivo ===" >> "$OUT"
    # Remove "package in" duplicados
    grep -v "^package in" "$arquivo" >> "$OUT"
done

# Serviços
for arquivo in src/servicos/*.uz; do
    grep -v "^package in" "$arquivo" >> "$OUT"
done

# Componente Home (deve ser o último)
grep -v "^package in" src/main.uz >> "$OUT"

echo "Concatenado em: $OUT"
```

L.5 README Recomendado

Todo projeto deve ter um README.md com no mínimo:

72. Nome e descrição curta do projeto.
73. Como executar (incluindo versão da UVLM requerida).
74. Exemplo de uso ou screenshot.
75. Como contribuir (se for projeto aberto).
76. Licença.

L.6 .gitignore Recomendado

.gitignore

```
# Bytecode gerado
build/
*.uzb

# Arquivos selados não devem ser versionados
*.uzs

# DEVICE KEY pessoal
.uvlm/

# Editores
.vscode/
.idea/
*.swp
```

```
.DS_Store

# Logs
*.log

# Mas mantenha ZettaSource IDE configs do projeto se desejar
!.zettasource/
```

L.7 Documentação Inline

Para projetos compartilhados, documente cada classe e função pública:

```
/*
 * Classe: Pedido
 * Representa um pedido de compra com itens, total e status.
 *
 * Estados possíveis: "rascunho", "submetido", "aprovado", "cancelado"
 *
 * Uso típico:
 *   let p.Pedido = criarPedido(1001);
 *   p.adicionarItem(produto);
 *   p.submeter();
 */
class.public Pedido {
    /* ... */
}
```

L.8 Convenções de Commit

Para projetos versionados em Git, adote padrão consistente como Conventional Commits:

```
feat: adiciona suporte a desconto progressivo
fix: corrige overflow em cálculo de juros
docs: atualiza exemplo de uso da classe Pedido
refactor: extrai validação para função separada
test: adiciona testes de borda para Conta
chore: atualiza dependência da UVLM
```

L.9 Quando Dividir Arquivos

Use estas heurísticas para decidir se vale separar:

- Arquivo passou de 500 linhas: considere dividir por responsabilidade.
- Mais de 5 classes no mesmo arquivo: provavelmente cada uma merece seu próprio.
- Você precisa rolar muito para encontrar onde está editando: divida.
- Múltiplas pessoas editam o mesmo arquivo simultaneamente: divisão reduz conflitos.

L.10 Checklist de Saúde do Projeto

77. README atualizado com instruções claras.
78. Versão atual documentada (CHANGELOG ou tag).
79. Testes existem e passam.
80. Compilação limpa sem warnings.
81. Código formatado consistentemente.
82. Sem código comentado/morto sem justificativa.
83. Dependências externas (versão da UVLM) explícitas.
84. Backup das DEVICE KEYs envolvidas em arquivos selados.